Rogue Like Game analysis

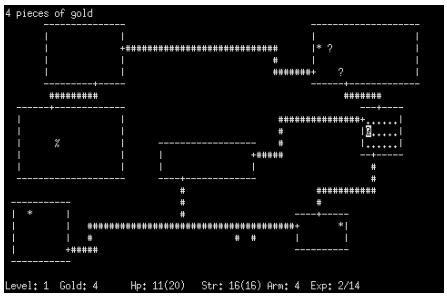
Table of Contents

Analysis

Background

History of Rogue

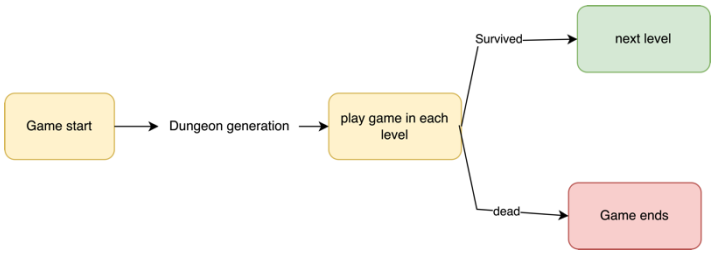
A Rogue-like game is a well-known genre of role-playing game. In 1980, a game called “Rogue”, which is an ASCII based game that runs in the computer terminal, the graphic of



the game is fully represented by characters, for example, player is represented by a “@”, this game started a entire new genera called “Rogue Like”.

During the game play, player will explore and adventure in a dungeon, which is randomly generated, with items and enemy also randomly generate**(Requirement G1)** in each room, the game terminates as player dies, and next run through will be entirely different because everything is entirely randomly generated.

The most interest part of this game was about its procedural generation of the map, making every game play different, player cannot expect what will happen next until he enter the next room. (**Requirement G1 a)**



Process Diagram of Game Rogue

Modern Rogue like game

Here comes the essential part and the spirit of “Rogue-Like” game, the randomicity. The game Rogue had algorithm that randomly generates the level of dungeon, in fact, every play through is unique to the user, and the feeling of not being able to predict what’s next gives thrill and keeping interest from players, by providing reward to player’s action. For example, as player progress to harder levels, they are more likely to find more powerful weapons, higher levels of chest.

Another common feature of this type of game, is they often has a huge choice of item **(Requirement G6)** you can use, including weapons, ability, potion etc. They are very different in terms of the game effect in most of the case. For example, one of the famous Rogue-like game on mobile phone “Soul Knight”, has gigantic number of weapons in different levels, which will randomly generate in the chest for player, and they are never the same. Some of them can shoot multiple bullets, or freeze enemy, even bounce off the wall some time. Compare to traditional game which involves a variety or upgradeable weapons, rogue like game provide a greater functionality for weapons or skills system, modern game like “Cyber punk”, “resident evil” also

has a good weapon system, but more focus on the stats of the weapons, for example, damage, critical hit rate, whereas in roguelike game player can have their "style" when playing **(Requirement B3, Requirement G6)**, because the strong degree of freedom on combination of item, so that player can organize their equipment in the way they like and have a joint effect.

A combination of these features has the effect of "1 + 1 > 2", large number of different weapons, skills set, variety of enemies, with the randomicity in terms of generation method of the game, making each of game play (each round) unique. The difficulty of the game is increase proportionally with time, and player is getting stronger simultaneously, potentially, player could find a weapon or skill that can overkill at this stage of game; therefore, player would be constantly looking for powerful weapons, this is the trick the game use to make players always start a new game if they die, just like the way gambling makes people thrill, player will believe next play through could be very good, or even better.

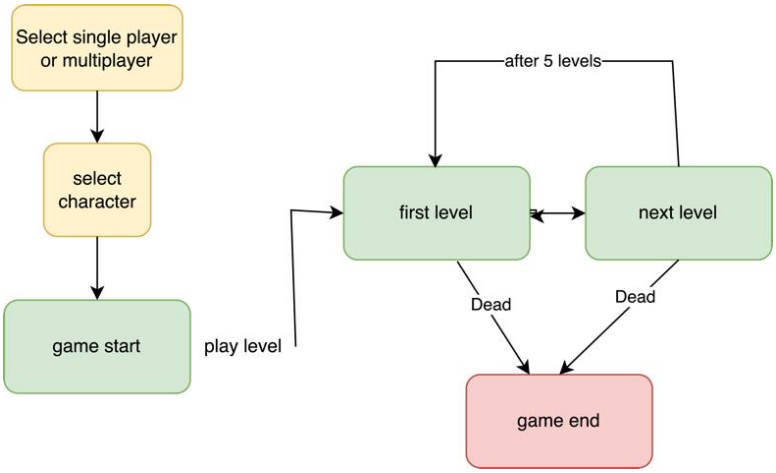
Existing system

There are many outstanding examples of this type of game out there, and I will use some examples particularly as an example.

Risk Of Rain

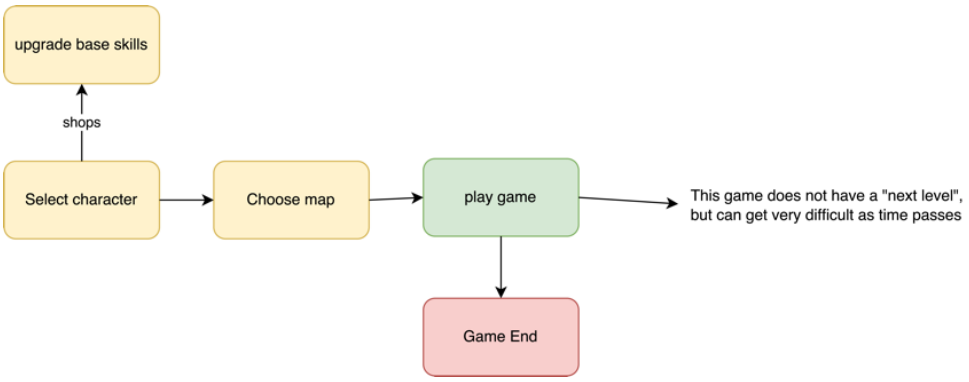
First, "Risk of rain" is the roguelike game that I enjoy the most, it is a single player/multiplayer game, in 3 dimensions. You can select characters which each of them has different abilities, and there are 5 levels in each go of the game, they can repeat after you completed them. Although it may sound you can easily get board of, but the fun part is the item you can find in the chest **(Requirement G1 b)**, and you can have duplicate of the same item, the effect of it is combined, and there is great many of them. Furthermore, the difficulty of the game increase as you survive longer. This game stands out by its abundant content in each game play and different events, and its multiplayer mode.

Procedure diagram for Risk of rain 2



Vampire Survivor

I'm massively inspired by this game, it has the essential part of roguelike game, randomicity during game flow, gigantic weapon system, and increasing difficulty with time. This game gives player an intuitional feeling of fun, and easy game control, literally all you have to do is to move around, and choose item as you level up, the attack is done automatically. It does sound boring to spend hours just to move around, but the interesting part of this game is the weapon system, combination of item can have a "combo effect" on the attack, called weapon evolution in game. For example, "Axe + Candelabrador" = Death Spiral, this shoots out 9 spinning scythes around the player. Each scythe has extremely high pierce and are bigger, with damage reduction to compensate, a combination of evaluated item can have even more effects, player has a variety equipment, and so that makes of choices to choose their each game play different.



Procedural diagram for vampire survivor

Problem statement

Basic Content included in game

I will be developing a two-dimensional rogue like game; the game logic is similar to the previous example. First of all, when game is launch, user will be able to choose from a list of characters, they will all be designed graphically **(Requirement B1, B2**), which will include buttons, images user can click and interact with. Each character will have a different appearance, and will initially have a skill, which could define the ways they attack, for example, character could be either throwing fireballs, or using a sword. Then player will be able to choose a map, the map does not affect the actual generation logic of physical item in the map, for example, walls or room sizes, but it will have a different theme, which includes the background image, and also the category of enemy being generated, for example, a forest map will likely to generate tree monster, whereas a mountain cave map will likely to be generating rock monster as game proceed **(Requirement B3)**. There will rooms player can enter with in the map, and chest can be found there, player will be able to find items that is randomly generated by the game, which will give an effect on player, for example, increase 20% of speed, increase 10% damage per hit etc. (**Requirement G6)** Of course, enemy will be generated in the game,

player can kill them and use the money to open the chest in the rooms. During the game play, there will be a timer that record how much time has passed since the game has started.

Logics and rules in game

The purpose from player is to simply survive as long as possible, and kill as much enemy as they can, these are two factors that is considered in the score calculation at the end of each game play. Like most of the rogue like game, difficulty is proportional to time passed, the enemy is going to be stronger, meanwhile, the chances of player discover a stronger also increases, to maintain the balance of game, between being too hard and too easy. To make each game play different, the physical structure of the map (rooms and doors etc) will be randomly generated in each game play.

For the chest system in game, it will be act like a shop, player can earn coins from enemy, and use it to open chest to get items.

The game will be based on mono game libraries in c sharp, so the graphics of game will be smooth with high frame per second, and there will be two-dimensional motion for player, attacks enemy etc, rather than predesigned images on each state of the game.

User identification

For non- gamers

This game will be friendly to people who don't often play games, because the controls of the game is designed to be simple and intuitive, you will use "wasd" on keyboard to move, and simply press anywhere on screen to attack in that direction, any other state in the game such as choosing item, manage bags, choosing map will have buttons assigned to it, so it will be straightforward to play without any knowledge of the game.

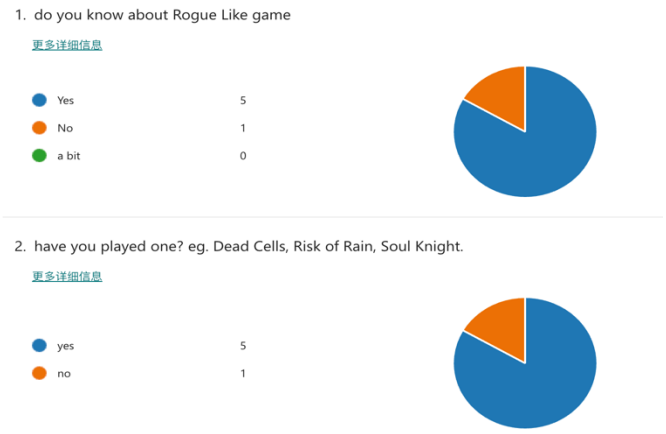
For gamers

If you are a gamer, especially interested in rogue like game, then this will be a very interesting project to you. People with gaming experiences can learn the controls of the game much quicker, and focus on earning high score every game play. This project offers a variety of option in game, you can pick characters, choose maps, and use a combination of item in game, to fight against enemies. Moreover, as game proceed to later stage, there is random event can happen, which offers more possibility to player. Overall, there is no prior knowledge about gaming or rogue like game is required to play, and it will be enjoyable to either groups people.

Interview with user

I made a Microsoft form to student about this Rogue-Like game design, and I received 6 responses.

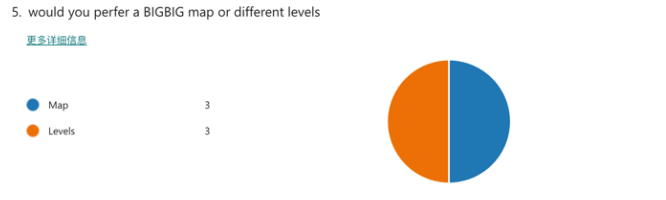
1. First, most people know or tried a Rogue-like game before. There are 83% of the participants have played one and provided their favorite ones.



2. In the favorite game provided by the responses, they are all different. But all of them are the popular ones in recent years, especially Hollow knight, hades. Not all of them have amazing graphic or design, but they are all very entertaining, and consist great game mechanics.

hades
Hollow knight
Soul Knight
Enter the gungeon / Darkest dungeon
Risk of Rain

3.Interestingly, in the perspective of game design, a half of people prefer a continuous or a big map for them to play, whereas the other half prefer game divided into different levels. In this case, I would take the advantage of both of them, I will make 3 different maps instead of 3 levels, and they will all be randomly generating big map**. (Requirement B3 a)**



4. player seems to interest more on using both keyboard and mouse to play. **(Requirement P2, a, Requirement P1 a.)**

Overall, the purpose of this interview is to get information about the familiarity of the category Rogue-like game for the group of interviewees, and ensure it is appropriate to develop a game like this, moreover, to recognize a pattern of interest from players for my future design.

Solution requirement

Objectives

Before Game Start

Requirement B1. User interface

a. The menu should be a button-controlled system, player can use mouse to simply click on buttons to make their choices.
b. There should be some UI design, including background image, titles, icons, that shows the correct message like “start”, “choose.”
c. The menu should be able to conduct page changes, when player has made choices, it should refresh the page and display the next page for player to choose.
d. The pages should include:
   1. welcome page
   2. map choosing page.
   3.character choosing page.

Requirement B2. Options available to select from the menu.

a. Player should be able to choose the character they want to play.
b. Player should be able to choose the map they want to play in.
c. There should be a start button that starts the whole game process.

Requirement B3. Content that is represented by the choice on the screen.

a. Maps
   1. Forest: a map with forest background, and high potential of generating forest typed enemy (see later context)
   2. Cave: a map with Mountain cave background, and high potential of generating rock typed enemy (see later context)
   3. Town: a map with city town background, and high potential of generating human typed enemy (see later context)
b. Characters:

1. Fireball hero: a character use long range attack mode, throws fireball, his attack can go through walls
2. Sword hero: a character shoots blade effect out, and can teleport to where the last blade thrown.
3. Gun hero: a character use long range attack mode, using a snipper gun which has very high damage.

After Game start

From side of player

Requirement P1. Motion Control of player

a. player can move around the map using keys “w, a, s, d” to control where they want to move the character.
b. player should be able to move.
    1. horizontally.
    2. Vertically
c. player’s speed should be able to change during game play.

Requirement P2. Attack Control of player

a. player should be able to use mouse left click to perform attack.
b. The position they click on the screen is the direction that the attack is directed to.
c. The amount of attack player can perform per second can be changed, player is not allowed to throw infinite amount of attack at a time, the APS (attack per second) is set to a value but can be changed.
d. The attack should contains values for its damage, and flying speed.

Requirement P3. Player’s property:

a. player should have the following property:
    1. health, if health decrease to 0 game will end.
    2. Speed, how fast player can move
    3. The money the player currently has

From the side of enemy

Requirement E1. Enemy motion control

a. enemy should be able to move:
    1. horizontally
    2. vertically
    3. diagonally

b. There should be an algorithm that is responsible to help enemy to track player, and direct enemy to move to player, this algorithm should be able to direct enemy to go around walls, or through rooms.

1. enemy should be able to get to player's position
2. enemy should be able to calculate its path and avoid hitting walls.
3. each individual enemy should have its own path calculation.

Requirement E2. Enemy attack control.

a. General enemy can cause damage to player if they move to player and hit player, precisely when they intersect with player.
b. Enemy which are specialize to a map are typically stronger, and they are able to conduct long range attack, which means they are able to throw attack to shoot player just like player.

Requirement E3. Enemy Property

a. Enemy should have a value represent its health.
b. Enemy should have a value for its speed.
c. Enemy should have an image for its appearance.
d. Enemy should have a value represent the money they drop after player killed them

Requirement E4. Different enemies

a. There should be 10 different types of general enemy, 6 of them specifically relate to specific map, they will all have different property (see requirement E3)

From the Side of Game

Requirement G1. Map generation.

a. The Game should generate Rooms in the map, a room has a following property.
1. It should be a rectangle space.
2. It should have a space recognize as a floor.
3. It should have 4 walls around it.
4. It should have a door player or enemy can enter.
5. The position of the room should be random.
6. The size of the room should be random, this means the width, and height of the room
7. The position of the door can be on any side on a wall but there is only one for each roomTheThe number of rooms generated in each game run should be random.

b. chests

1. map should also generate chest that is place in each room.

2. The item in the chest should be random.
3. The chest should display the name of item and the price for it on top of it.
4. When player successfully purchased an item it should display this item on top of the screen.

Requirement G2. Attack detection.

a. The game should be able to detect if an attack was made and hit from both player or enemy, and do right adjustment on the subject' health.

b. the game should remove enemy from the game if it is killed.

c. Money is given to player after they kill an enemy

Requirement G3. Display

a. The game should have a camera system, to center the viewport on player, as player move through a big map.

b. as player moves, the camera system should be constantly updated according to player's position, which keeps the player in the center of the screen.

1. camera should also update sensibly, when player is approaching the edge of the screen, the camera should stop updating to avoid showing the outside of the map itself.

C. There should be some information display during the game play, this includes

1. player's health, showed as a bar on the top left corner of the screen, the length of it represents player's health.
2. Timer, represent total time passed in the game placed at top of the screen.
3. The money player owns
4. The item player owns.

Requirement G4. Enemy generation

a. The game should be able to generate enemy in an interval of time, for example, generate enemy per 5 seconds, and this value should be editable.

b. The enemy generation should be random, not generating same types of enemies all the time.

c. the enemy generation should be more intense after game time passes, more enemy should be generated.

Requirement G5. Score Board

a. The game should display the following info of the game at the end of each game play:

1. The enemy player has killed
2. The damage player has caused.
3. The total time player has survived

Requirement G6. Items

a) The game should have the following item player can pick from:
   a. power runes, increase damage by 4
   b. flying boots, increase moving speed by 1
   c. gun powder, increase attack flying speed by 1
   d. speed gear, increase attacking speed by 0.05 seconds, that means you time difference between each attack is 0.05 seconds less.
   e. enlarge potion, attack will appear to be bigger on the screen, every time the side will increase by 10 pixels.
   f. ancient ice, froze the motion of all enemy by 5 seconds when this item is picked up.
   g. Demon force, this will remove all of the enemy on screen now.

b) Player is allowed to have multiple of same item; the effect of item will simply add up.

Requirement G7. Difficulties

a. game should be able to increase level of difficulties according to time passed.

1. enemy will have higher health.
2. enemy will have higher damage.
3. enemy should drop more money to player.

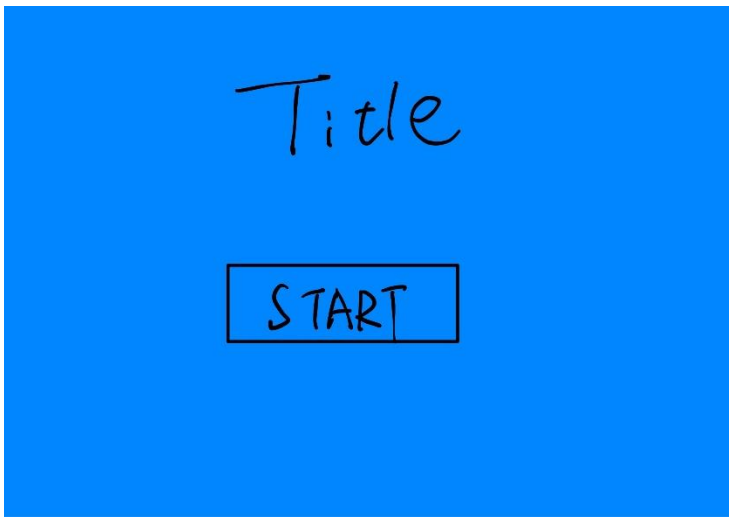
Limitations of proposed solution

1. The art in game s won't be designed in a commercial level, as this is not the focus of the game.
2. The Game won't have a massive number of different types of enemy and items.
3. The Game won't support online mode, or multiplayer control on same computer.
4. The Game won't support settings to change graphics in game.

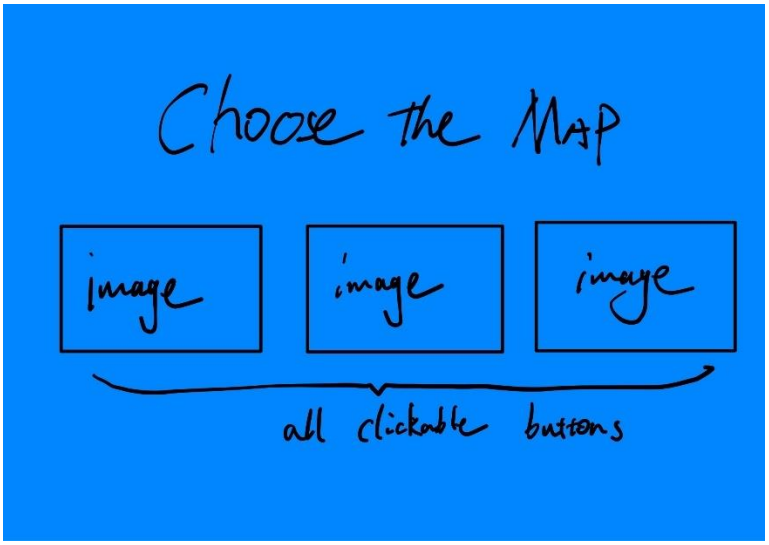
Modelling and prototypes

Menu System

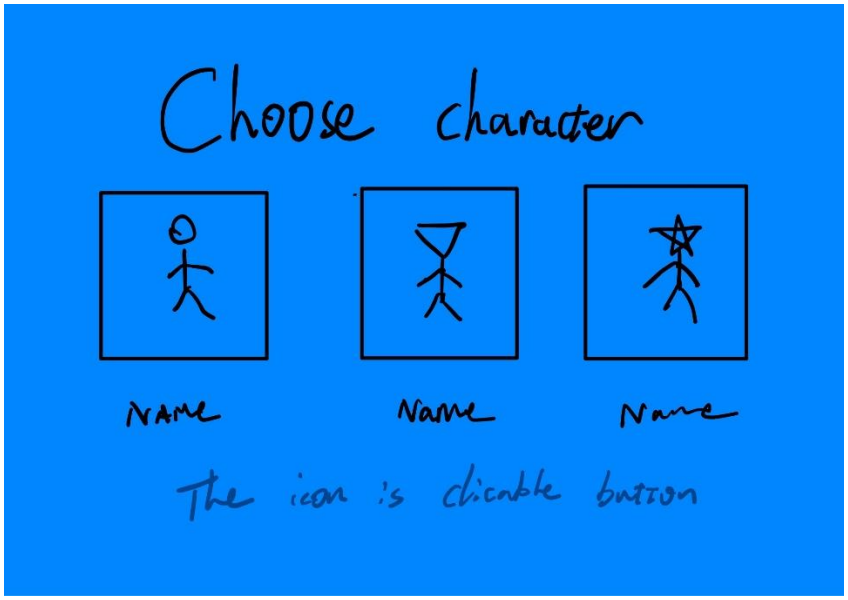
Sketch of the welcome page of menu



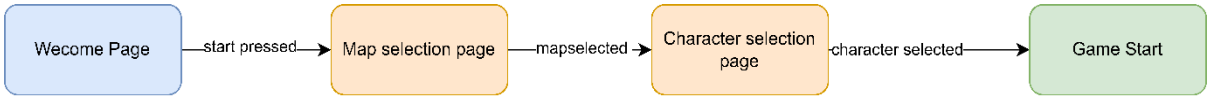
Sketch of the map choosing stage of menu



Sketch of the character choosing stage of the menu



Menu Proccesing flow chart



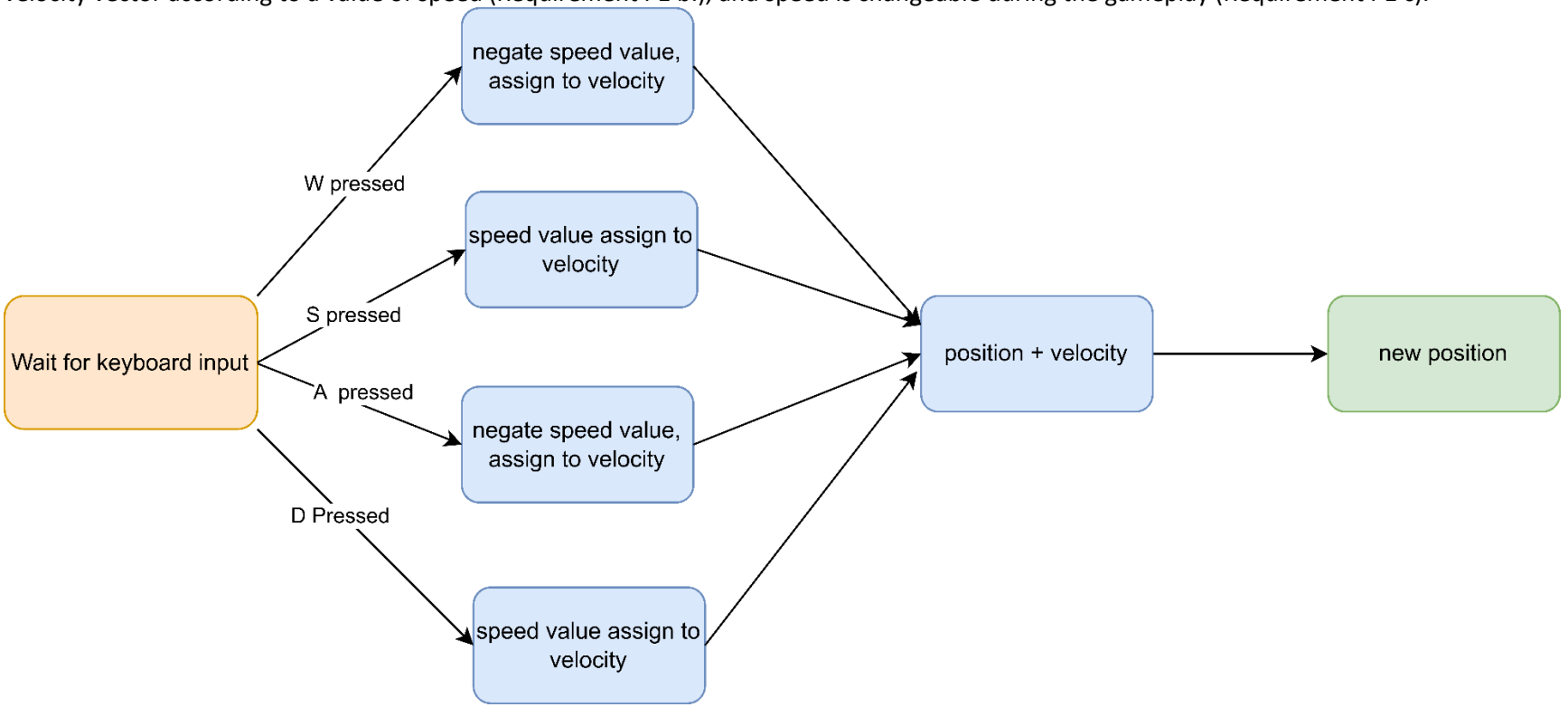
Player motion control

```
public Vector2 position;
public Vector2 velocity;
private float speed = 5.7f;

if (control.IsKeyDown(Keys.W) && !(position.Y < 0))
            {
                velocity.Y = -speed;
            }
            else if (control.IsKeyDown(Keys.S) && !(position.Y >= screenedge.Y - hero.Height))
            {
                velocity.Y = speed;
            }
            else
            {
                velocity.Y = 0;
            }
            //move for A and D
            if (control.IsKeyDown(Keys.A) && !(position.X <= 0))
            {
                velocity.X = -speed;
            }
            else if (control.IsKeyDown(Keys.D) && !(position.X >= screenedge.X - hero.Width))
            {
                velocity.X = speed;
            }
            else
            {
                velocity.X = 0;

public void Update(velocity)//player update
        {
            position += velocity;
        }
```

This is the code for motion control by keyboard for moving character on screen (Requirement P1 a.), it allows player to move by constantly changing the velocity vector according to a value of speed (Requirement P1 b.), and speed is changeable during the gameplay (Requirement P1 c).



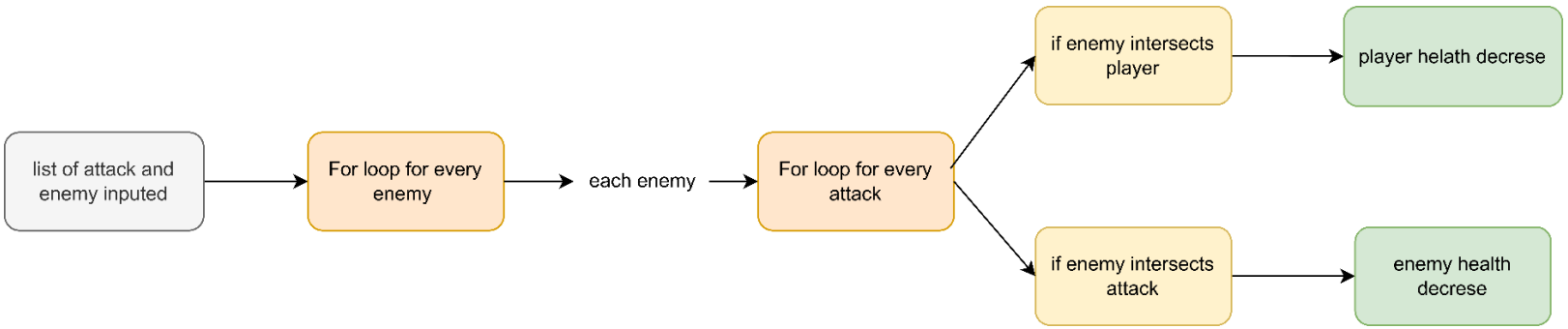
Player motion Control Flow chart

Checking if an attack was made

```
public void attackhitcontrol(List<enemy> enemies,List<attack> attacks)//collision detection, check for any damage
        {
            foreach (enemy e in enemies)
            {
                for(int i =0; i<attacks.Count; i++)//check attack hit enemy
                {
                    if (attacks[i].fireballhitbox.Intersects(e.Flyingpurplehitbox))
                    {
                        e.health -= attacks[i].Attackdamage;
                        attacks.RemoveAt(i);
                    }
                }
                if(e.Flyingpurplehitbox.Intersects(player.playerhitbox)&& temptimer_damage >= 0.1)//check enemy hit hero(also
limit the speed of attack, not decreasing health every frame)
                {
                    player.Health -= e.damage;
                    temptimer_damage = 0;
                }
            }
        }
```

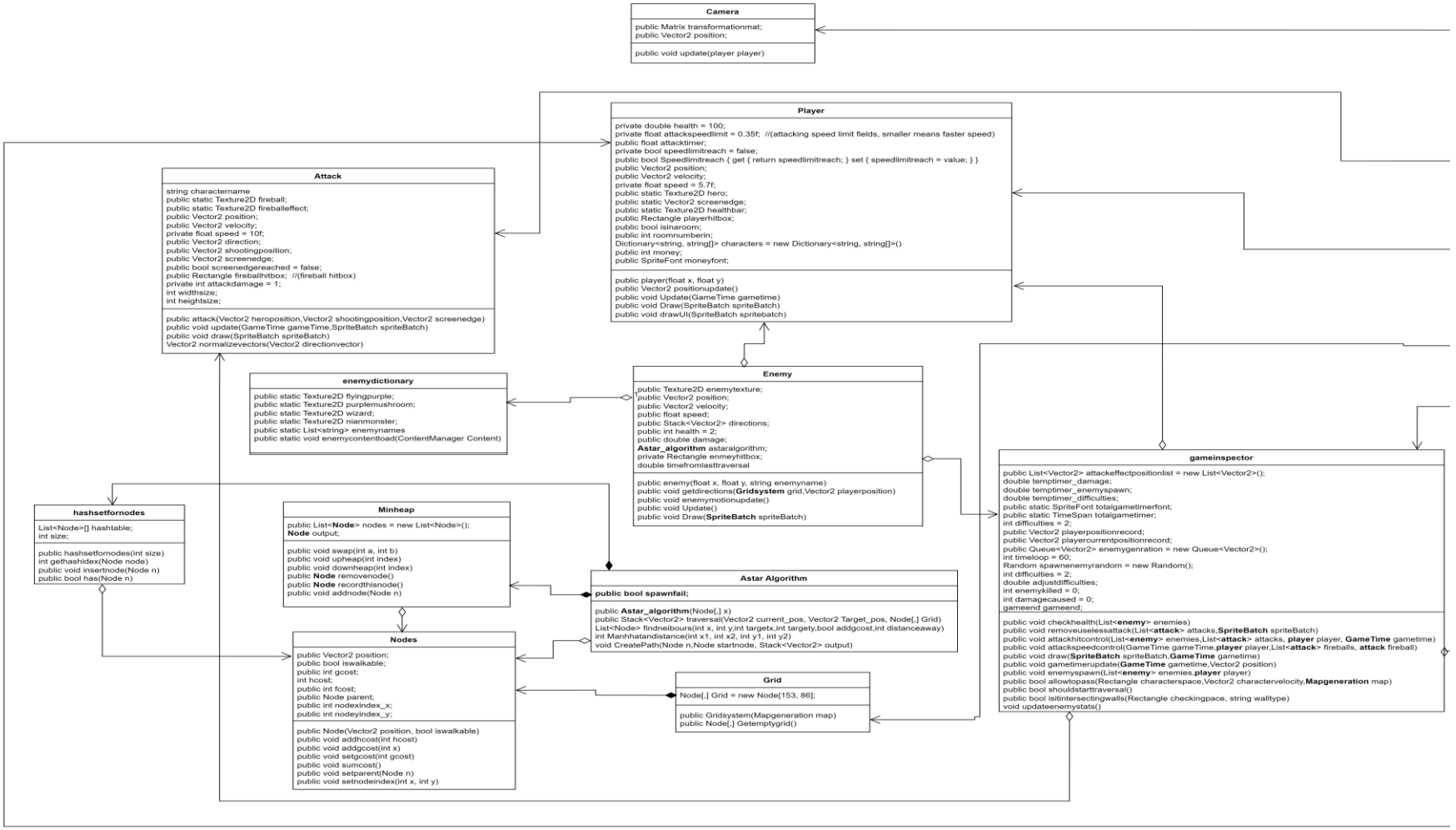
This is a prototype of designing the collision detection system for both hero and enemy (Require, there is an list of attack and list of enemies, as an argument feed into this function, the hitbox is and rectangle in game that represent an area that can be hit by an attack in game, and by constantly checking the intersection of any hit box, to determine an attack is hit on a target, if an attack hit this attack is removed (Requirement G2 b)

Attack detection flow chart

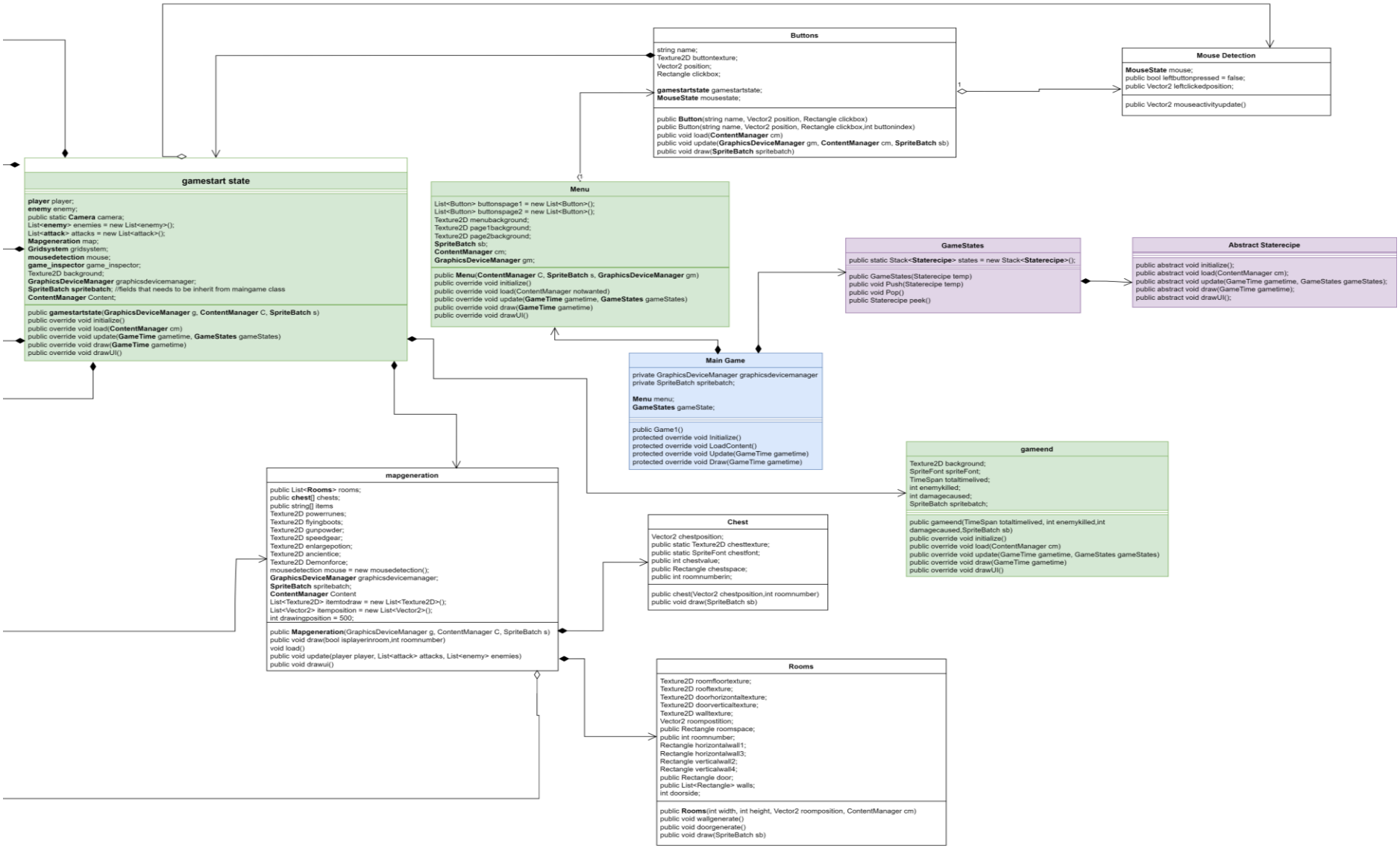


Design

Main Class UML Diagram



UML diagram for classes(figure1)



Analysis of the diagram above

The diagram(figure1) displays all the classes in the system, and their relationship with each other. The Blue part is the main program, where the game start from, the purple part shows the classes which are responsible of managing the flow between the states of the game, and the green parts are the states that is in the system.

The arrow with filled diamond pointing towards it shows a composition between two classes, and a diamond with empty space between shows aggregation between classes.

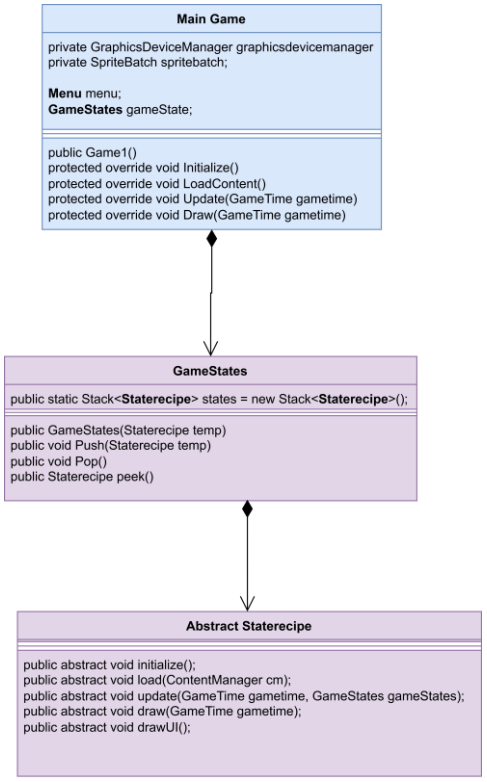
In the diagram each table is divided into 2 sections, on the top part shows the fields that are used in the diagram, and bottom part shows the method that is in the class, all the objects that is used in either section has been marked as bold text.

System description

State changes during game.

This part is mainly focused on Requirement B1. C, I will illustrate how does my system manage the changing state of the game.

UML class diagram break done

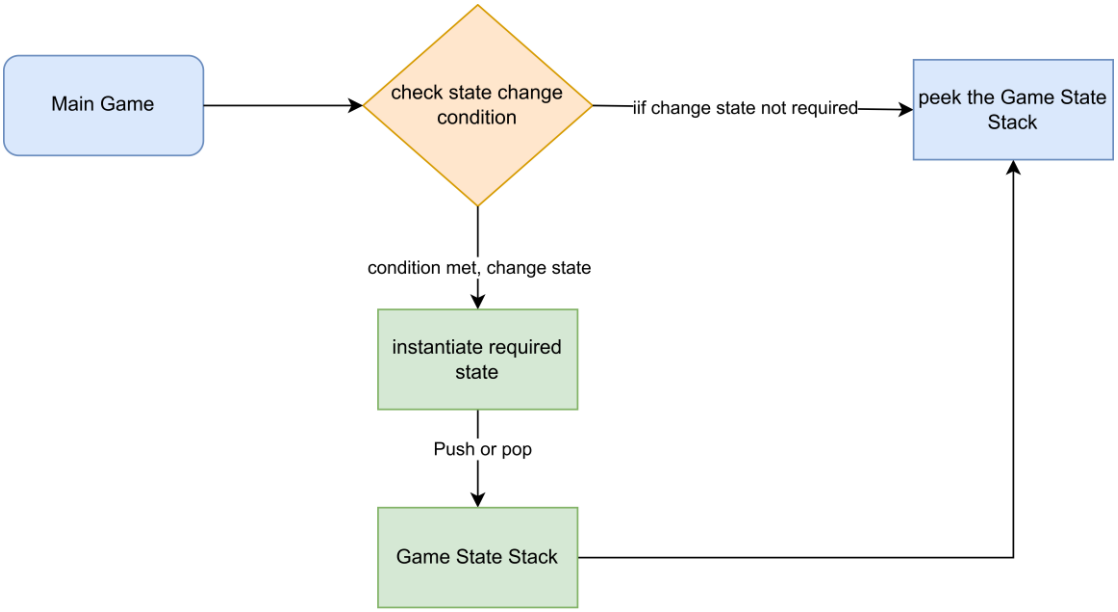


This is the classes that is responsible of managing states in the game flow, Main Game class is the Game state class first instantiated, and the State recipe is instantiated within the Game State class.

GameStates: Contains a public static stack of **staterecipe** classes, stack is used here, because whenever the game state changes when condition met, for example, a button is pressed or player is dead, an according state is pushed into this stack, so the **Main game** class can peek the stack, to get the state that is currently running, and run the content that is in this state. The stack is set to be public static so that we can access this stack at anywhere in game, so we don't have to worry instantiate a new **Gamestate** class in other part of the game.

Staterecipe: this is an abstract class that contains the method initialize (), load(), update(),draw(), drawUI(). An abstract class is chosen and included with theses method because it matches the format of monogame module in c sharp, the **Main Game** is run in this way so in every state, it should be in this format, it acts like a recipe for all the states, tells that every game state should be in this format, and when coding for states I can just simply overrides the methods in the abstract class, and push the state into the stack, so game can perform change of states.

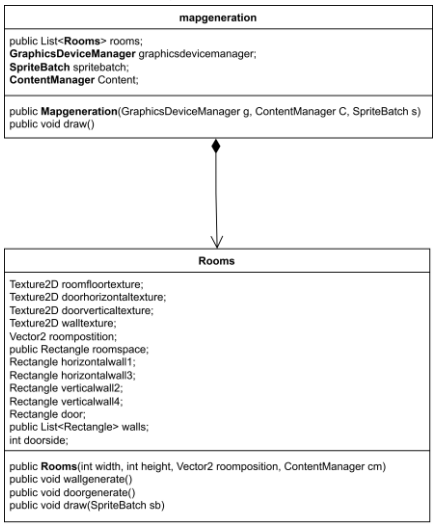
Flowchart for state changing system



Map Generation

This section is mainly focused on all sections in Requirement G1, which provide a system that generates physical object in the map, which are rooms.

UML class diagram break done



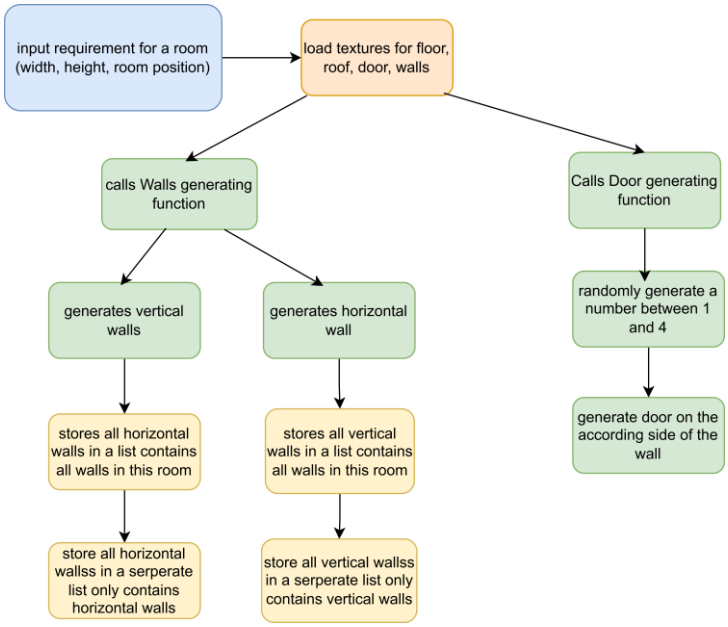
These two classes are responsible for generating rooms in the game.

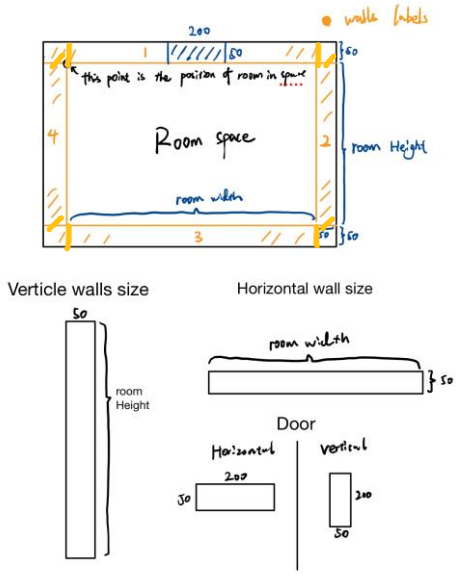
Mapgeneration: this class consist a list of **rooms** it helps with managing individual rooms in the game, it uses a list to store rooms because the room in every game run is totally random (Requirement G1 e), and it also has a draw function which displays the room in the game

Rooms: this class is responsible for creating room in game, it has attributes such as texture, door, walls, it constructs the rooms in the game, importantly, each room class holds 3 arrays representing: all walls in this room, vertical walls in this room, horizontal walls in this room. An array is chosen because it's more memory efficient, and each room will have fix number of walls.

This system is instantiated in the Main game, **mapgeneration** is first created, and its constructor will use a random number generator to generate random amount of room, by instantiate the **Rooms** class and call the functions in rooms to create rooms, and store it in a list, I will illustrate the process of this system in next page to demonstrate separately how both classes work and how they work together.

Individual Room generation flow chart





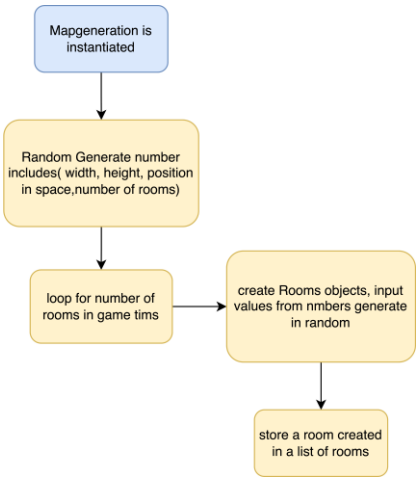
Flow chart for construction of room(figure 2) **Diagram for strcture of room(figure 3)**

The flow chart demosntrates the procedure of function calls during generating individual rooms, and what each function does. When a Room is asked to be generated, it is given the width, height, position values, and a room is made up of 6 rectangles. The first Rectangle is the room space means the floor, or the space avaliable in room**(Requirement G1. a.2, a.3)**, then the next four rectangles represents 2 horizontal walls, 2 verticle walls**(Requirement G1. a.4)**, the last rectangle is the door placed on any side of walls**(Requirement G1 a.4)**.

When walls is generated is given labels, the two horizontal walls is labelled as 1 and 3, vertical walls is 2 and 4, figue 3 shows the structure of room is made up of, and the labels on each wall, also the size of everything in terms of pixels on a bitmap image. During the generation process, a random number between 1 o 4 is used to determain which side of wall the door should be placed on**(Requirement G1 d.)**. a, and I can also use whether the number is odd or even to determain if I need a horizontal door or vertical doorFlow chart for generation of all the rooms when game starts.

For diagram figure3, all numbers represent there length on the screen, in pixels.

Flow chart for generation of all rooms



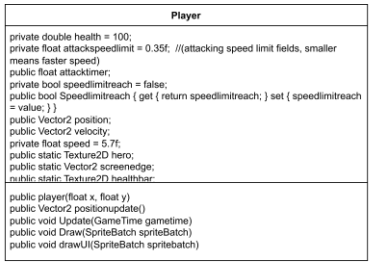
This Flowchart shows the processes of the mapgeneration class, it is used mainly to store and generate all the rooms when the game start, it uses a random generator to determine the number of the rooms **(Requirement G1. e),** and it uses a for loop for this number of time and instantiate room objects to store them into a list.

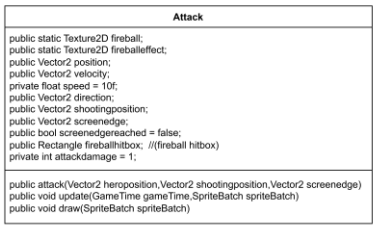
The other random numbers is then pass into the constructor of the rooms accordingly, so that each individual room will have random number of sizes **(Requirement G1. c),** random positions in the game **(Requirement G1.b)**

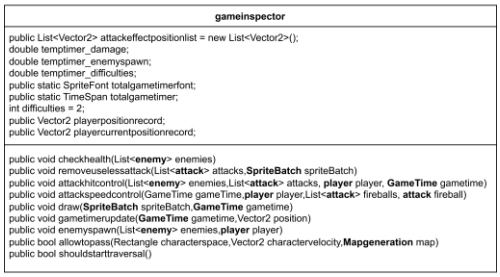
Player Motion Control

This section is mainly focused on all sections in **Requirement P1,** and some sections in **Requirement P2,** which is a system for user to control player's motions, and attacks.

UML Class diagram break down







These sections are responsible to achieve taking input from user and perform movement of player and attack enemy.

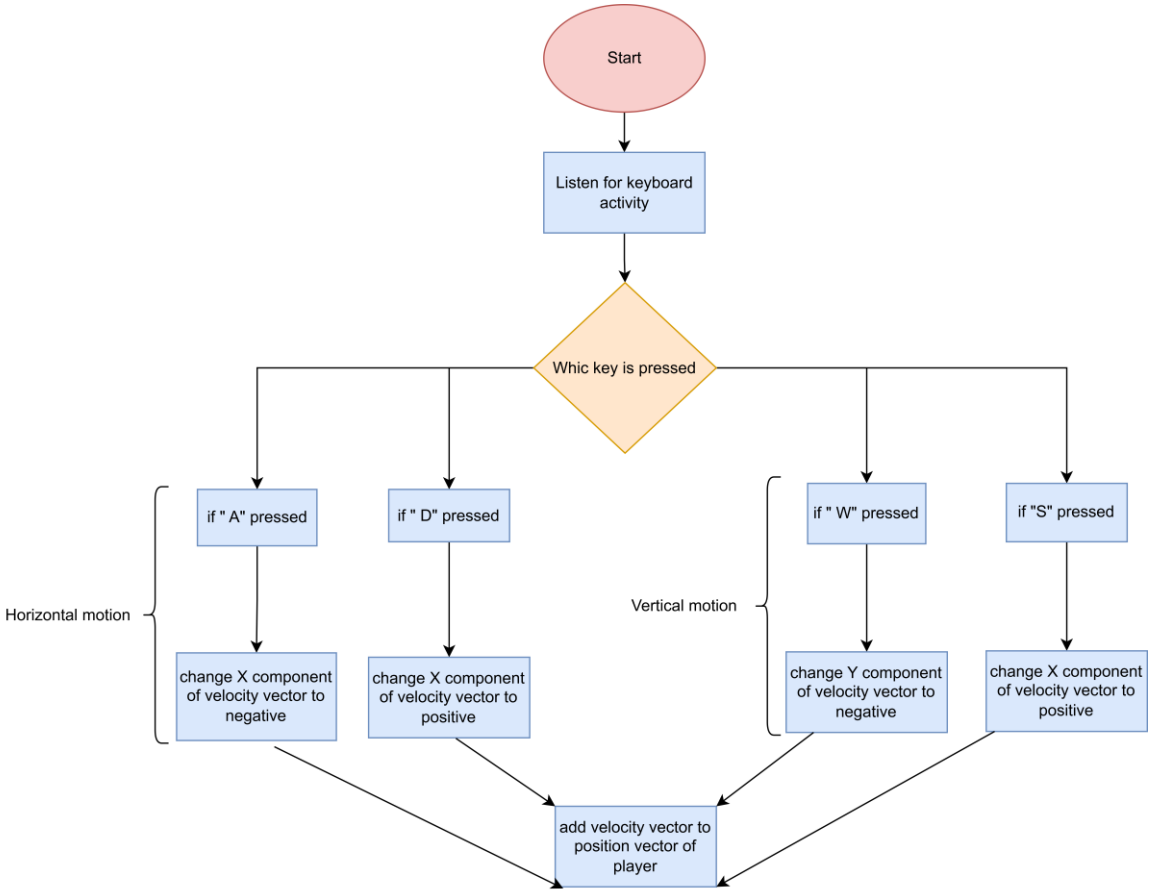
Player: responsible for managing player's stats, such as health, speed, money. (Requirement **P3 a. 1, 2, 3)**, and also manage displaying player and manage motion of player from user input. (**Requirement P1. All)**

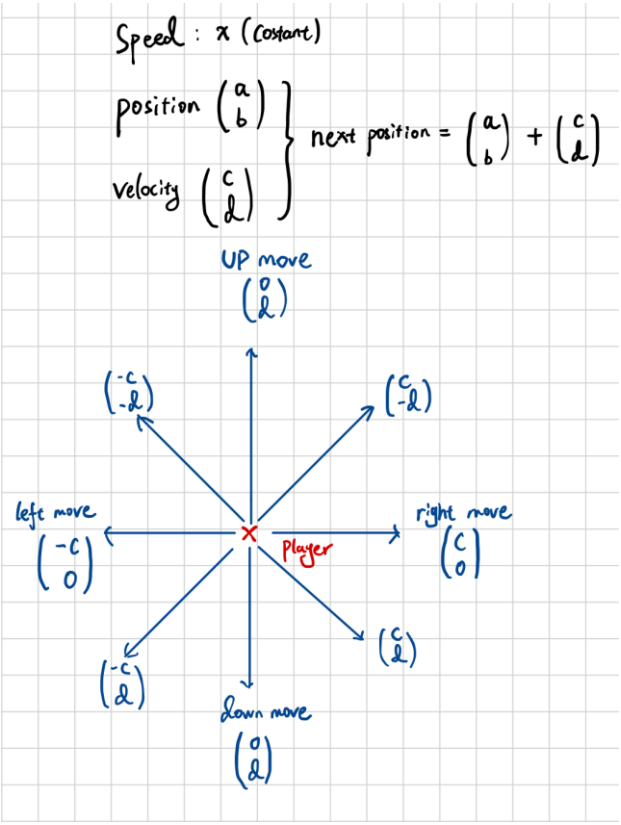
Attack: holds the stats of attacks, such as damage, speed (**Requirement P2**. **d.1**), also manage the display of attack, and motion of attack.

gameinspector: this class is often used in the **gamestart-state,** the state that the game runs on, it consists of range of method and timers, such as total game time passed, collision detection, enemy generation, removing attack that has already hit a target etc. It acts like a robot that constantly inspects the game flow, and making changes according to the game rule, lots of utility classes can also be found here. In this context, it will be responsible for preventing players going through walls in the game.

I will break down each part of this system, explain the system control player's motion, making attacks, and preventing player not going through walls in the following page.

Player motion control analysis and flowchart





The Game is in 2 dimensions, so everything in the game will have a coordinate x and y, to move the player to any directions I like, I've chose to use vectors to calculate player's position. I have used the knowledge in math mechanics, to store 3 values for player, a scalar speed, a vector position (like displacement), a vector velocity, so that by changing the values in the velocity component, and add to position vector will result a change in position, which moves the player to any direction (**Requirement P1 b. 1,2,3)**

This solution also helps with changing the speed of player smoothly, whenever I want to change the speed of player, I only have to adjust the scalar speed of player. (**Requirement P1 c)**

The flow chart demonstrate how player respond to user input. In the game flow when player gets updated, it will listen for a keyboard input, "W, A, S, D", and for each key the system will decide to negate a component in velocity vector and add the velocity to position. (**Requirement P1 a)**

Preventing player going through walls

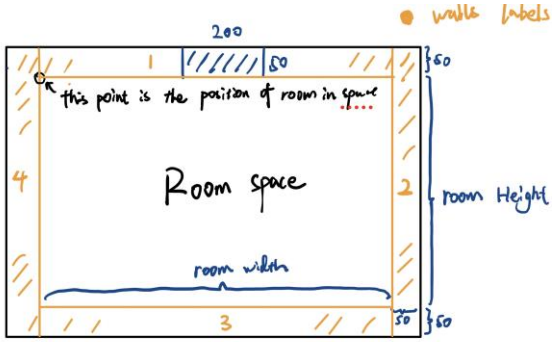
I haven't written any requirement for wall act as an obstacle which can block player in game, because I think it is obvious that player shouldn't be walking through walls, but to be able to help the system understand the relationship between player and walls, a collision system is required, this is where the **gameinspector** class is involved.

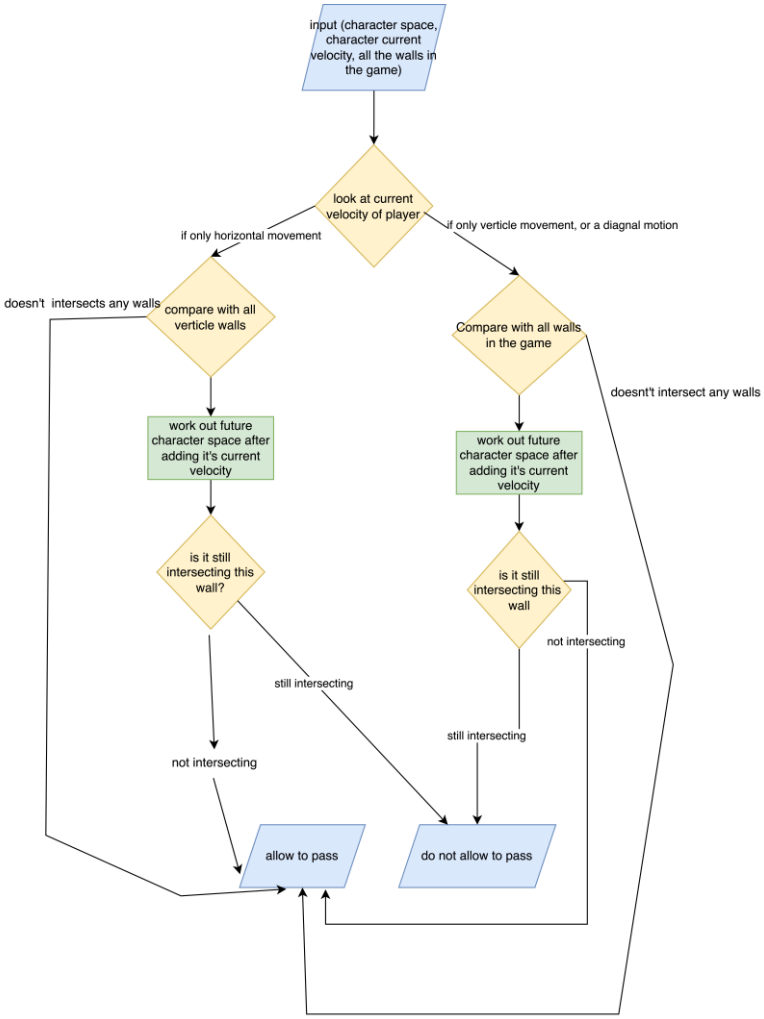
gameinspector

```
public List<Vector2> attackeffectpositionlist = new List<Vector2>();
double temptimer_damage;
double temptimer_enemyspawn;
double temptimer_difficulties;
public static SpriteFont totalgametimerfont;
public static TimeSpan totalgametimer;
int difficulties = 2;
public Vector2 playerpositionrecord;
public Vector2 playercurrentpositionrecord;
public Queue<Vector2> enemygenration = new Queue<Vector2>();
int timeloop = 60;
Random spawnenemyrandom = new Random();
```

```
public void checkhealth(List<enemy> enemies)
public void removeuselessattack(List<attack> attacks,SpriteBatch spriteBatch)
public void attackhitcontrol(List<enemy> enemies,List<attack> attacks, player player, GameTime gametime)
public void attackspeedcontrol(GameTime gameTime,player player,List<attack> fireballs, attack fireball)
public void draw(SpriteBatch spriteBatch,GameTime gametime)
public void gametimerupdate(GameTime gametime,Vector2 position)
public void enemyspawn(List<enemy> enemies,player player)
public bool allowtopass(Rectangle characterspace,Vector2 charactervelocity,Mapgeneration map)
public bool shouldstarttraversal()
public bool isitintersectingwalls(Rectangle checkingpace, string walltype)
```

Diagram for structure of room and flowchart for system



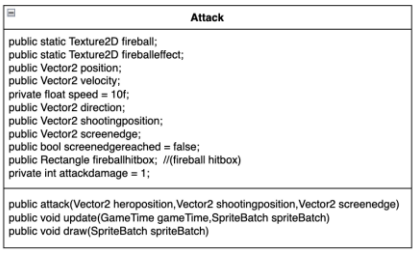


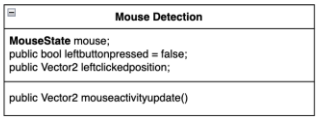
The method **allowtopass and isitintersectingwalls** is used to do this job, this method can take an input of(an rectangel represent the rectangle of the any moving object, a vector of velocity that this object is moving at, and the map object in the system).

The diagram of the room is included to as a reference for logic in this system. The flowchart to at the left demonstrates the flow of this system, if will first check the horizontal and vertical component of player's current velocity, due to the structure of the room (see diagram), if player is moving with only horizontal velocity, it's impossible to collide with any horizontal walls, so only verticle walls has to be compared to, make program run more efficient for only checking half of the walls in game. However, the horizontal walls are built inside and between vertical walls (see diagram), to make a room fully wrapped around with walls, therefore, when player is moving vertically or diagonally, the system must check through all the walls in game, because otherwise player can slide into the vertical walls if only the horizontals are being checked. When designing this system, I have considered of likeliness of player's different motion during game, and after a few tests I found out the most common move is horizontal move because the screen is essentially a rectangle with longer width, so player will spend more time moving horizontally, and that's why rooms was designed like the diagram, in most cases, only the vertical walls are going to be checked.

Perform attacks

This section is describing a system for **(Requirement P2. a,b,c),** which allows player to perform attack in the direction of mouse click on the screen, in this case I will be using fireball as an example. **Attack** class is responsible for damage of attacks, and calculates the motion of attack.

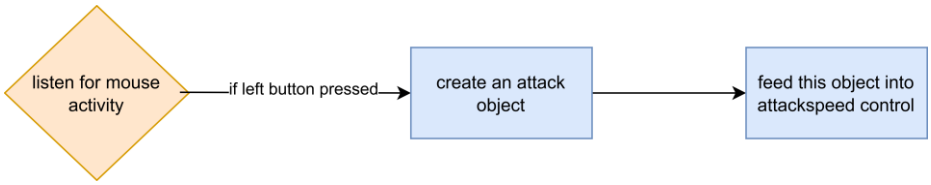




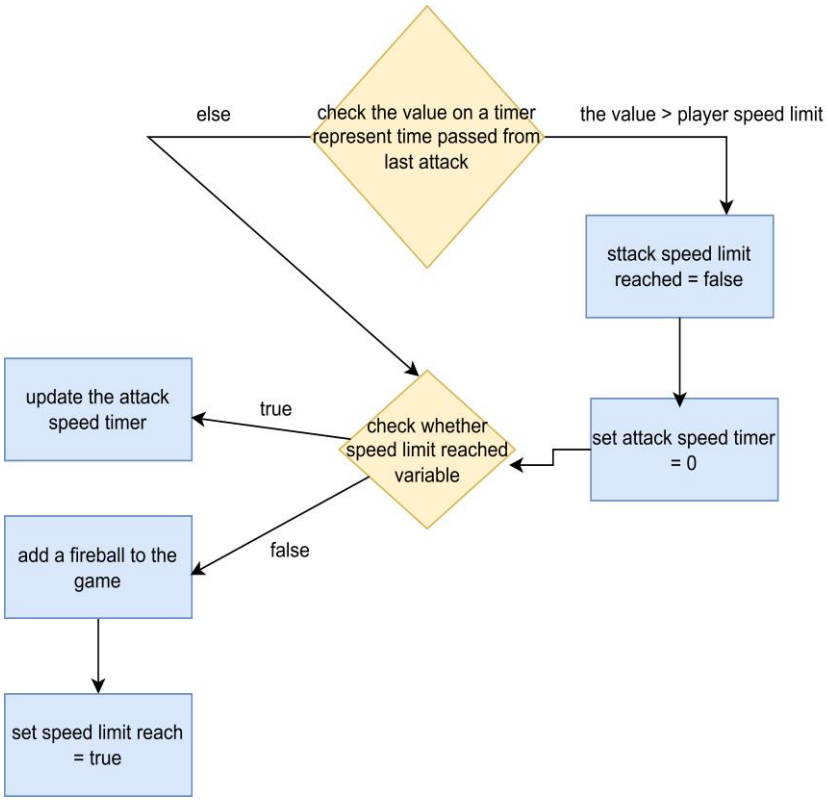
In order to make the controls of the game easy for players, the control of performing attacks in this game was designed as player can throw and attack by left click on screen, and the attack will fly towards that direction (**Requirement P2. b).**

How was attack created

Flow chart for system which create attack object



This is a rather simple system, it basically listens for left click and decide to create an attack or not, the crucial part of it is the **attackspeed control** function in **gameinspector.(Requirement P2 a)**



Attackspeed control is a system that decide whether to add an attack to the game by constantly checking and resetting a timer, this timer in the game starts whenever when an attack is made. The system first checks the timer against the value of attack speed limit (represent how often player is allowed to attack), if the timer is bigger than this value, the attack limit reached is set to false and the timer value is reset to 0. Every time the game update loop calls this function, it will always check this attack limit reached variable, when it is true, only the timer is updated, when it is false, means the player haven't reached the attack speed limit, so a fireball is then added, and the speed limit reach is set back to true, because after this attack the timer should start from 0 then time until next attack. The purpose of this system is to provide control on how often player can perform attacks **(Requirement P2.c)** and allow it to be managed easily, whenever I want to increase or decrease the attack speed, I will only have to change the player's speed limit value.

How was attack directed

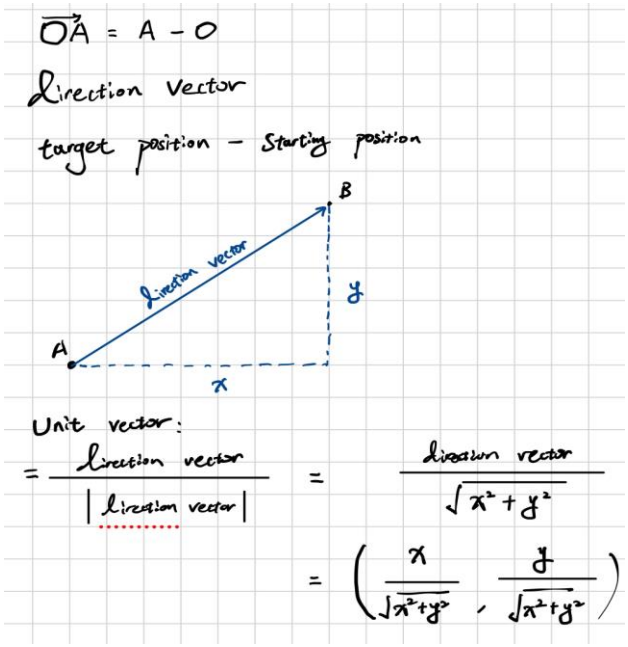
Here I will explain how my system direct the motion of each individual attack, for them to fly for a straight line across the screen towards where the mouse was clicked.

Attack

```
public static Texture2D fireball;
public static Texture2D fireballeffect;
public Vector2 position;
public Vector2 velocity;
private float speed = 10f;
public Vector2 direction;
public Vector2 shootingposition;
public Vector2 screenedge;
public bool screenedgereached = false;
public Rectangle fireballhitbox;  //(fireball hitbox)
private int attackdamage = 1;
```

```
public attack(Vector2 heroposition,Vector2 shootingposition,Vector2 screenedge)
public void update(GameTime gameTime,SpriteBatch spriteBatch)
public void draw(SpriteBatch spriteBatch)
Vector2 normalizevectors(Vector2 directionvector)
```

Whenever an attack is instantiated, the position it should start and player's click on the screen is all passed into the constructor as an vector, in order to make the attack fly an straight line in the direction of mouse click on the screen, I used my knowledge on vectors in math to achieve this.

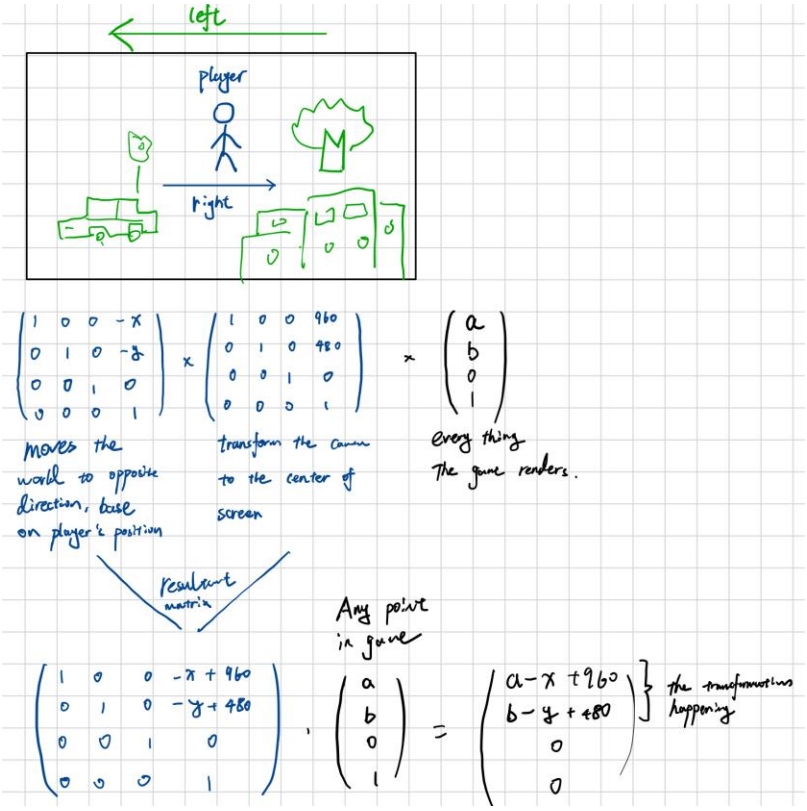


First, I need to work out the direction vector of the attack, which is simply the target position – starting position, this vector represents the attack should fly towards. Secondly, to make the attack fly at unit length on this path, I worked out the unit vector of it, and add this unit vector just like a velocity each time to the attack, it is worked out by the original vector divided by its magnitude, which is calculated from Pythagoras, this will help attack to move an exactly straight line towards this direction. (**Requirement P2.b)**

Camera System

This is system is built to achieve **Requirement G3. a, b.** the camera system was built because it will center the screen on the player, as player moves around, the camera will follow, this allows player to move in a large map without losing focus on the player himself.

Camera
public Matrix transformationmat; public Vector2 position;
public void update(player player)



A transformation matrix was chosen to achieve this, although this transformation will only be used in game render processes, but a matrix makes it easier if I want to move the transformed coordinate back, which I will only have to use the invert of transformation matrix.

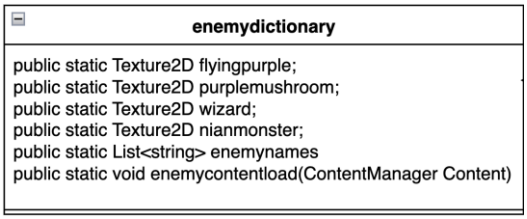
The transformation matrix is former by 2 parts, the first matrix basically takes the values which is the invert of player's position, so when player moves in any direction, it will in fact move everything else to the opposite direction, this provide the visual effect which camera is following the player, by moving the entire game world to opposite direction of player.

The second matrix takes the value of half of the screen width and height, which is helps center the player in the middle of screen by moving half right, half down**.**

(Requirement G3.a)

When those two matrices are multiplied together, I will get the transformation matrix, and by applying this matrix to game rendering processes, it will provide a visual effect of camera following and centering the player**. (Requirement G3.b)**

Different Enemies

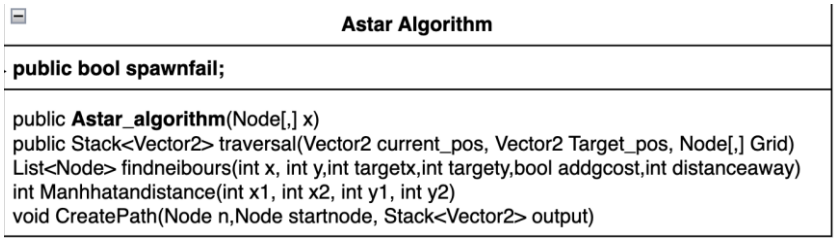


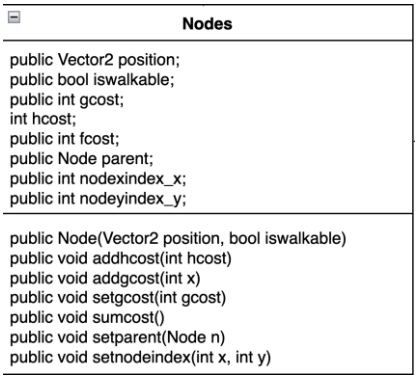
This section is aimed to achieve **Requirement E2, E3**, which is having different types of enemies in game and allowing them to have different property, **enemydictionary** class is used to achieve that.

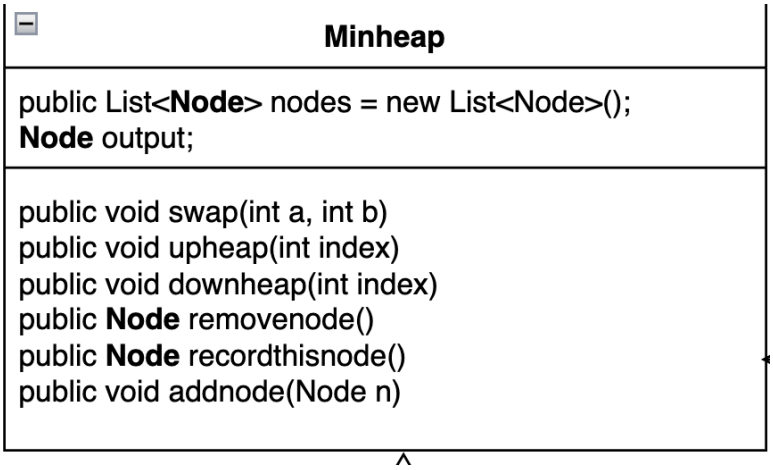
This class contains all information about each individual enemy, by having a **dictionary** with the name of enemy being the key, and store all the information such as, speed, damage, health as strings, whenever an enemy is required to be instantiated, by specifying the name of enemy, the constructor of enemy will fetch the according data required from this dictionary, this is allow me to have a more centered control of attribute for every enemy, by changing the according string value in the dictionary, and this also allows me to add a new enemy to the game with ease.

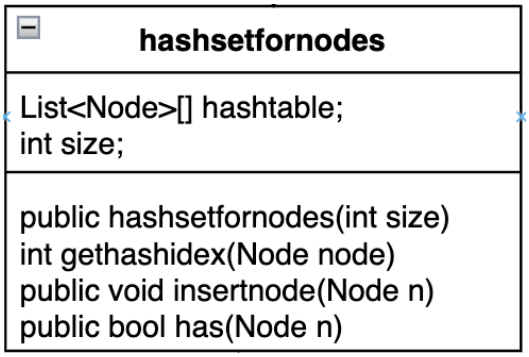
Furthermore, using the name of the enemy as the key allow me to change the enemy to be added depending on different stages of game or the map chosen, it's a sensible unique identifier.

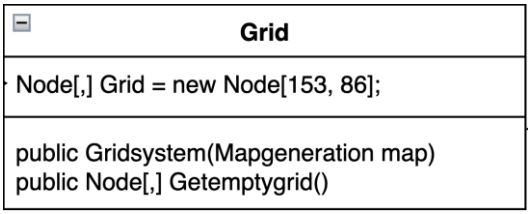
Enemy Path Finding





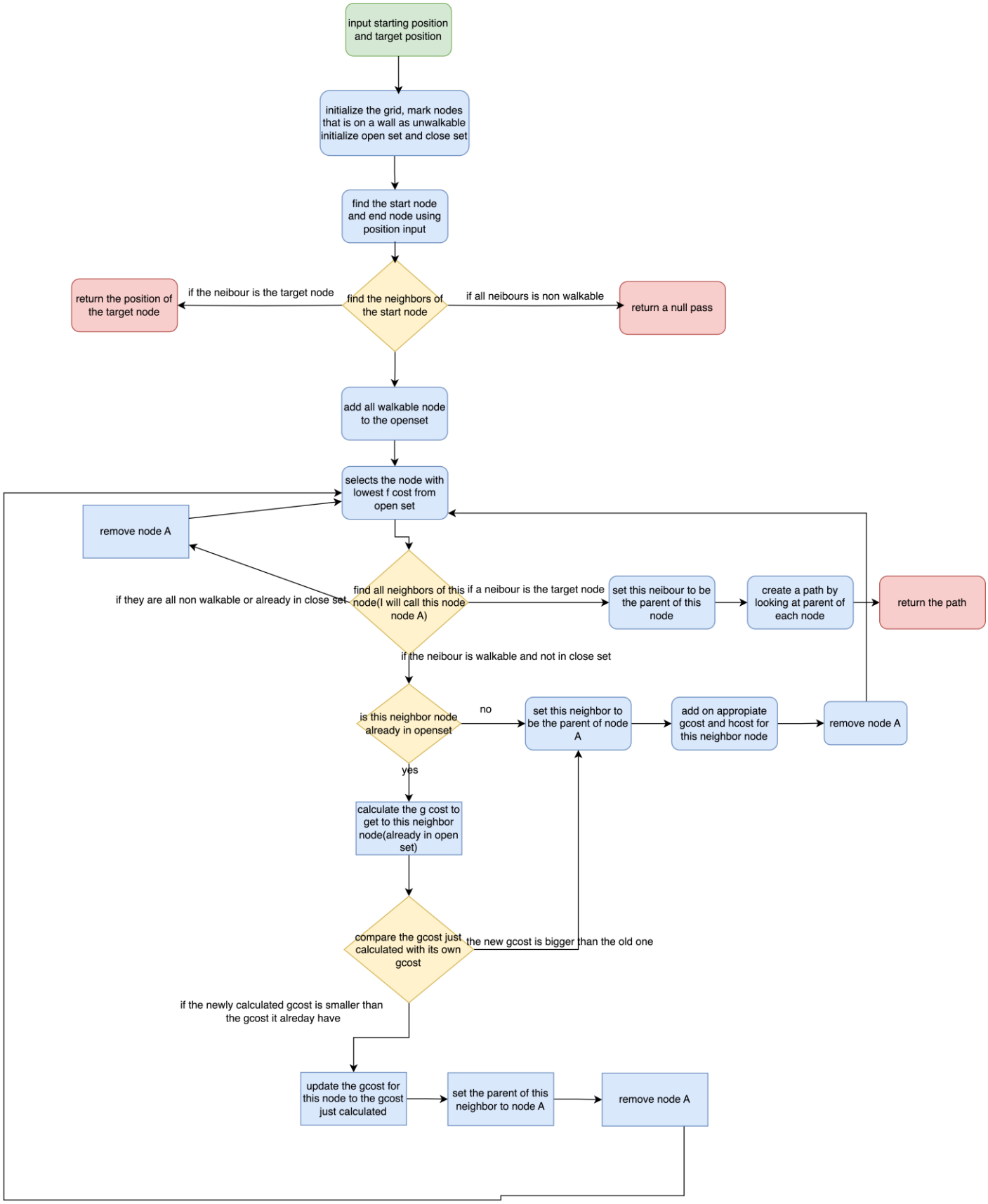






This is a huge part of my game, it is aimed to achieve (**Requirement E1.2 all),** using astar algorithm to achieve path finding function. It consists of five classes, **Astaralgorithm** class is a class to find a path for enemy, all the logical calculation for the path is done in this class, and it links the three other class together. **Node** class represent a singular node in the map, it holds information such as the position of the node, the index of node on grid. **Grid** class create a grid of nodes on the map when game first started. **Minheap** class is a class which does **heap sort** based on the fcost of the node, it represents the open set in the astar algorithm, because the algorithm needs to find the node of lowest f cost, having a sorted nodes makes the search much easier. **Hashsetfornode** is a class which store the nodes using a **hashing table**, because the algorithm needs to constantly check whether a node is in the close set, using a hash set provides O (1) time complexity for searching.

Astar algorithm



Flow chart for astar algorithm logics.

How does it work?

I have chosen to use the astar algorithm to handle the path finding function for my enemies, is because it guarantees to find an optimum pass between two nodes if there is one, and it is very efficient when dealing with a large map, in my case lots of nodes. I will illustrate the logic for astar algorithm generally, then I will go into details about how each individual class was used and process along side with this algorithm.

The ability of astar algorithm to find an optimum path relies on evaluation of 3 costs for each node
G cost: the actual distance to go from start node to this node
H cost: the heuristic distance (estimated distance) to go from the start node to this node.
F cost: this is the sum of the G cost and H cost.

And two sets:
Open set: this set contains the node that awaits to be evaluated, or available nodes for not.
Close Set: this set contains the node that is already traversed, or not walkable, so nodes here won't be looked up.

The algorithm starts from the starting node, and gradually works its way towards the target node by constantly finding neighbors of the nodes, it first finds neighbors of this node, and put the node which are non-walkable in the close set, the remaining nodes will all be put into openset, once this process completes the start node will also be put to close set, because it's fully explored.

After successfully instantiate the open set, the algorithm begins a while loop, while the openset isn't empty, the program will select the node which have the lowest f cost (sum of g cost and h cost), then find all neighbors of this node, I will call this node nodeA now. The code will determine whether to put a neighbor into open set or close set, or end the algorithm based on these four conditions.

1. If a neighbor node is not walkable, it will be put into closes set and never checked again.
2. If neighbor node is walkable, and not in close set, that means this node is a newly discovered node, the program will calculate the g cost and h cost for this node and add it to open set.
3. If neighbor node is walkable, but already in open set, that means it was found before but not evaluated yet, the program will calculate it's g cost now, which is the new g cost for this node (because it way found again), and compare this new g cost with its old g cost, if the new g cost is smaller, that means we have found a shorter path to this node, we will set the parent of this node to node A (because we are coming from node A), and then update its g cost to the newly calculated g cost.
4. If neighbor node is the target node, the program will begin to generate the path, by looking at the position of parent node in its way to the target recursively. The output format is a serial of position in game, and it is reversed since the path is generated from end to beginning, therefore a stack is used to push into the position of the node when a parent is looked up, and when enemy need to access these positions, it will only have to pop from this stack.

The algorithm will continue to loop until it finds an optimum path towards the player, a path is then created, a condition of 6500 is set to this loop which is the half of amount of node in the map, if target hasn't been found in 6500 loops, the program will terminate and remove this enemy.

Pseudocode for astar_algorithm core structure

```
Function heuristic(self, node, goal):
    Calculate and return the Manhattan distance between node and goal

Function get_neighbors(self, node):
    Define possible movements (left, right, up, down)
    For each movement direction:
        Calculate neighbor's position
        If neighbor is within grid bounds and is not an obstacle:
            Add neighbor to list of neighbors
    Return list of neighbors

Function move_cost(self, node):
    Define cost for moving from a node to its neighbor (e.g., 1 for adjacent moves)
    Return the move cost

Function search(self):
    While open_list is not empty:
        Find the node in open_list with the lowest f score (g + heuristic)
        If current node is the goal:
            Construct path from goal to start by tracing parents
            Return the path
        Remove current node from open_list and add it to closed_list
        For each neighbor of the current node:
            If neighbor is in closed_list:
                Continue to next neighbor
            Calculate tentative_g_score (current's g + move cost to neighbor)
            If neighbor not in open_list or tentative_g_score < g[neighbor]:
                Update neighbor's parent to current
                Update neighbor's g score to tentative_g_score
                If neighbor not in open_list:
                    Add neighbor to open_list
    Return failure if no path is found
```

Heuristic cost

Heuristic cost plays a huge part when evaluating the costs for nodes, because it represent the estimated distance to get to the target node from the current node, in my case, hence I have a grid system and the neighbours of the nodes is the top and bottom, right and left of the current node, so my heuristic cost is calculated as the modulus distance of X + modulus distance of Y.

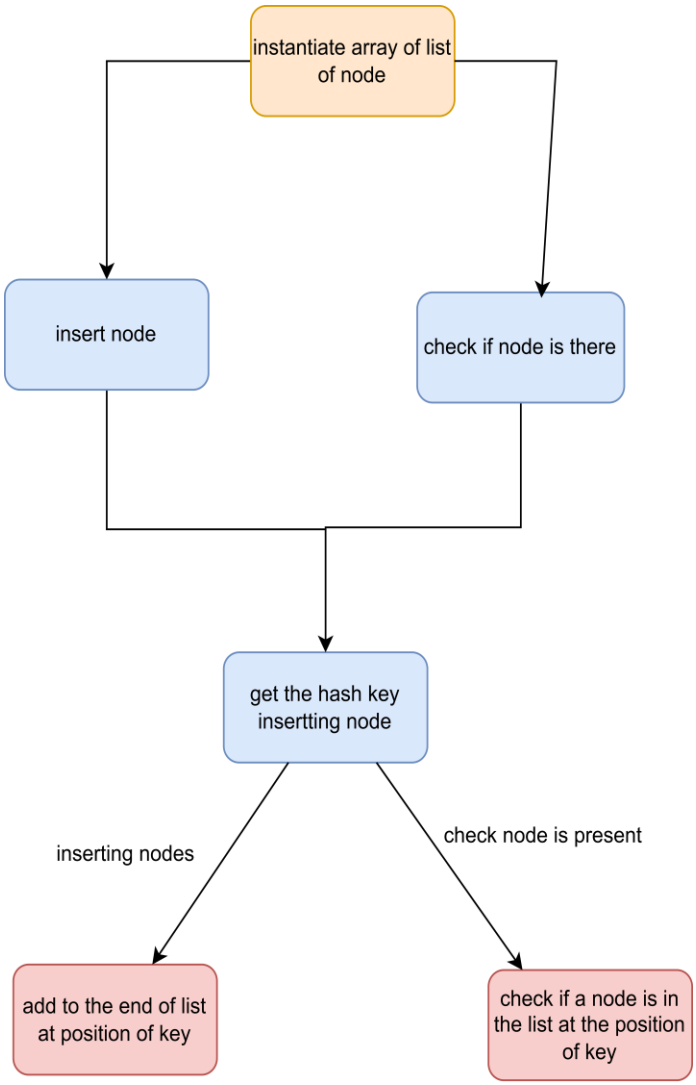
Hash Table

hashsetfornodes
List<Node>[] hashtable; int size;
public hashsetfornodes(int size) int gethashidex(Node node) public void insertnode(Node n) public bool has(Node n)

The close set in a star algorithm is frequently used, and the code often checks if a node is already present in the close set, so I chose to use a hash table because it has O(1) time complexity no matter how many data there is (if no collision occur).

My hash table is stored as an array of list and the value stored are nodes, the hash key of the node is calculated as (x index of node * 23 + y index of node) %size of hash table, prime number 23 is use to make the result of the calculation as unique as possible to avoid collision. When a node is being decided to put It into close set, the gethashidex calculated the key of the node, and add the node to the according position to the list, and if collision occur it will simply be added to the end of the list. To check if a node was in the hash table, the process is very similar, the key is calculated and the program checks if the list contains the node at the according position.

flow chart for hash table process



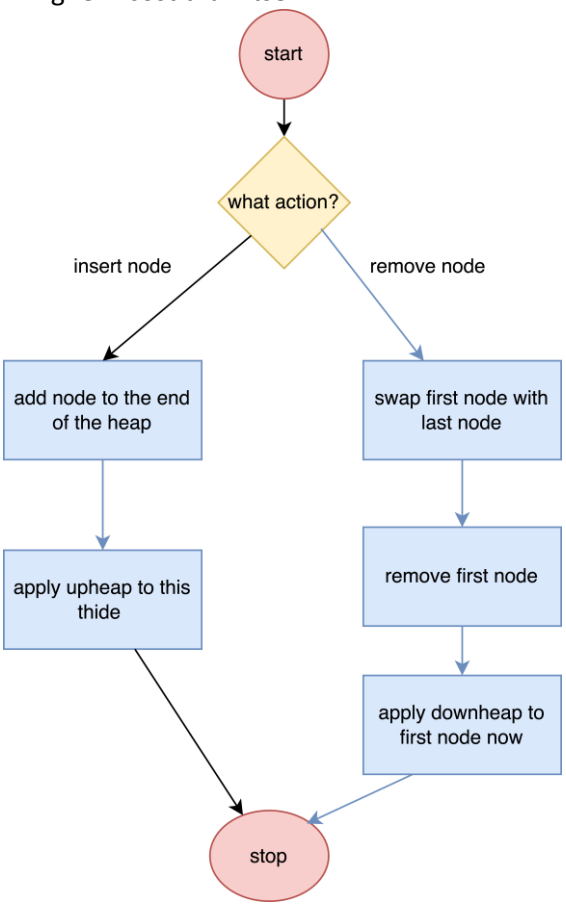
Min Heap

Minheap
public List<**Node**> nodes = new List<Node>(); **Node** output;
public void swap(int a, int b) public void upheap(int index) public void downheap(int index) public **Node** removenode() public **Node** recordthisnode() public void addnode(Node n)

I have used a heap sort to represent my open set, since the length of the open set can change dramatically during the algorithm because neighbours are constantly being found but node is also being removed when the algorithm finish checking with it. During the algorithm, a crucial step is to select the node with the lowest f cost, while the open set sizes can vary a lot, having a sorted list of nodes will make my program more efficient, because each time I will only have to find the first item in my open set, because it was sorted in ascending order based on the f cost of the node, using a heap sort.

I have used a list of nodes to represent a binary tree, also the open set, the root of the tree (first item) is the node with smallest f cost. When adding node, to the heap, it will simply add a node to the end of the heap, and call a **upheap** function to this node, the upheap function would basically check the f cost of this node with its parent, if the f cost of this node is smaller, swap it with its parent node, repeat the process until it find a parent which has a smaller f cost than itself, or it ran out of parent (it is the smallest f cost node in the list).

A star algorithm also involves moving nodes from open set to close set after fully explored, so my minheap close also involves a function of **downheap**. Hence the only condition to remove a node from open set is after it's explored, so the node I'm removing is always the first node, because it was the node just explored (it has lowest f cost). After it's removed, the heap must be reconstructed, to make sure it is well maintained, this function does the follows, I will call the node I'm going to delete, node A. First, node A is swapped with the root node, and downheap is applied to this node. The downheap function will start from the root node and compare constantly compare with its left and right child, then swap with the one with smallest f cost, until the point its children node all has higher f cost than itself.



Flowchart for program flow in min heap class

Nodes and algorithm efficiency

The entire a star algorithm is about evaluating costs on nodes, and I have set each of my node to be a square that is 50 * 50 pixels square, because this is the size of my smallest enemy in game.

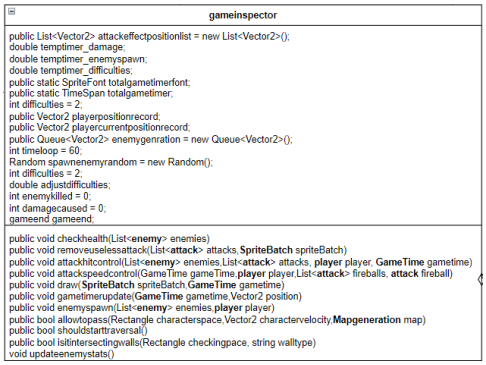
The grid of nodes is created at the beginning of the game, I have 153 x 86 nodes in my game calculated from the resolution of my map ground, and each node holds information about its costs, and function which sets and changes its costs, so the memory usage will be quite big when this algorithm is frequently called, especially as time passes and enemy increases.

So, I have set a detectable range for all my enemies to player, that will avoid enemy just generated which is very far from player in map, constantly trying to generate a path to get to player, another benefit for this is by setting a limit of distance which allow enemy calculate a path, enemy will only starts moving when they “see” player, also when player sees them which is more sensible, avoid unnecessary calculation.

To improve the efficiency of algorithm even more, I have added a timer to control the frequency of path finding, it will record the time from last traverse until a time limit, and call the algorithm again if the limit is reached, this limit is also a variable during the game, it changes based on the distance as well. The closed the enemy gets to player the smaller the limit is, because when enemy is close to player the path needs to be more precise in order to allow it to hit player and vice versa, when it is further from player, having the program to finds path in a given time limit isn’t very precise, but the general direction will be correct.

Enemy generation

This section is aimed to achieve **Requirement G7**



Again, game inspector was used at here to achieve spawning enemy, and adjust difficulties of the game according to time. Almost all the timers, including timer for attacking speed, enemy spawn, total game timer, position generation etc are created and being updated in game inspector, this provides a very convenient and centralized control of all the timers.

Just like all rogue like game, the difficulties increase as the time passes, my game will increase the difficulty every 30 seconds. At the first minute of the game, only the default enemy is spawned, and the special enemy (enemy which can perform long range attack, specialized to map) is spawned.

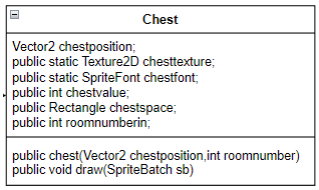
And the statistic of enemies such as health, damage, coin drop is updated by update the values in **enemy dictionary**.

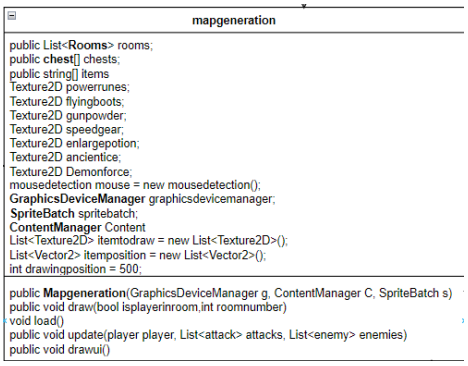
To ensure enemy won't spawn into walls or rooms, which makes the path finding a lot harder, a **Queue** of positions is used, the game will always enqueue some position which are not intersecting any walls or rooms into this queue, and enemy generated will simply pop from this queue to get the position, this also enable the game to generate a series of position when game is initialized, and add into this queue according to the time player survived, which acts as a buffer for generating enemies, because the position of enemy spawned at is random, if position is generated during enemy spawn, it could potentially take too long cause game to lag, so a queue that acts as a pool of position for enemy to access can maintain the game flow well.

Chest and Item

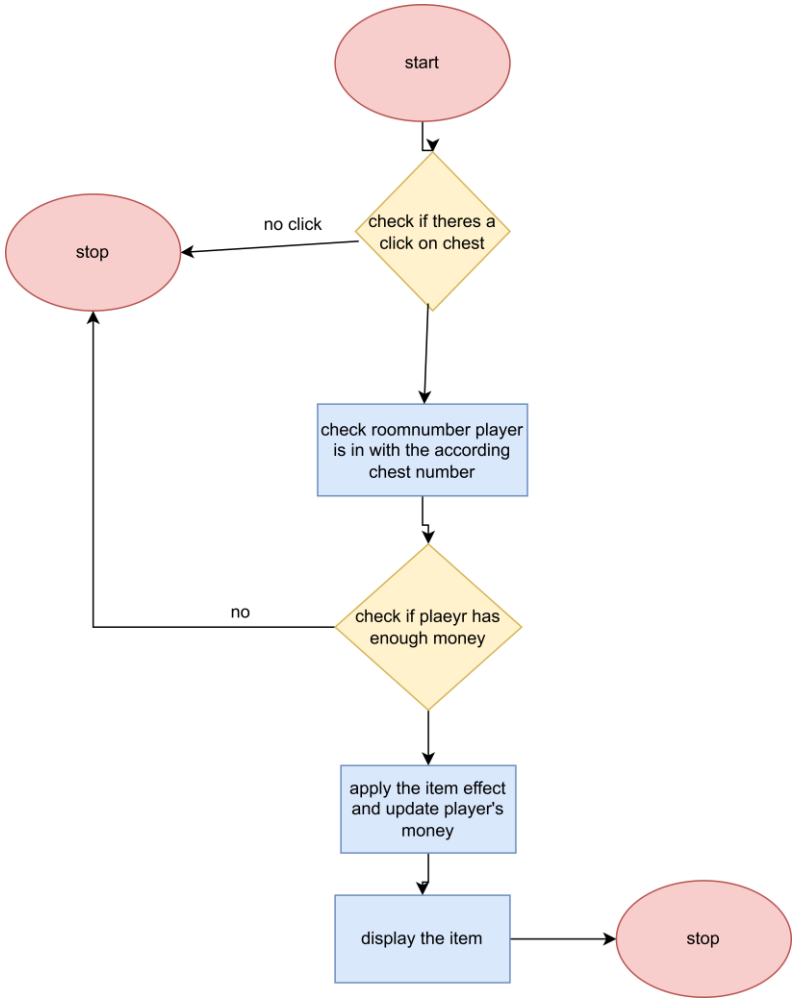
This section is aimed to achieve (**Requirement G1 b all, Requirement G6. B all)**

Chest class and mapgeneration class is used here.





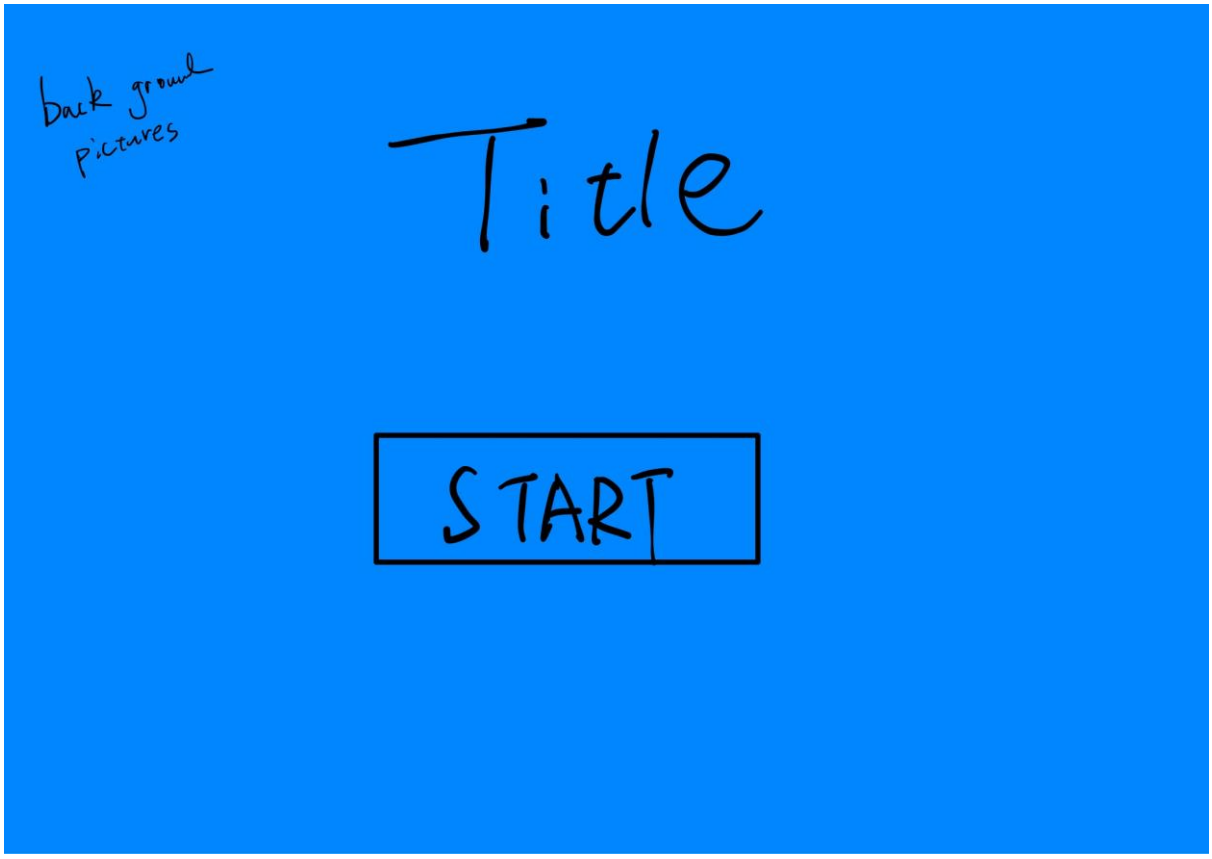
When the map is initialized, a chest is put into each of the rooms, and random value between 10 to 20 coins, with a random item, when player enter the room, player will see the chest with a value and name of item display on top of it, and when player click on the chest with sufficient coins, an item is purchased and display on top of screen, with according effect applied.



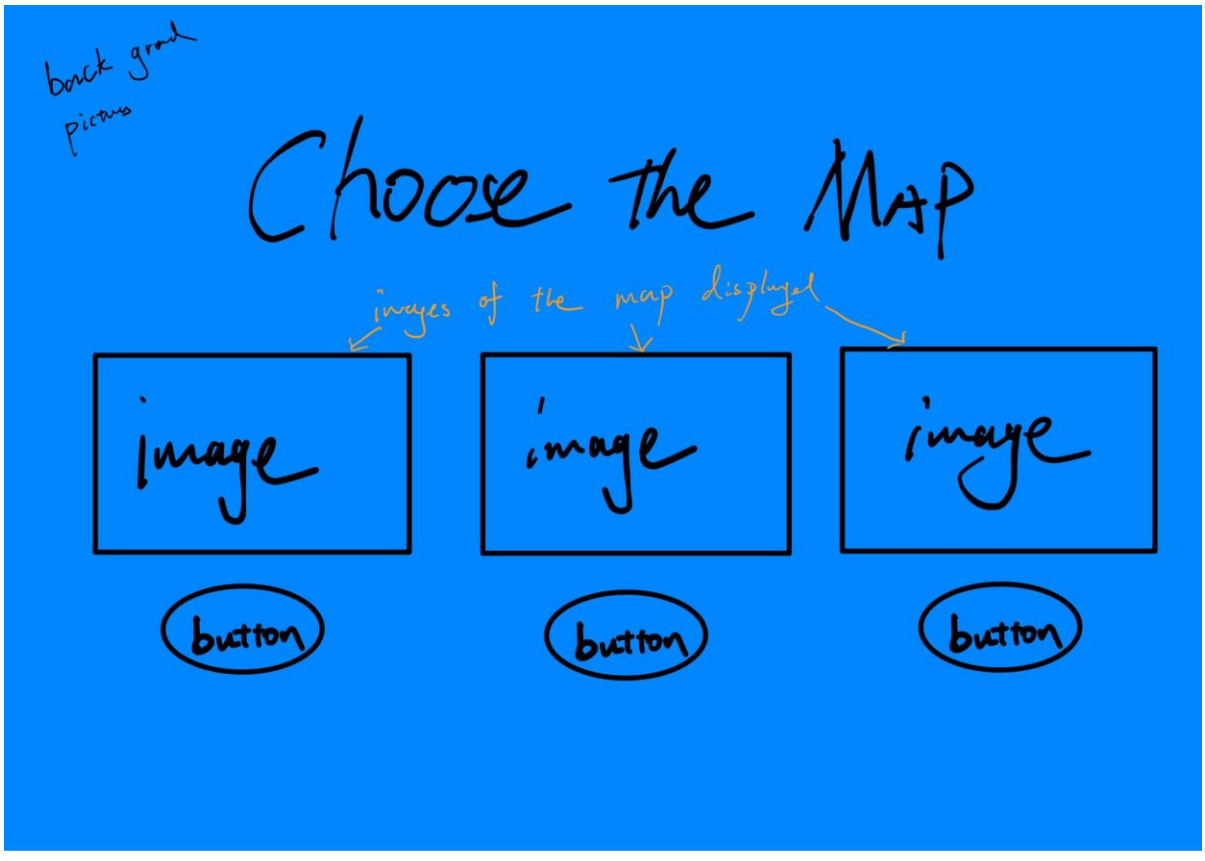
Flow chart for item system

GUI design

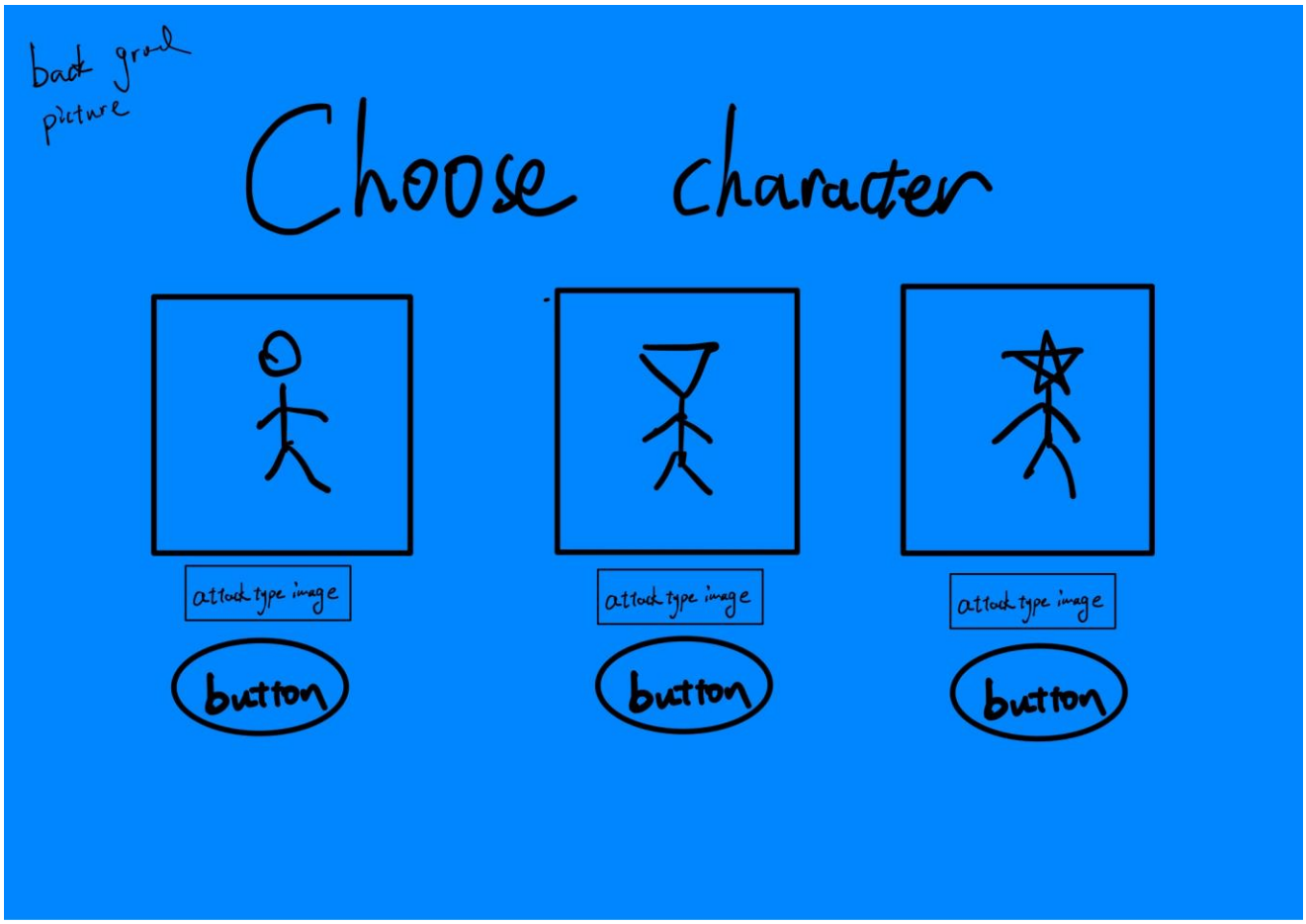
Welcome State



Map choosing State



Character choosing State



Technical Solution

Requirement B1, B2 all User interface and selectable options
Purpose: to create suitable user interface for player, including images, buttons and text messages guides player to choose the map and characters, the menu should contain 3 pages, a welcome plage, a map choosing page and a character choosing page, control by button clicks
Class used: Menu, Game State, state recipe, buttons
Advanced OOP and Design Patterns Utilization of inheritance, encapsulation, and polymorphism. The Menu class extends Staterecipe, and Button encapsulates button attributes and behaviour. **List Manipulation**: Dynamic management of UI components via List<Button>. **Event Handling**: mouse click detection for UI interaction. **Overloading:** overloading method **Abstract Classes and Methods**: game state is overriding abstract classes and methods, and the flow of game state is controlled by a stack of game states, where next state is push into the stack. **Dynamic generation of Gamestate** based on player's selection. **GUI pre designed menu** to guide players with buttons being clickable applying collision detection between mouse The map pictures are designed and loaded, and each time player activate a button, it will record player's choices and pass into the next state as parameters.

Menu class
`using Microsoft.Xna.Framework.Content;`
`using Microsoft.Xna.Framework.Graphics;`
`using Microsoft.Xna.Framework;`
`using System.Collections.Generic;`
`using prototype.Utilities_classes;`
 	
`namespace prototype.GameState.States`
`{`
`  internal class Menu : Staterecipe //inheritance`
`  {`
`  //classes(buttons)`
`  List<Button> buttonspage1 = new List<Button>();`
`  List<Button> buttonspage2 = new List<Button>();`
`  Texture2D menubackground;`
`  Texture2D page1background;`
`  Texture2D page2background;`
`  //fields that needs to be inherit from maingame class`

```
        SpriteBatch sb;
        ContentManager cm;
        GraphicsDeviceManager gm;
        //menu flow
        int currentpage = 1;
        //the maingame state
        gamestartstate gamestartstate;
        string mapchoice;
        string characterchoice;
        public Menu(ContentManager C, SpriteBatch s, GraphicsDeviceManager gm)
        {
            sb = s;
            cm = C;
            this.gm = gm;
        }

        public override void initialize()
        {
            //add buttons here
            buttonspage1.Add(new Button("startbutton", new Vector2(800, 700), new Rectangle(800, 700, 200, 100)));
            //overloading here, and index for buton is required in page 2
            buttonspage2.Add(new Button("choosebutton", new Vector2(300, 700), new Rectangle(300, 700, 200, 100),1));
            buttonspage2.Add(new Button("choosebutton", new Vector2(900, 700), new Rectangle(900, 700, 200, 100),2));
            buttonspage2.Add(new Button("choosebutton", new Vector2(1500, 700), new Rectangle(1500, 700, 200, 100),3));
        }
        public override void load(ContentManager notwanted)
        {
            foreach (Button b in buttonspage1)
            {
                b.load(cm);
            }
            foreach (Button b in buttonspage2)
            {
                b.load(cm);
            }
            menubackground = cm.Load<Texture2D>("menubackground");
            page1background = cm.Load<Texture2D>("page1background");
            page2background = cm.Load<Texture2D>("page3background");
        }
        public override void update(GameTime gametime, GameStates gameStates)
```

```
        {
            switch (currentpage)
            {
                case 1:
                    foreach (Button b in buttonspage1)
                    {
                        if(b.update())
                        {
                            currentpage = 2;
                        }
                    }
                    break;
                case 2:
                    foreach(Button b in buttonspage2)
                    {
                        if (b.update())
                        {
                            switch(b.buttonindex)
                            {
                                case 1:
                                    mapchoice = "background";
                                    currentpage = 3;
                                    break;
                                case 2:
                                    mapchoice = "cave";
                                    currentpage = 3;
                                    break;
                                case 3:
                                    mapchoice = "town";
                                    currentpage = 3;
                                    break;
                            }
                        }
                    }
                    break;
                case 3: // I can use the buttons in page2 hence it's also 3 choices
                    foreach (Button b in buttonspage2)
                    {
                        if (b.update())
                        {
```

```
                        switch (b.buttonindex)
                        {
                            case 1:
                                characterchoice = "hero";
                                gamestartstate = new gamestartstate(gm, cm, sb,mapchoice,characterchoice);
                                gamestartstate.initialize();
                                gamestartstate.load(cm);
                                GameStates.states.Push(gamestartstate);
                                break;
                            case 2:
                                characterchoice = "swordhero";
                                gamestartstate = new gamestartstate(gm, cm, sb, mapchoice, characterchoice);
                                gamestartstate.initialize();
                                gamestartstate.load(cm);
                                GameStates.states.Push(gamestartstate);
                                break;
                            case 3:
                                characterchoice = "gunhero";
                                gamestartstate = new gamestartstate(gm, cm, sb, mapchoice, characterchoice);
                                gamestartstate.initialize();
                                gamestartstate.load(cm);
                                GameStates.states.Push(gamestartstate);
                                break;
                        }
                    }
                }
                break;
        }
    }
    public override void draw(GameTime gametime) //display pages
    {
        switch (currentpage)
        {
            case 1:
                sb.Draw(menubackground, Vector2.Zero, Color.White);
                foreach (Button b in buttonspage1)
                {
                    b.draw(sb);
                }
                break;
```

```
                case 2:
                    sb.Draw(page1background, Vector2.Zero, Color.White);
                    foreach (Button b in buttonspage2)
                    {
                        b.draw(sb);
                    }
                    break;
                case 3:
                    sb.Draw(page2background, Vector2.Zero, Color.White);
                    foreach (Button b in buttonspage2)
                    {
                        b.draw(sb);
                    }
                    break;
            }
        }
        public override void drawUI()
        {

        }
    }
}



Button class
using Microsoft.Xna.Framework;
using Microsoft.Xna.Framework.Content;
using Microsoft.Xna.Framework.Graphics;
using Microsoft.Xna.Framework.Input;
using prototype.GameState;
using prototype.GameState.States;

namespace prototype.Utilities_classes
{
    internal class Button
    {
        public string name;
        public int buttonindex;
        Texture2D buttontexture;
```

```
        Vector2 position;
        Rectangle clickbox;
        //classes used
        //MouseState mousestate;
        mousedetection mouse = new mousedetection();
        // name, position, clickbox rectangle
        public Button(string name, Vector2 position, Rectangle clickbox)
        {
            this.name = name;
            this.position = position;
            this.clickbox = clickbox;
        }
        public Button(string name, Vector2 position, Rectangle clickbox,int buttonindex)
        {
            this.name = name;
            this.position = position;
            this.clickbox = clickbox;
            this.buttonindex = buttonindex;
        }

        public void load(ContentManager cm)
        {
            buttontexture = cm.Load<Texture2D>(name);
        }
        public bool update()
        {
            mouse.mouseactivityupdate(true);//check mouse click and check if it was click on the button
            if(new Rectangle ((int)mouse.leftclickedposition.X,(int)mouse.leftclickedposition.Y, 1,1).Intersects(clickbox)&&
mouse.leftbuttonpressed)
            {
                return true;
            }
            return false;

        }
        public void draw(SpriteBatch spritebatch)
        {
            spritebatch.Draw(buttontexture, position, Color.White);
        }
    }
```

```
}
```

StateRecipe class

```
using Microsoft.Xna.Framework;
using Microsoft.Xna.Framework.Content;
using Microsoft.Xna.Framework.Graphics;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace prototype.GameState
{
    abstract class Staterecipe
    {
        public abstract void initialize();
        public abstract void load(ContentManager cm);
        public abstract void update(GameTime gametime, GameStates gameStates);
        public abstract void draw(GameTime gametime);
        public abstract void drawUI();
    }
}
```

GameState class

```
using prototype.GameState.States;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Security.Cryptography.X509Certificates;
using System.Text;
using System.Threading.Tasks;

namespace prototype.GameState
{
    internal class GameStates
    {
        public static Stack<Staterecipe> states = new Stack<Staterecipe>();
        public GameStates(Staterecipe temp)
```

```
        {
            states.Push(temp);
        }
        public void Push(Staterecipe temp)
        {
            states.Push(temp);
        }
        public void Pop()
        {
            states.Pop();
        }
        public Staterecipe peek()
        {
            return states.Peek();
        }
    }
}
```

Requirement P1. Motion Control of player all
Purpose: To allow player to move in a 2-dimensional space, including movement in any direction from keyboard pressed
Class: player
Techniques: **Math Vectors manipulation**, I used scaler quantity and vector quantity to control the motion in map, and with editable speed Condition control of update method **Condition control collision detection** between walls and player, utilize checking condition based on player's motion, alongside with room design improves efficiency **Error Handling:** prevent player to move outside of game map by defining and edge of the screen.
Section of code in player class

```
// player motion fields
        public Vector2 position;
        public Vector2 velocity;
        private float speed;
        public float Speed   //speed a and m
        {
            get { return speed; } set { speed = value; }
```

```
        }

    public Vector2 positionupdate()//the position update logic for player
        {
            KeyboardState control = Keyboard.GetState();
            //move for W and S
            if (control.IsKeyDown(Keys.W) && !(position.Y < 0))
            {
                velocity.Y = -speed;
            }
            else if (control.IsKeyDown(Keys.S) && !(position.Y >= screenedge.Y - hero.Height))
            {
                velocity.Y = speed;
            }
            else
            {
                velocity.Y = 0;
            }
            //move for A and D
            if (control.IsKeyDown(Keys.A) && !(position.X <= 0))
            {
                velocity.X = -speed;
            }
            else if (control.IsKeyDown(Keys.D) && !(position.X >= screenedge.X - hero.Width))
            {
                velocity.X = speed;
            }
            else
            {
                velocity.X = 0;
            }
            return velocity;
        }
        public void Update(GameTime gametime,Mapgeneration map)//player update
        {
            position += positionupdate();
            playerhitbox = new Rectangle((int)position.X, (int)position.Y,hero.Width,hero.Height);
            foreach(Rooms r in map.rooms)
            {
                if (playerhitbox.Intersects(r.roomspace)) // this loop here check whether player is in a room
```

```
                {
                    roomnumberin = r.roomnumber;
                    isinaroom = true;
                    break;
                }
                else { isinaroom = false;}
            }
        }
        public void Draw(SpriteBatch spriteBatch)
        {
            spriteBatch.Draw(hero, position, Color.White);
        }
        public void drawUI(SpriteBatch spritebatch)
        {
            spritebatch.Draw(healthbar, new Vector2(0, 0), new Rectangle(0, 0, (int)health * 3, 15), Color.White);  //draw
the health bar player
            string moneyText = $"Money: {money}";
            spritebatch.DrawString(moneyfont, moneyText, new Vector2(50,50), Color.Gold, 0f, Vector2.Zero, 2f, SpriteEf-
fects.None, 0f);
        }
    }
}


Section of gameinspector class which shows collision detection between player and walls

public bool allowtopass(Rectangle characterspace,Vector2 charactervelocity)//to stop any chracters walk through a wall
        {                       //current position, the velocity vector, the rectangle of the character space, the current
map object
            for(int i = 0; i < map.rooms.Count; i++)//this is essetially every room in map
            {
                if (charactervelocity.Y == 0)// if player is wallking horizontal, is not going to intersects horizontal
walls
                {
                    foreach(Rectangle wall in map.rooms[i].verticalwalls)
                    { //while player intersecting walls but not intersecting doors
                        while (characterspace.Intersects(wall) && (!characterspace.Intersects(map.rooms[i].door)))//this is
to locate a specific wall in the map player met
                        {
                            Vector2 temp = new Vector2((int)characterspace.X, (int)characterspace.Y) + charactervelocity;
```

```
                            Rectangle futureposition = new Rectangle((int)temp.X, (int)temp.Y, characterspace.Width, char-
acterspace.Height);
                            if (futureposition.Intersects(wall))
                            {
                                return false;
                            }
                            else
                            {
                                return true;
                            }
                        }
                    }
                }
                else
                {
                    for(int j = 0; j < 4; j++)
                    {
                        while (characterspace.Intersects(map.rooms[i].walls[j]) && (!characterspace.Inter-
sects(map.rooms[i].door)))//this is to locate a specific wall in the map player met
                        {
                            Vector2 temp = new Vector2((int)characterspace.X, (int)characterspace.Y) + charactervelocity;
                            Rectangle futureposition = new Rectangle((int)temp.X, (int)temp.Y, characterspace.Width, char-
acterspace.Height);
                            if (futureposition.Intersects(map.rooms[i].walls[j]))
                            {
                                return false;
                            }
                            else
                            {
                                return true;
                            }
                        }
                    }
                }
            }
            return true;
        }
```

Requirement P2. Attack Control of player, Requirement B3.b

Purpose: allow player to perform attack, where the direction of attack is going towards the mouse click on screen, also having attack speed that can be modify, and frequency of attack is as well

Class: Attack, gameinspector, gamestartstate

Techniques used:

Complex User-defined Algorithms: The normalization of vectors for attack trajectory calculation.

Object-oriented Programming: Implementing dynamic behavior through polymorphism, encapsulated within properties for speed and attackdamage.

Resource Management: Dynamically loading and applying different textures based on the game state or character actions, showcasing efficient handling of game assets.

Math Vector Mathematics for Movement: Utilizes normalization for calculating straight-line motion

Math Transformation matrix for reversing the transformed mouse click position to actual position in game

Switch case statement for different characters, see (**Requirement B3.b),**

Dynamic creation of attacks: all the attack is created based on mouse control, and stored and update accordingly.

Section of code in Attack class

```
using DryIoc.ImTools;
using Microsoft.Xna.Framework;
using Microsoft.Xna.Framework.Content;
using Microsoft.Xna.Framework.Graphics;
using Microsoft.Xna.Framework.Input;
using prototype.GameState.States;
using System;
using System.Runtime.InteropServices;

namespace prototype
{
    internal class attack
    {
        //appearance
        string charactername;
        public static Texture2D fireball;
        public static Texture2D fireballeffect;
        public static Texture2D specialenemyattack;

        //fireball motion fields
        public Vector2 position;
        private float speed = 10f;
        public float Speed   //speed a and m
```

```
        {
            get { return speed; }
            set { speed = value; }
        }
        public Vector2 direction;

        //attack states
        Vector2 shootingposition;
        public Vector2 screenedge;
        public bool screenedgereached = false;
        public Rectangle fireballhitbox;  //(fireball hitbox)
        private int attackdamage;
        public static int damageadder = 0;
        public int Attackdamage
        {
            get { return attackdamage; } set { attackdamage = value; }
        }
        public bool enemyattack = false;
        //for item effect
        int widthsize = 0;
        int heightsize = 0;
        public static int sizeadder = 0;
        public static float speedadder = 0;
        public attack(Vector2 heroposition, Vector2 targetposition, Vector2 screenedge, string charactername)
        { //pass in essential field that helps generating an attack
            this.shootingposition = targetposition;
            this.charactername = charactername;
            switch (charactername)
            {
                case "hero":
                    this.speed = 10f + speedadder;
                    this.Attackdamage = 1 + damageadder;
                    widthsize = 30 + sizeadder;
                    heightsize = 30 + sizeadder;
                    break;
                case "swordhero":
                    this.speed = 15f + speedadder;
                    this.Attackdamage = 2 + damageadder;
                    widthsize = 50 + sizeadder;
                    heightsize = 50 + sizeadder;
```

```
                    break;
                case "gunhero":
                    this.speed = 30f + speedadder;
                    this.attackdamage = 4 + damageadder;
                    widthsize = 30 + sizeadder;
                    heightsize = 30 + sizeadder;
                    break;
            }
            position = heroposition;//shoot from heroposition
            //mouse position changes with camera moves, so apply a transform of mouse posstion and a invert of camera's
transformation matrix to make it the actual position on map
            Vector3 mapposition = Vector3.Transform(new Vector3(shootingposition.X, shootingposition.Y, 0), Matrix.In-
vert(gamestartstate.camera.transformationmat));
            this.shootingposition = new Vector2(mapposition.X, mapposition.Y);
            this.screenedge = screenedge;
            // normalize to make bullet travel towards the click on the screen
            direction = shootingposition - position;
            direction = normalizevectors(direction);
        }
        //overload
        public attack(bool specialenemy, Vector2 startingpos, Vector2 targetpos,Vector2 screenedge,ContentManager cm,string
enemyname)
        {
            specialenemyattack = cm.Load<Texture2D>(enemyname);
            speed = 12f;
            attackdamage = 6;
            widthsize = 30;
            heightsize = 30;
            enemyattack = specialenemy;
            this.screenedge = screenedge;
            position = startingpos;
            direction = targetpos - startingpos;
            direction = normalizevectors(direction);
        }

        public void update(game_inspector inspector,player player)
        {
            if (!enemyattack)
            {
                switch (charactername)
```

```
                {
                    case "hero":
                        if (position.X > 0 && position.Y > 0 && position.X < screenedge.X - fireball.Width && position.Y <
screenedge.Y - fireball.Height)
                        {
                            position += direction * speed;
                            fireballhitbox = new Rectangle((int)position.X, (int)position.Y, widthsize, heightsize);
                        }
                        else
                        {
                            screenedgereached = true;
                        }
                        break;
                    case "swordhero":
                        if (inspector.isitintersectingwalls(fireballhitbox, "all"))
                        {
                            screenedgereached = true;
                        }
                        else if (position.X > 0 && position.Y > 0 && position.X < screenedge.X - fireball.Width && posi-
tion.Y < screenedge.Y - fireball.Height)
                        {
                            position += direction * speed;
                            fireballhitbox = new Rectangle((int)position.X, (int)position.Y, widthsize, heightsize);
                            KeyboardState control = Keyboard.GetState();
                            if (control.IsKeyDown(Keys.E))
                            {
                                player.position = position;
                            }
                        }
                        else
                        {
                            screenedgereached = true;
                        }
                        break;
                    case "gunhero":
                        if (inspector.isitintersectingwalls(fireballhitbox, "all"))
                        {
                            screenedgereached = true;
                        }
```

```
                        else if (position.X > 0 && position.Y > 0 && position.X < screenedge.X - fireball.Width && posi-
tion.Y < screenedge.Y - fireball.Height)
                        {
                            position += direction * speed;
                            fireballhitbox = new Rectangle((int)position.X, (int)position.Y, widthsize, heightsize);
                        }
                        else
                        {
                            screenedgereached = true;
                        }
                        break;
                    }
                }
                else
                {
                    if (inspector.isitintersectingwalls(fireballhitbox, "all"))
                    {
                        screenedgereached = true;
                    }
                    else if (position.X > 0 && position.Y > 0 && position.X < screenedge.X - fireball.Width && position.Y <
screenedge.Y - fireball.Height)
                    {
                        position += direction * speed;
                        fireballhitbox = new Rectangle((int)position.X, (int)position.Y, widthsize, heightsize);
                    }
                    else
                    {
                        screenedgereached = true;
                    }
                }

            }

            public void draw(SpriteBatch spriteBatch)
            {
                if (!enemyattack)
                {
                    spriteBatch.Draw(fireball, position, Color.White);
                }
                else
```

```
            {
                spriteBatch.Draw(specialenemyattack, position, Color.White);
            }
        }
        Vector2 normalizevectors(Vector2 directionvector) //produces a stright line motion for attack using unit vectors
        {
            float magnitude = (float)Math.Sqrt(Math.Pow(directionvector.X,2)+ Math.Pow(directionvector.Y, 2));
            if(magnitude > 0)
            {
                return new Vector2(directionvector.X/magnitude, directionvector.Y/magnitude);
            }
            return new Vector2(0,0);
        }
    }
}


Section of main game loop where attack is created

mouse.mouseactivityupdate();
if (mouse.leftbuttonpressed) //(check mouse input to add attack, and attack speed control)
{
   attack fireball = new attack(player.position, mouse.leftclickedposition, new Vector2(7680, 4320), choices[1]);
                game_inspector.attackspeedcontrol(gametime, player, attacks, fireball); //(add fireballs and controll at-
tacking speed}


Section of game inspector class which control how the frequency of attack
public void attackspeedcontrol(GameTime gameTime,player player,List<attack> fireballs, attack fireball)  //control attack-
ing speed
        {
            if (player.attacktimer >= player.Attackspeedlimit) { player.Speedlimitreach = false; }
            if (player.Speedlimitreach == false)
            {
                fireballs.Add(fireball);
                player.Speedlimitreach = true;
                player.attacktimer = 0;
            }
            else { player.attacktimer += (float)gameTime.ElapsedGameTime.TotalSeconds; }
        }
```


Requirement E1. Enemy motion control all
Purpose: use traversal algorithm to create movement of enemy to player, complex path finding algorithm which can avoid walls and rooms
Class used: astaralgorithm, enemy, player
Techniques used: **Graph/Tree Traversal:** A* algorithm, an efficient pathfinding and graph traversal method. **Dynamic Generation of OOP**: Utilizes objects like Node, minheap, and hashsetfornodes dynamically based on game state. **Complex User-defined Algorithms:** Custom logic for calculating Manhattan distance, finding neighbors, and creating paths. **Resource Management**: Efficiently handles resources by dynamically adjusting pathfinding based on player-enemy distance to optimize performance. **Recursive Algorithms:** The CreatePath method uses recursion to backtrack from the target node to the start node, constructing the path stack. **Complex Data Structures**: Utilizes custom data structures such as minheap for the open set and hashsetfornodes for the closed set, optimizing search and insertion operations crucial for the A* algorithm's efficiency. **Efficient Pathfinding:** The A* algorithm's implementation, with its heuristic (Manhattan distance) and dynamic cost calculation (gcost, hcost) **Stack manipulation:** uses a stack as an output of the path, which was using recursion to generate a path from the end to start, and stack maintain the order of the positions in the path. **Boundary data handling** For **Requirement E1 a,** since each individual enemy calls the path finding function itself, they will have its own path based on its position, so they rarely overlap each other.

Astar algorithm class
`using Microsoft.Xna.Framework;`
`using Microsoft.Xna.Framework.Content;`
`using prototype.Map;`
`using System;`
`using System.Collections.Generic;`
`using System.Globalization;`
`using System.IO.IsolatedStorage;`
`using System.Linq;`
`using System.Security.Cryptography.X509Certificates;`
`using System.Text;`
`using System.Threading;`
`using System.Threading.Tasks;`
 	
`namespace prototype.enemies.enemy_treversal_algorithm`
`{`

```
internal class Astar_algorithm
{
    //153x86 grid
    public bool spawnfail;

    public Stack<Vector2> traversal(Vector2 current_pos, Vector2 Target_pos, Node[,] emptyGrid)
    {
        Node[,] Grid = emptyGrid;
        minheap openset = new minheap();
        hashsetfornodes closeset = new hashsetfornodes(400);
        int numofiteration = 0;
        spawnfail = false;
        // find the node position of current node and target node
        int currentnode_x = (int)(current_pos.X / 50);
        //if(currentnode_x > 0) { currentnode_x -= 1; }//to keep it the same index as the 2d array
        int currentnode_y = (int)(current_pos.Y / 50);
        //if(currentnode_y > 0) { currentnode_y -= 1; }
        int targetnode_x = (int)(Target_pos.X/ 50);
        //if(targetnode_x > 0) {  targetnode_x -= 1; }
        int targetnode_y = (int)(Target_pos.Y / 50);
        //if(targetnode_y > 0) { targetnode_y -= 1; }

        Node startnode = Grid[currentnode_x, currentnode_y];
        startnode.setgcost(0);
        Node endnode = Grid[targetnode_x, targetnode_y];
        closeset.insertnode(startnode);

        //this is intialization of the openset, and after I can begin the algorithm to find the target
        foreach (Node n in findneibours(currentnode_x, currentnode_y,targetnode_x,targetnode_y,true,Grid))
        {
            if(n == endnode)
            {
                Stack<Vector2> output = new Stack<Vector2>();
                output.Push(endnode.position);
                return output;
            }
            if (n.iswalkable)    //walkable openset, otherwise closeset
            {
                n.setparent(startnode);
                openset.addnode(n);
```

```
                }
                else
                {
                    closeset.insertnode(n);
                }
            }
            if(openset.nodes.Count == 0) //if the open set is empty, return a empty list for deleting enemy
            {
                spawnfail = true;
                return null;
            }
            //the algorithm begins here
            while (openset.nodes.Count > 0 && numofiteration < 6500)
            {//if the closest node found is the target node, out put the position of it
                numofiteration++;
                Node tempnode = openset.nodes[0];
                //this is a loop to test if it gointo a deadend
                List<Node> listtoaddtoopenset = new List<Node>();
                foreach (Node n in findneibours(tempnode.nodexindex_x, tempnode.nodeyindex_y, targetnode_x, targetnode_y,
false,Grid))
                {
                    if (n == endnode) //if the neibour is the target, use a stack as output of this path
                    {
                        endnode.setparent(tempnode);
                        Stack<Vector2> output = new Stack<Vector2>();
                        Node temp = endnode;
                        CreatePath(temp, startnode, output);
                        return output;
                    }
                    if(n.iswalkable && !closeset.has(n)) //if it's walkable and not in close set
                    {
                        if (openset.nodes.Contains(n)) //if the node already in open set
                        {
                            //if the gcost to walk from this neibour to a node in the open set is shorter than calculated
before, use this gcost and set
                            //this node's(already in openset) parent the node we currently at

                            Node nodetocheck = n; //this is the neiboured found (the one already in open set)
                            nodetocheck.addgcost(tempnode.gcost); //calculate the new gcost to walk from node we currently
at
```

```
                            int thenalreadytheregcost = n.gcost; //this node's original g cost
                            int thenewpathgcosttoadd = nodetocheck.gcost; //the new calculated gcost
                            if (thenewpathgcosttoadd < thenalreadytheregcost)
                            {
                                n.setgcost(thenewpathgcosttoadd); //set g cost for this node the new g cost calculated
                                n.sumcost();
                                n.setparent(tempnode); //sets the parent for this node(already there) the node we currently
add
                            }
                        }
                        else //walkable, and not in openset
                        {
                            n.setparent(tempnode);
                            n.addgcost(tempnode.gcost);
                            listtoaddtoopenset.Add(n);
                        }
                    }
                    else if (!n.iswalkable) //if not walkable add to close set
                    {
                        closeset.insertnode(n);
                    }
                }
                if (listtoaddtoopenset == null)//that means all of neibours is not walkable or already in close set, this
is a deadend
                {
                    closeset.insertnode(openset.removenode());//remove this node we are not going to this one
                }
                else
                {
                    closeset.insertnode(openset.removenode());//remove this node, it's fully explored, and put this node in
close set
                    foreach (Node n in listtoaddtoopenset)
                    {
                        openset.addnode(n);
                    }
                }
            }
            return null;
        }
```

```
        List<Node> findneibours(int x, int y,int targetx,int targety,bool addgcost, Node[,] Grid) //this will return the
valid neibours of a node it is currently on
        {
            List<Node> temp = new List<Node>();
            if(x-1 > 0)
            {
                if(y >= 0 && y < 86)
                {
                    temp.Add(Grid[x - 1, y]);
                    Grid[x - 1, y].addhcost(Manhhatandistance(x - 1, targetx, y, targety));
                    if (addgcost)
                    {
                        Grid[x - 1, y].addgcost(Grid[x,y].gcost);
                    }
                    Grid[x - 1, y].sumcost();
                }
            }
            if(x+1 < 153) these condition are use to handle boundary data
            {
                if (y >= 0 && y < 86)
                {
                    temp.Add(Grid[x + 1, y]);
                    Grid[x + 1, y].addhcost(Manhhatandistance(x + 1, targetx, y, targety));
                    if (addgcost)
                    {
                        Grid[x + 1, y].addgcost(Grid[x, y].gcost);
                    }
                    Grid[x + 1, y].sumcost();
                }
            }
            if(y-1 > 0)
            {
                if(x >= 0 && x < 153)
                {
                    temp.Add(Grid[x, y - 1]);
                    Grid[x, y - 1].addhcost(Manhhatandistance(x, targetx, y - 1, targety));
                    if (addgcost)
                    {
                        Grid[x, y - 1].addgcost(Grid[x, y].gcost);
                    }
```

```
                    Grid[x, y - 1].sumcost();
                }
            }
            if (y+1 <86)
            {
                if (x >= 0 && x < 153)
                {
                    temp.Add(Grid[x, y + 1]);
                    Grid[x, y + 1].addhcost(Manhhatandistance(x, targetx, y + 1, targety));
                    if (addgcost)
                    {
                        Grid[x, y + 1].addgcost(Grid[x, y].gcost);
                    }
                    Grid[x, y + 1].sumcost();
                }
            }
            return temp;
        }

        int Manhhatandistance(int x1, int x2, int y1, int y2) //this is the method to estimate the h cost between node
        {
            return(Math.Abs(x1-x2) + Math.Abs(y1-y2));
        }

        void CreatePath(Node n,Node startnode, Stack<Vector2> output)
        {
            output.Push(n.position);
            n = n.parent;
            if (n != startnode && n != null)
            {
                CreatePath(n, startnode, output); //recurstion
            }
        }
    }
}
```

How enemy is updated using this traversal algorithm, this is section of enemy class

```
public bool getdirections(Gridsystem grid,Vector2 playerposition,Rectangle playerhitbox,bool isitinroom, List<attack> at-
tacks,ContentManager cm)
```

```
        {
            playerposition = new Vector2(playerposition.X + 50, playerposition.Y + 50);
            double modulusdistancetoplayer = Math.Sqrt(Math.Pow(playerposition.X-position.X, 2) + Math.Pow(playerposi-
tion.Y- position.Y, 2));
            if(modulusdistancetoplayer > 2000) //this is how progrma adjust the frequency of calling path finding method
base on distance between player and enemy
            {
                return true;
            }
            else if(modulusdistancetoplayer > 1000 && modulusdistancetoplayer < 2000) { traversecontrol = 2; }
            else if(modulusdistancetoplayer > 500 && modulusdistancetoplayer < 1000) { traversecontrol = 1.5; }
            else if( modulusdistancetoplayer < 500) { traversecontrol = 1; }
            if(timefromlasttraversal >= traversecontrol && !enmeyhitbox.Intersects(playerhitbox) && !isitinroom)
            {// these conditions are, traverse cool down finish, enemy isn't already on player, player is not in room
                if (specialenemy) { attacks.Add(new attack(true, position, playerposition, new Vector2(7680, 4320), cm,
charactername)); }
                timefromlasttraversal = 0;
                directions = new Stack<Vector2>();
                directions = astaralgorithm.traversal(position, playerposition, grid.Getemptygrid());
                if (astaralgorithm.spawnfail == true) { return false; }
            }
            return true;
        }
        public void enemymotionupdate(GameTime gametime)//update enemy's velocity
        {
            if(timefromlasttraversal < 4) //just to make sure some enemy arene't met during the whole game play, so this
time won't get constantly updated, takes up too much memory space
            {
                timefromlasttraversal += gametime.ElapsedGameTime.TotalSeconds;
            }
            speedcontrol += gametime.ElapsedGameTime.TotalSeconds;
            if (ancientice) //this is the timer for a item ancient ice
            {
                ancienticetimer += gametime.ElapsedGameTime.TotalSeconds;
                if(ancienticetimer > 8)
                {
                    ancientice = false;
                }
            }
            if(speedcontrol >= speed)
```

```
            {
                if(directions != null)
                {
                    if (directions.Count != 0 && !ancientice)
                    {
                        position = directions.Pop();
                        speedcontrol = 0;
                        enmeyhitbox = new Rectangle((int)position.X, (int)position.Y, enemytexture.Width, enemytex-
ture.Height);
                    }
                }
            }
        }

Section of gamestartstate, shows how enemy is updated
//enemy update
            for (int i = 0; i < enemies.Count(); i++)
            {  //stop enemy going through walls
                if(!enemies[i].getdirections(gridsystem, player.position,player.playerhitbox,player.isinaroom,attacks,Con-
tent))
                {
                    enemies.RemoveAt(i);
                }
            }
            foreach(enemy e in enemies)
            {
                e.enemymotionupdate(gametime);
            }
            game_inspector.checkhealth(enemies,player,spritebatch,Content); //(remove dead enemy)
```

Requirement E1. Enemy motion control. b

Pupose: work alongside with astar algorithm, this is the heap sort use in a star algorithm for open set.

Class used: minheap

Techniques used:
Complex Data Structure Manipulation: Uses a list to dynamically manage nodes in a heap structure, optimizing pathfinding computations.
Heap Operations (upheap and downheap): Implements heapify operations to maintain the heap order property, keeping the nodes with lowest f cost at the root of a binary tree.

The minheap class also involves manipulating a binary tree through its upheap and downheap methods, for maintaining the heap's order property. This manipulation is crucial for the efficient organization of the heap, allowing for rapid insertion and removal of nodes, which directly impacts the performance of pathfinding algorithms like A*.

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Linq;
using System.Text;
using System.Threading;
using System.Threading.Tasks;

namespace prototype.enemies.enemy_treversal_algorithm
{
    internal class minheap
    {
        public List<Node> nodes = new List<Node>();
        Node output;
        public void swap(int a, int b)//swap 2 nodes in the tree
        {
            Node tempnode = nodes[a];
            nodes[a] = nodes[b];
            nodes[b] = tempnode;
        }
        public void upheap(int index)//when add item at the end, put it at right position int he list
```

```
        {
            while(index > 0)
            {
                if (nodes[index].fcost < nodes[(index - 1) / 2].fcost)
                {
                    swap(index, (index - 1) / 2);
                    index = (index - 1) / 2;
                }
                else
                {
                    break;
                }
            }
        }
        public void downheap(int index)//when remove item, restructure the bianry tree to correct order
        {
            int length = nodes.Count;
            while(index < length)
            {
                int currentindex = index;
                int leftchildindex = index * 2 + 1;
                int rightchildindex = index * 2 + 2;
                if (leftchildindex < length && nodes[leftchildindex].fcost < nodes[currentindex].fcost)
                {
                    currentindex = leftchildindex;
                }
                if (rightchildindex < length && nodes[rightchildindex].fcost < nodes[currentindex].fcost)
                {
                    currentindex = rightchildindex;
                }
                if(currentindex != index)
                {
                    swap(currentindex, index);
                    index = currentindex;
                }
                else
                {
                    break;
                }
            }
```

```
        }
        public Node removenode()//put swap first and last node, remove the last node, and restructure the tree
        {
            output = nodes[0];
            swap(0,nodes.Count()-1);
            nodes.RemoveAt(nodes.Count()-1);
            downheap(0);
            return output;
        }
        public Node recordthisnode()
        {
            removenode();
            return output;
        }
        public void addnode(Node n)//inserting node
        {
            nodes.Add(n);
            upheap(nodes.Count - 1);
        }
    }
}
```

Requirement E1. Enemy motion control. b

Purpose: work alongside with astar algorithm, this is the hash table use in a star algorithm for close set.

Class used: hashsetfornodes

Techniques used:

Hash Table: Employs a hash table to store nodes, optimizing lookup, insertion, and check if a node was there

Custom Hash Function: Uses a custom hash function (gethashidex) based on node grid positions,

Collision Handling: Implements lists within hash table slots to handle collisions.

Complex data structure: use of array of list.

```
using Microsoft.Xna.Framework.Content;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Xml;

namespace prototype.enemies.enemy_treversal_algorithm
{
    internal class hashsetfornodes
    {
        List<Node>[] hashtable;
        int size;
        public hashsetfornodes(int size)
        {
            this.size = size;
            hashtable = new List<Node>[size];
            for (int i = 0; i < size; i++)
            {
                hashtable[i] = new List<Node>();
            }
        }
        int gethashidex(Node node)//calculate the index of this node using its grid index * a prime number
```

```
        {
            return (node.nodexindex_x * 23 + node.nodeyindex_y) % size;
        }
        public void insertnode(Node n)
        {
            int index = gethashidex(n);
            hashtable[index].Add(n);
        }
        public bool has(Node n)//check whether the hashset contians this node
        {
            if (hashtable[gethashidex(n)].Contains(n))
            {
                return true;
            }
            else
            {
                return false;
            }

        }

    }
}
```

Requirement E2 all. Enemy attack control, Requirement G2.

Purpose:
To allow general enemy to cause damage to player when they intersect, and allow special enemy to perform attacks just like player

Class used:
Attak, enemy, gameinspector

Techniques used:
Collision Detection and Handling: Implementing collision detection for attacks, affecting game entities' states (health, position)
Polymorphism: differentiating behaviors between player and enemy attacks through method overloading, allowing for dynamic interaction based on the entity type.
OOP : passing objects as parameters for collision detection to differentiate between players and enemy, encapsulate individual characters data.

Section of code in enemy class

```
public bool getdirections(Gridsystem grid,Vector2 playerposition,Rectangle playerhitbox,bool isitinroom, List<attack> at-
tacks,ContentManager cm)
        {
            playerposition = new Vector2(playerposition.X + 50, playerposition.Y + 50);
            double modulusdistancetoplayer = Math.Sqrt(Math.Pow(playerposition.X-position.X, 2) + Math.Pow(playerposi-
tion.Y- position.Y, 2));
            if(modulusdistancetoplayer > 2000) //this is how 74rogram adjust the frequency of calling path finding method
base on distance between player and enemy
            {
                return true;
            }
            else if(modulusdistancetoplayer > 1000 && modulusdistancetoplayer < 2000) { traversecontrol = 2; }
            else if(modulusdistancetoplayer > 500 && modulusdistancetoplayer < 1000) { traversecontrol = 1.5; }
            else if( modulusdistancetoplayer < 500) { traversecontrol = 1; }
            if(timefromlasttraversal >= traversecontrol && !enmeyhitbox.Intersects(playerhitbox) && !isitinroom)
            {// these conditions are, traverse cool down finish, enemy isn't already on player, player is not in room
                if (specialenemy) { attacks.Add(new attack(true, position, playerposition, new Vector2(7680, 4320), cm,
charactername)); } //allow special enemy to throw attacks
                timefromlasttraversal = 0;
                directions = new Stack<Vector2>();
                directions = astaralgorithm.traversal(position, playerposition, grid.Getemptygrid());
                if (astaralgorithm.spawnfail == true) { return false; }
            }
            return true;
        }
```

How attack class differentiate enemy attack and player attack, section of code in attack class

```
//overload
        public attack(bool specialenemy, Vector2 startingpos, Vector2 targetpos,Vector2 screenedge,ContentManager cm,string
enemyname)
        {
            specialenemyattack = cm.Load<Texture2D>(enemyname);
            speed = 12f;
            attackdamage = 6;
            widthsize = 30;
            heightsize = 30;
            enemyattack = specialenemy;
            this.screenedge = screenedge;
            position = startingpos;
            direction = targetpos - startingpos;
            direction = normalizevectors(direction);
        }
```

Collision detection for attack from special enemy and general enemy, section of code in game inspector

```
public void attackhitcontrol(List<enemy> enemies,List<attack> attacks, player player, GameTime gametime)   //collision de-
tection, check for any damage
        {
            foreach (enemy e in enemies)
            {
                for(int i =0; i<attacks.Count; i++)//check attack hit enemy
                {//check attack hit enemy
                    if (attacks[i].fireballhitbox.Intersects(e.Flyingpurplehitbox) && !attacks[i].enemyattack)
                    {
                        e.health -= attacks[i].Attackdamage;
                        damagecaused += attacks[i].Attackdamage; //record player's damage
                        attackeffectpositionlist.Add(e.position);
                        attacks.RemoveAt(i);
                    }
                    if(i< attacks.Count&&attacks[i].fireballhitbox.Intersects(player.playerhitbox) && attacks[i].ene-
myattack)
                    {//check special enemy attack hits player
                        player.Health -= attacks[i].Attackdamage;
                        attackeffectpositionlist.Add(e.position);
```

```
                        attacks.RemoveAt(i);
                    }
                }
                if(e.Flyingpurplehitbox.Intersects(player.playerhitbox)&& temptimer_damage >= 0.1)//check general enemy hit
hero(also limit the speed of attack, not decreasing health every frame)
                {
                    player.Health -= e.damage;
                    temptimer_damage = 0;
                }
            }
            temptimer_damage += gametime.ElapsedGameTime.TotalSeconds;
        }
```

Requirement E3 all, E4, Enemy Property, player property

Purpose:

Having different enemies in the game, and uses dictionary to store all the stats of enemy, create variety of different enemy game with different characteristic.

Using the same techniques, stores players property

Class used: enemydictionary, enemy, player

Techniques used:

Resource Management: loading and assigning graphical resources efficiently.

Dynamic Data Structures: Employs dictionaries and lists to manage enemy attributes and names

Data-driven Design: the data in dictionary can be managed with ease and provide central management of behaviours of enemy

Code for enemydictionary class:

```
using Microsoft.Xna.Framework.Content;
using Microsoft.Xna.Framework.Graphics;
using System.Collections.Generic;
namespace prototype.enemies
{
    internal class enemydictionary
    {
        //appearance
        public static Texture2D flyingpurple;
        public static Texture2D purplemushroom;
        public static Texture2D wizard;
        public static Texture2D nianmonster;
        public static Texture2D forest1;
        public static Texture2D forest2;
        public static Texture2D cave1;
        public static Texture2D cave2;
        public static Texture2D town1;
        public static Texture2D town2;

        //load content
        public static void enemycontentload(ContentManager Content)
        {
            flyingpurple = Content.Load<Texture2D>("enemy");
            purplemushroom = Content.Load<Texture2D>("purplemashroom");
            wizard = Content.Load<Texture2D>("wizard");
            nianmonster = Content.Load<Texture2D>("nianmonster");
            forest1 = Content.Load<Texture2D>("forest1");
```

```
            forest2 = Content.Load<Texture2D>("forest2");
            cave1 = Content.Load<Texture2D>("cave1");
            cave2 = Content.Load<Texture2D>("cave2");
            town1 = Content.Load<Texture2D>("town1");
            town2 = Content.Load<Texture2D>("town2");

        }

        //list of their name
        public static List<string> enemynames = new List<string> { "flyingpurple", "purplemushroom", "wizard", "nianmon-
ster", "forest1", "forest2", "cave1", "cave2", "town1", "town2" };
        //enemy database
        public static Dictionary<string, string[]> enemystats = new Dictionary<string, string[]>()
        {
            // structure: health, damage,speed, apearance,special,coint drop + throw attack damage
            {"flyingpurple", new string[] {"1","1","0.3","enemy","false","1"} },
            {"purplemushroom", new string[] {"3","1","0.5","purplemushroom","false","1"} },
            {"wizard", new string[] {"2","1.28", "1.5625", "wizard","false","2"} },
            {"nianmonster",new string[] {"3", "1.5625", "2","nianmonster","false","2"} },
            {"forest1" , new string[] {"5","3", "3", "forest1", "true","5","6"} },
            {"forest2" , new string[] {"5","3", "3", "forest2", "true","5","6"} },
            {"cave1" , new string[] {"5","3", "3", "cave1", "true","5","6"} },
            {"cave2" , new string[] {"5","3", "3", "cave2", "true","5","6"} },
            {"town1" , new string[] {"5","3", "3", "town1", "true","5","6"} },
            {"town2" , new string[] {"5","3", "3", "town2", "true","5","6"} },
        };

    }
}
```

How enemy uses these values, section of enemy class code

```
public enemy(int x, int y, string enemyname)//get all the stats for an enemy
        {
            position = new Vector2(x, y);
            health = Convert.ToInt32(enemydictionary.enemystats[enemyname][0]);
            damage = Convert.ToDouble(enemydictionary.enemystats[enemyname][1]);
            speed = double.Parse(enemydictionary.enemystats[enemyname][2]);
            coindrop = Convert.ToInt16(enemydictionary.enemystats[enemyname][5]);
            specialenemy = (enemydictionary.enemystats[enemyname][4].Length == 4);
            string temp_name = enemydictionary.enemystats[enemyname][3];
```

```
            switch (temp_name)
            {
                case "enemy":
                    enemytexture = enemydictionary.flyingpurple;
                    enmeyhitbox = new Rectangle((int)position.X, (int)position.Y, enemytexture.Width, enemytexture.Height);
                    break;
                case "purplemushroom":
                    enemytexture = enemydictionary.purplemushroom;
                    enmeyhitbox = new Rectangle((int)position.X, (int)position.Y, enemytexture.Width, enemytexture.Height);
                    break;
                case "wizard":
                    enemytexture = enemydictionary.wizard;
                    enmeyhitbox = new Rectangle((int)position.X, (int)position.Y, enemytexture.Width, enemytexture.Height);
                    break;
                case "nianmonster":
                    enemytexture = enemydictionary.nianmonster;
                    enmeyhitbox = new Rectangle((int)position.X, (int)position.Y, enemytexture.Width, enemytexture.Height);
                    break;
                case "forest1":
                    charactername = "forestattack";
                    enemytexture = enemydictionary.forest1;
                    enmeyhitbox = new Rectangle((int)position.X, (int)position.Y, enemytexture.Width, enemytexture.Height);
                    break;
                case "forest2":
                    charactername = "forestattack";
                    enemytexture = enemydictionary.forest2;
                    enmeyhitbox = new Rectangle((int)position.X, (int)position.Y, enemytexture.Width, enemytexture.Height);
                    break;
                case "town1":
                    charactername = "townattack";
                    enemytexture = enemydictionary.town1;
                    enmeyhitbox = new Rectangle((int)position.X, (int)position.Y, enemytexture.Width, enemytexture.Height);
                    break;
                case "town2":
                    charactername = "townattack";
                    enemytexture = enemydictionary.town2;
                    enmeyhitbox = new Rectangle((int)position.X, (int)position.Y, enemytexture.Width, enemytexture.Height);
                    break;
                case "cave1":
                    charactername = "caveattack";
```

```
                    enemytexture = enemydictionary.cave1;
                    enmeyhitbox = new Rectangle((int)position.X, (int)position.Y, enemytexture.Width, enemytexture.Height);
                    break;
                case "cave2":
                    charactername = "caveattack";
                    enemytexture = enemydictionary.cave2;
                    enmeyhitbox = new Rectangle((int)position.X, (int)position.Y, enemytexture.Width, enemytexture.Height);
                    break;
            }
        }

Section of code for player
Dictionary<string, string[]> characters = new Dictionary<string, string[]>()
        {
            // structure: health, damage,speed, apearance,method(boolean)
            {"hero", new string[] {"100","0.35","8.7"} }, //(health,attackspeed,speed)
            {"swordhero", new string[] {"10","0.4","10"} },
            {"gunhero", new string[] {"80","0.8","7"} },

        };
        public player(float x, float y,string charactername)
        {
            position = new Vector2(x, y);
            this.name = charactername;
            this.health = Convert.ToDouble(characters[charactername][0]);
            this.attackspeedlimit = float.Parse(characters[charactername][1]);
            this.speed = float.Parse(characters[charactername][2]);

        }
```

Requirement G1. Map generation. all

Purpose:
Generate the map, store the information about rooms, doors walls and chest for future use in code

Class used:
Mapgeneratino, chest, rooms

Techniques used:
Complex user defined algorithm: complex structure of room generation with labelling rooms and walls relationship.
Dynamic and Random Generation: Rooms and chests are dynamically generated with random attributes,
Randomization for Game Element Variability: Implements randomness in the generation of room sizes, positions, and chest contents, adding unpredictability and complexity to the game environment.
Graphical User Interface (GUI) Management: Drawing textures and text for rooms and chests,
Collision Detection for Room Overlap: basic spatial awareness by preventing room overlaps, crucial for map integrity and player navigation.

```
Section of mapgeneration class which creates rooms and put chest in room
public Mapgeneration(GraphicsDeviceManager g, ContentManager C, SpriteBatch s)
        {
            this.Content = C;
            this.spritebatch = s;
            //load all items texture
            load();
            //this constructor will generate a list of rooms, without overlapping each other
            rooms = new List<Rooms>();
            Random a = new Random();//number of rooms
            Random b = new Random();  //settings for each room
            Rooms temproom = new Rooms(b.Next(1000, 1500), b.Next(800, 1000), new Vector2(b.Next(200, 5980), b.Next(200,
3120)), Content,0);
            rooms.Add(temproom);
            for (int i = 0; i < a.Next(4,9); i++) //generate rooms
            {
                bool overlap = true;
                whilestart: //use of labels
                while (overlap)
                {
                    temproom = new Rooms(b.Next(1000, 1500), b.Next(800, 1000), new Vector2(b.Next(200, 5980), b.Next(200,
3120)), Content,i+1);
                    foreach (Rooms r in rooms)
                    {
                        if (temproom.roomspace.Intersects(r.roomspace))
                        {
```

```
                            goto whilestart;
                        }
                    }
                    overlap = false;
                }
                rooms.Add(temproom);
            }
            //code here is handling chest and item
            chests = new chest[rooms.Count];
            int counter = 0;
            foreach(Rooms r in rooms)
            {
                chest c = new chest(new Vector2(r.roomspace.X + r.roomspace.Width / 2, r.roomspace.Y + r.roomspace.Height /
2), r.roomnumber, items[a.Next(0,7)]);
                chests[counter] = c; counter++;
            }
        }
```

Room class which shows the structure of the rooms

```
using Microsoft.Xna.Framework;
using Microsoft.Xna.Framework.Content;
using Microsoft.Xna.Framework.Graphics;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace prototype.Map
{
    internal class Rooms
    {
        //room position and size
        Texture2D roomfloortexture;
        Texture2D rooftexture;
        Texture2D doorhorizontaltexture;
        Texture2D doorverticaltexture;
        Texture2D walltexture;
        public Vector2 roompostition;
        public Rectangle roomspace;
```

```
        public int roomnumber;
        //room structure
        Rectangle horizontalwall1;
        Rectangle horizontalwall3;
        Rectangle verticalwall2;
        Rectangle verticalwall4;
        public Rectangle door;
        public Rectangle[] horizontalwalls;
        public Rectangle[] verticalwalls;
        public Rectangle[] walls;
        int doorside;
        public Rooms(int width, int height, Vector2 roomposition, ContentManager cm, int roomnumber)
        {
            this.roompostition = roomposition;
            roomspace = new Rectangle((int)roomposition.X, (int)roomposition.Y, width, height);
            //load textures
            roomfloortexture = cm.Load<Texture2D>("floortexture");
            doorhorizontaltexture = cm.Load<Texture2D>("doorhorizontal");
            doorverticaltexture = cm.Load<Texture2D>("doorvertical");
            walltexture = cm.Load<Texture2D>("walltexture");
            rooftexture = cm.Load<Texture2D>("rooftexture");
            //randomly select which side is the door in the room
            Random r = new Random();
            doorside = r.Next(1, 5);
            //generate door and walls
            wallgenerate();
            doorgenerate();
            this.roomnumber = roomnumber;
        }
        public void wallgenerate()//generate walls aroudn the room space, 2 vertical and 2 horizontal
        {
            horizontalwalls = new Rectangle[2];
            verticalwalls = new Rectangle[2];
            walls = new Rectangle[4];
            //horizontal walls  //ababa
            horizontalwall1 = new Rectangle((int)roompostition.X, (int)roompostition.Y - 50, roomspace.Width, 50);
            horizontalwalls[0] = horizontalwall1;
            walls[0] = horizontalwall1;
            horizontalwall3 = new Rectangle((int)roompostition.X, (int)(roompostition.Y + roomspace.Height),
roomspace.Width, 50);
```

```
            horizontalwalls[1] = horizontalwall3;
            walls[1] = horizontalwall3;
            //vertical walls
            verticalwall4 = new Rectangle((int)(roompostition.X - 50), (int)(roompostition.Y-50),50,roomspace.Height+100);
            verticalwalls[0] = verticalwall4;
            walls[2] = verticalwall4;
            verticalwall2 = new Rectangle((int)(roompostition.X + roomspace.Width), (int)(roompostition.Y-50), 50,
roomspace.Height +100);
            verticalwalls[1] = verticalwall2;
            walls[3] = verticalwall2;
        }
        public void doorgenerate()//randomly place the door on one of the wall
        {
            switch (doorside)
            {
                case 1://door on wall 1
                    door =  new Rectangle((int)(roompostition.X + (roomspace.Width / 2) - 100), (int)(roompostition.Y -
50), 200, 50);
                    break;
                case 2://door on wall 2
                    door = new Rectangle((int)(roompostition.X + roomspace.Width), (int)(roompostition.Y +
(roomspace.Height / 2) - 100), 50, 200);
                    break;
                case 3: // door on wall3
                    door = new Rectangle((int)(roompostition.X + (roomspace.Width / 2) - 100), (int)(roompostition.Y +
roomspace.Height), 200, 50);
                    break;
                case 4: //door on wall4
                    door = new Rectangle((int)(roompostition.X - 50),(int)(roompostition.Y+(roomspace.Height/2)-
100),50,200);
                    break;
            }
        }
        public void draw(SpriteBatch sb, bool isplayerinroom)
        {
            if(isplayerinroom)
            {
                sb.Draw(roomfloortexture, roompostition, roomspace, Color.White);//draw the floor
            }
            else
```

```
            {
                sb.Draw(rooftexture, roompostition, roomspace, Color.White); // only draw roof when player isn't in room
            }
            foreach (Rectangle r in walls)//draw the walls
            {
                sb.Draw(walltexture, new Vector2(r.X, r.Y), r, Color.White);
            }
            switch (doorside)//draw the doors
            {
                case 1://this is for the horizontal wall texture
                case 3:
                    sb.Draw(doorhorizontaltexture, new Vector2(door.X, door.Y), Color.White);
                    break;
                case 2://this is for the vertical texture
                case 4:
                    sb.Draw(doorverticaltexture, new Vector2(door.X, door.Y), Color.White);
                    break;
            }


        }

    }

}
```

Chest class which shows information about the chest

```
using Microsoft.Xna.Framework;
using Microsoft.Xna.Framework.Content;
using Microsoft.Xna.Framework.Graphics;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Numerics;
using System.Text;
using System.Threading.Tasks;
using Vector2 = Microsoft.Xna.Framework.Vector2;

namespace prototype.Map
{
```

```
    internal class chest
    {
        //information for a chest
        Vector2 chestposition;
        public static Texture2D chesttexture;
        public static SpriteFont chestfont;
        public int chestvalue;
        public Rectangle chestspace;
        public int roomnumberin;
        public string itemname;
        public chest(Vector2 chestposition,int roomnumber, string itemname)
        {
            this.chestposition = chestposition;
            Random r = new Random();
            chestvalue = r.Next(10, 20); //randomly generate chest value
            chestspace = new Rectangle((int)chestposition.X, (int)chestposition.Y, 80, 80);
            roomnumberin = roomnumber;
            this.itemname = itemname;
        }
        public void draw(SpriteBatch sb)
        {
            sb.Draw(chesttexture,chestposition,Color.White);
            sb.DrawString(chestfont, chestvalue.ToString(), new Vector2(chestposition.X - 100, chestposition.Y - 130),
Color.Cyan, 0f, Vector2.Zero, 3, SpriteEffects.None, 0f);
            sb.DrawString(chestfont, itemname, new Vector2(chestposition.X-100,chestposition.Y-80), Color.Red, 0f, Vec-
tor2.Zero, 3, SpriteEffects.None, 0f);
        }
    }
}
```

Requirement G3 a,b Camera system

Purpose:

Provide a camera system in game which move with player and keep player at centre of the screen.

Class used:

Camera, maingame

Techniques used:

Dynamic update camera based on player class

View Management: Dynamically adjusts the camera's position based on the player's position, ensuring the player remains centered on the screen.

Clamping Mechanism: Utilizes MathHelper.Clamp to restrict the camera's movement within specified bounds, enhancing gameplay by preventing the camera from showing areas outside the game world.

Math Matrix Transformations: Employs Matrix.CreateTranslation for camera movement, allowing for smooth transitions and a focused view of the game environment.

Camera class

```
using Microsoft.Xna.Framework;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace prototype.Utilities_classes
{
    internal class Camera
    {
        public Matrix transformationmat;
        public Vector2 position;
        public void update(player player)
        {
            position = player.position;
            //clamp method keep the positions of the camera focus in this range, so I doesn't show the view beyond the
background size
            position.X = MathHelper.Clamp(position.X, 960, 6720);
            position.Y = MathHelper.Clamp(position.Y, 540, 3780);
            //first translation moves the map opposite direction to the player, second translation kept player at center of
the screen
            transformationmat = Matrix.CreateTranslation(new Vector3(-position.X, -position.Y, 0)) * Matrix.CreateTransla-
tion(new Vector3(960,480, 0));
```

```
        }

    }
}
```

How camera is applied in main game rendering, section of code in main game program

```
protected override void Draw(GameTime gametime)
        {
            GraphicsDevice.Clear(Color.Black);
            var currentstate = gameState.peek();
            if (currentstate is Menu)
            {
                spritebatch.Begin();
                gameState.peek().draw(gametime);
                spritebatch.End();
                base.Draw(gametime);
            }
            else if(currentstate is gamestartstate)
            {
                //draw the game world
                spritebatch.Begin(SpriteSortMode.Deferred, BlendState.AlphaBlend, null, null, null, null, gamestart-
state.camera.transformationmat);
                gameState.peek().draw(gametime);
                spritebatch.End();
                //draw the UI
                spritebatch.Begin();
                gameState.peek().drawUI();
                spritebatch.End();
                base.Draw(gametime);
            }
            else if(currentstate is gameend)
            {
                spritebatch.Begin();
                gameState.peek().draw(gametime); spritebatch.End();
            }
        }
```

Requirement G4. Enemy generation, Requirement G7 difficulties

Purpose:

Generates enemy at random position in game world, and generates different types of enemies, avoid generating them in rooms.

Class used:

gameinspector

Techniques used:

Dynamic Difficulty Adjustment: Adjusting enemy spawn rates and types based on game duration and difficulty level

Complex Data Structure Manipulation: Employing queues to manage spawn points and lists to handle active enemies, also manipulate data in enemydictionary for lateron enemy generation.

Conditional Logic for Game State Management: Implementing checks on game time and difficulty levels to dynamically adjust game behaviour

Dynamic generation of enemy : dynamically generates enemy randomly in map without them being in any rooms or walls.

Section of code responsible for initial generation of enemy in the first minute of game and later generation with special enemies, gameinspector

```
public game_inspector(Mapgeneration map, string mapname)//initial spawn of enemy
        {
            this.map = map;
            for (int i = 0; i < 25; i++) // push 25 position into a queue that is not intersecting any walls, and enemy
will spawn at these positions
            {
                Rectangle temprectangle = new Rectangle(spawnenemyrandom.Next(0, 7200), spawnenemyrandom.Next(0, 4000), 50,
50);
                while (isitintersectingwalls(temprectangle, "rooms"))
                {
                    temprectangle = new Rectangle(spawnenemyrandom.Next(0, 7200), spawnenemyrandom.Next(0, 4000), 50, 50);
                }
                enemygenration.Enqueue(new Vector2(temprectangle.X, temprectangle.Y));

            }

            this.mapname = mapname;
        }

public void enemyspawn(List<enemy> enemies) //spawn enemies according to time and difficulties
        {
            if(totalgametimer.TotalMinutes > 1)// when game pass 1 minitue
            {
                if(adjustdifficulties>= 30) // every 30 seconds increase the difficulties
                {
```

```
                    updateenemystats();
                    adjustdifficulties = 0;
                    difficulties++;
                }
                if(temptimer_enemyspawn >= 5)//spawn enemy 5 seconds
                {
                    temptimer_enemyspawn = 0;
                    for (int i = 0; i < difficulties; i++)
                    {
                        Vector2 tempposition = enemygenration.Dequeue();
                        Random r = new Random();
                        if(r.Next(2) == 0)
                        {
                            enemies.Add(new enemy((int)tempposition.X, (int)tempposition.Y, enemydictionary.enemy-
names[spawnenemyrandom.Next(0,4)])); //use dequeue enemy when spawn
                        }
                        else
                        {
                            switch (mapname)
                            {
                                case "background":
                                    enemies.Add(new enemy((int)tempposition.X, (int)tempposition.Y, enemydictionary.enemy-
names[spawnenemyrandom.Next(4,6)]));
                                    break;
                                case "cave":
                                    enemies.Add(new enemy((int)tempposition.X, (int)tempposition.Y, enemydictionary.enemy-
names[spawnenemyrandom.Next(6, 8)]));
                                    break;
                                case "town":
                                    enemies.Add(new enemy((int)tempposition.X, (int)tempposition.Y, enemydictionary.enemy-
names[spawnenemyrandom.Next(8, 10)]));
                                    break;
                            }
                        }
                    }
                }
            }
            else
            {
                if (temptimer_enemyspawn >= 5)
```

```
                {
                    for (int i = 0; i < 2; i++)
                    {
                        Vector2 tempposition = enemygenration.Dequeue();
                        enemies.Add(new enemy((int)tempposition.X, (int)tempposition.Y, enemydictionary.enemy-
names[spawnenemyrandom.Next(difficulties - 2, difficulties)])); //use dequeue enemy when spawn
                    }
                    temptimer_enemyspawn = 0;
                }
            }
        }

Section of code in game inspector for increase difficulties
void updateenemystats() //increase the health, damage, and coin drop from the enemy
        {
            foreach(var key in enemydictionary.enemystats.Keys.ToList())
            {
                var stats = enemydictionary.enemystats[key];
                stats[0] = Convert.ToString(Convert.ToInt16(stats[0]) + 3);
                stats[1] = Convert.ToString(Convert.ToInt16(stats[1]) + 1);
                stats[5] = Convert.ToString(Convert.ToInt16(stats[5]) + 2);
            }
        }
```

Requirement G6.a. Items

Purpose:
Applies the effect of item when player successfully purchased them

Class used:
map generation

Techniques used:
Resource Management: Dynamically adjusts player and attack attributes based on the items obtained from chests
Conditional Logic for Gameplay Mechanics: uses switch case statement to apply item effects, reflecting complex logic tailored to game rules and mechanics.
OOP: encapsulate variables for player, enemy and attacks, and dynamically update them based on item.
GUI design: dynamically add texture of items into a List to display on screen for player to know item they currently owns.

Section of code in mapgeneration class

```
public void update(player player, List<attack> attacks, List<enemy> enemies) //check player open a chest and add on according effect
        {
            mouse.mouseactivityupdate(true);

            if(mouse.leftbuttonpressed)
            {
                chest c;
                foreach (chest temp in chests)
                {
                    if (temp.roomnumberin == player.roomnumberin)
                    {
                        c = temp;
                        Vector3 mapposition = Vector3.Transform(new Vector3(mouse.leftclickedposition.X,
mouse.leftclickedposition.Y, 0), Matrix.Invert(gamestartstate.camera.transformationmat));
                        Vector2 clickposition = new Vector2(mapposition.X, mapposition.Y);
                        if (new Rectangle((int)clickposition.X,(int) clickposition.Y, 1, 1).Intersects(c.chestspace) &&
player.money >= c.chestvalue)
                        {
                            player.money -= c.chestvalue;
                            switch (c.itemname)
                            {
                                case ("powerrunes"): //increase damage by 4
                                    foreach(attack a  in attacks) { a.Attackdamage += 4; }
                                    itemtodraw.Add(powerrunes);
                                    itemposition.Add(new Vector2(drawingposition, 0));
```

```
                drawingposition += 50;
                break;
            case ("flyingboots"): //increases player's moving speed by 1
                player.Speed += 1;
                itemtodraw.Add(flyingboots);
                itemposition.Add(new Vector2(drawingposition, 0));
                drawingposition += 50;
                break;
            case ("gunpowder"): //increases attacks flying speed of player by 1
                attack.speedadder += 5f;
                itemtodraw.Add(gunpowder);
                itemposition.Add(new Vector2(drawingposition, 0));
                drawingposition += 50;
                break;
            case ("speedgear"): //increases attack frequency by 0.05 secibds
                player.Attackspeedlimit -= 0.05f;
                itemtodraw.Add(speedgear);
                itemposition.Add(new Vector2(drawingposition, 0));
                drawingposition += 50;
                break;
            case ("enlargepotion"): //make the hitbox of attacks bigger
                attack.sizeadder += 10;
                itemtodraw.Add(enlargepotion);
                itemposition.Add(new Vector2(drawingposition, 0));
                drawingposition += 50;
                break;
            case ("ancientice"): //freeze enemy for 8 seconds
                foreach(enemy e in enemies)
                {
                    e.ancientice = true;
                }
                itemtodraw.Add(ancientice);
                itemposition.Add(new Vector2(drawingposition, 0));
                drawingposition += 50;
                break;
            case ("Demonforce"): // remove all enemy currently inthe game
                enemies.Clear();
                itemtodraw.Add(Demonforce);
                itemposition.Add(new Vector2(drawingposition, 0));
                drawingposition += 50;
```

```
                                    break;
                                }
                            }
                            break;
                        }
                    }
                }
            }
```

Testing

About the tests:

I have in total of 21 tests for my game, I have done it to each of requirement in my game. in the test videos, you might see some test video is also satisfying requirements other the requirement stated in the second column, but for the most precise test referring back the requirement, to be most requirement specific, I have made lots of videos but each is specific to a requirement, so you might find some of the videos are repetitive for a content in game, but all of them are unique, and it is recorded for only the requirement state in column 2.

Table showing:

Test Number	Requirement(s) being tested	Test Description	Test Data	Expected Result	Video Timestamp	Result
1	**Requirement B1. User interface all**	Test if menu stage of my game is working, does it display the message for player to choose, and whether it hits all requirement in Requirement B1	Menu class	Menu proceed smoothly with button all functioning, and able to conduct page changes, and whether it remember player's choices, and displays the right message	See video test 1	successful
2	**Requirement B2. Options available to select from the menu. all**	Test if menu provides required option for player to choose and play	Menu class	Options correctly displayed and player can choose by clicking buttons. Message displayed should include start page, map choosing page, character choosing page.	See video test 1	successful
3	**Requirement B3. Content that is represented by the choice on the screen. a**	Test if game correctly proceed to the map player has chosen	Menu class	The game start with the right map player has chosen	See video test2	successful
4	**Requirement B3. Content that is represented by the choice on the screen. b**	Test if menu correctly proceed to game with right player chosen, and whether player	Menu class	Game correctly applies the character player chosen, and attack as described	See video test3	successful

		can attack as described.				
5	**Requirement P1. Motion Control of player a, b**	Test if program listen to keyboard input and allow player to move in all direction	Player class	program correctly listen to keyboard input and allow player to move in all direction	See video test4	successful
6	**Requirement P1. Motion Control of player c**	Test if player's speed can change during game play	Player class	player's speed can change during game play	See video test 5	successful
7	**Requirement P2. Attack Control of player all**	Test to see functionality of attack player perform, whether it cover all requirement in P2	Attack class	Attack is directed to right direction, mouse click successfully detected, frequency of attack can change and they have a value of damage	See test video 6	successful
8	**Requirement P3. Player's property: all**	Test if player has the property health, speed, and money	Player class	Player does have the required property and they can be changed	See test video 7	successful
9	**Requirement E1. Enemy motion control a all, b 1,3**	Test if enemy can move as described in requirement, and whether the path finding algorithm work	Enemy class Astar algorithm class	Enemy does move in all direction, and it can find a path to player, also evidence for each individual enemy calculating its own path	See test video 8	successful
10	**Requirement E1. b. 2**	Test if enemy can avoid obstacles	Enemy class Astar algorithm class Minheap class Node class	Enemy can get to player position and void rooms in its way here	See test video 9	Successful

			Grid class Hashtable class			
11	**Requirement E2. Enemy attack control.** **all**	Test if different enemies are able to make attack and cause damage to player	Enemy class Attack class	General enemy (enemy which spawn in first minute, appear in all maps) can cause damage when it intersects player, special enemy can shoot bullet just like player, and can cause damage	See test video 10	successful
12	**Requirement E3. Enemy Property** **All**	Test if enemy has different attribute correctly applied to its stats such as health and coindrop etc.	Enemy class Enemy dictionary class	Editable attribute that can change the stats of enemy in game, there should be value represent enemy's health, speed, appearance, and coindrop which is the money they drop after killed	See test video 11	Successful
13	**Requirement E4. a**	Test if game actually generate different types of enemies, and whether special enemy are created after one minute	Enemy class Enemy dictionary class	Different types of enemies randomly generated during game play, includes general enemy and special enemies.	See test video 12	Successful
14	**Requirement G1. Map generation a**	Test if rooms in the map is successfully generated as requirement described	Mapgeneration class Rooms class	The game randomly generates rectangle room, for each of them, they should all have a floor, 4 walls around it, a door allow player to enter, position of the room should be random, also the size, and the door could be on any side of the room	see test video 13	Successful
15	**Requirement G1. Map generation b**	Test if chest is successfully placed into individual rooms	Mapgeneration class Rooms class	For each chest in individual room, they should be place at the middle, with name of item and price for it displayed on top of it, when player purchase them, it's displayed on top of the screen.	see test video 14	Successful

16	**Requirement G2.** **Attack detection.** **all**	Test if the attack system work as requirement described	Attack class Game inspector class	Player should be able to cause damage to enemy when it hits enemy, and vice versa for enemy, when enemy is killed, it is removed and money is rewarded to player	See test video 15	successful
17	**Requirement G3. Display** **all**	Test if the camera system work as requirement described, and whether the UI is displayed correctly.	Camera class Player class Mapgeneration class	The game will display player's health as a bar, the time player has survived, the item player owns, and most importantly a camera system which follows the player to keep it centered and updated	See test video 16	successful
18	**Requirement G4. Enemy generation** **all**	Test if enemy is successfully generated accordingly as the requirement	Game inspector class	The game should spawn enemy randomly in terms of positions, and types, the enemy that is generated per 5 second should increase as time increases after a minute.	See test video 17	successful
19	**Requirement G5. Score Board** **all**	Test if game correctly display the score board of the performance of player during game play after player died	Gameend class	The game should display message that says game over after player's health reduced to 0, it should tell player the time he survived, damage he caused, and enemy he killed.	See test video 18	successful
20	**Requirement G6. Items** **all**	Test the item system in game function as requirement described	Mapgenreation class	The player should be able to purchase the item from the chest, and different item should have the right effect apply on player. Because there are 7 item to test and items are randomly generated, I have tested 4 items in the first video, and rest of 3 in another one	See test video 19 (2) for item: "flying boots", " speed gear", "enlarge potion", "ancient ice", and **Requirement G6. b** See test 20 to see test for item "power runes"	successful

					“demon force”	
21	**Requirement G7. Difficulties all**	Test if difficulties do increases after game reached 1 minute	Gamer inspector class	Difficutlies increases as described in the requirement, enemy having higher health, higher damage, higher money drop	See video test 21	successful
22	**Testing for robustness**	Test for robustness, how game flow handle errors.	All code	The game flow should be error free; user should be able to make choices free from menu without error, and during game when purchasing item and attacking enemy should run without any problems	There’s no video.	See below

Test for robustness.

In the menu stage, user is prompt and only allowed to click on buttons, there is page title such as choose your character to direct user to choose what to do, and only the choices being selected will be applied.

During game, firstly, the camera system for player will always centre the player in the screen, it follows the player, but its position is clamped once player approach the edge of the map, so player can only see the content inside of the map, nothing outside. Secondly, there is a collision detecting system in the game loop that is constantly being updated on every moving object, an attack is remove from the game once it reaches the screen edge to save computation power, also when I hit any enemy or walls, user is prohibited from moving to outside of the game map or going through walls. Collision detection system is also applied to player, only allow player go through doors, lastly, during the generation of the rooms, rooms is only allowed to be generated without intersecting any walls.

Evaluation

How well does my solution achieve the requirements

Requirement B1. User interface and

Requirement B2. Options available to select from the menu.

I have fully achieved this requirement, it is a well-designed user interface and user friendly because the information is very clear displayed, I have only hand drawn some text such as “Game Start”, “choose your characters” etc, because I would like user to know which page he is currently on. Test 1 and 2 is a very detailed demonstration which covers every button and information in the menu page, and its functionality.

The page changes are very smooth, because I use a value which record current page user is on, and the game will only update set of buttons and render specific images based on the page number, just like a book.

Requirement B3. Content that is represented by the choice on the screen.

Test 3 and 4 has demonstrated further functionality of the menu, other than GUI, the menu correctly initializes the game after player has made his final choice, after the character choosing page, and passed in the choices made my user as parameter to the constructor of the next game state. The result is menu has successfully applied player’s choices, including map and characters.

Test 4 also demonstrated the attack generated by different characters, such as sword hero can teleport, fireball hero can shoot fireball through walls etc., so this requirement is well achieved.

Requirement P1. Motion Control of player

Test 5 successfully demonstrated the test the keyboard-controlled motion of player on screen, and player did move as expected, this requirement is achieved.

Test 6 demonstrated player’s speed is changeable during game play, using an item that increases player’s speed, this all requirement in P1 was achieved.

Requirement P2. Attack Control of player

Test 7 has demonstrated player using mouse to conduct attack, and attack being successfully directed to the point where mouse was clicked, and also demonstrated changeable frequency of attack per second, the requirement P2 d was demonstrated in test 4, as it shows different characters can throw attack and different flying speed initially.

Requirement P3. Player's property:

Test 8 is a video that talk along side the code, and shows the result in game by changing values of code, resulted health change, money change etc. this requirement was well achieved.

Requirement E1. Enemy motion control

Test 9 shows the motion of enemy can be horizontal and vertical, also demonstrated that enemy is working its way to player's position, and each of them are doing the path finding calculation for themselves.

Test 10 shows enemy successfully get to player's position without being blocked by walls, a successful demonstration of path finding algorithm, this requirement was well achieved.

Requirement E2. Enemy attack control

Test 11 demonstrates how general enemy attack player, and how player's health changes when damage is caused, also shows when special enemy are spawned, the attacks they perform and collision detection between their attack and player's attacks. This requirement was well achieved.

Requirement E3. Enemy Property

Test 12 demonstrated that all aspects of enemy properties are effectively implemented. It showed enemies with health values different speeds, unique appearances, and the money they drop, enhancing gameplay. Special enemies attacking the player were also covered, adding strategy to encounters, with well explanation of dictionary structure. This demonstrates a successful fulfillment of Requirement E3.

Requirement E4 Different enemies

Test 13 demonstrated the successful implementation of 10 different types of general enemies, with 6 types being specific to certain maps, each possessing unique properties as outlined in Requirement E3. This test effectively showed the variety in enemy design and properties, confirming a diverse and engaging enemy roster that enriches the gameplay experience. The distinction between general and map-specific enemies adds depth to game strategy, fulfilling Requirement E4.

Requirement G1. Map generation

Tests 14 and 15 demonstrated the game's capability to generate maps with rooms that meet all specified properties, confirming Requirement G1's successful implementation. Rooms were shown to be rectangular, with distinct floors, surrounded by four walls, and each containing a single door whose position varied, fulfilling spatial and structural criteria. Additionally, the randomness in room size, position, and door location was effectively showcased, adding variety to each game run. The inclusion of chests in rooms, with random items, visible item names and prices, and the display of purchased items at the screen's top further enriched the gameplay experience. These tests confirmed the dynamic and engaging nature of map generation and item distribution, aligning perfectly with the intended game design.

Requirement G2. Attack detection.

Test 16 effectively demonstrated the game's attack detection system, meeting all parts of Requirement G2. It showed that the game accurately detects attacks from both the player and enemies, adjusting their health appropriately when hit. Additionally, the test confirmed that enemies are removed from the game upon being killed, aligning with part (b). Following an enemy's defeat, the player is awarded money, satisfying part (c). This demonstration ensures the game's combat mechanics are both responsive and rewarding, providing a seamless and engaging experience for the player.

Requirement G3. Display

Test 17 demonstrated the game's effective implementation of a camera system that centers the viewport on the player, ensuring the player remains at the screen's center as they move through the map, fulfilling part (a) and (b) of Requirement G3. This constant update of the camera system in relation to the player's position enhances navigation and gameplay experience. Additionally, the test showcased the in-game display features, including a health bar for the player located at the top left corner, whose length accurately represents the player's health, and a timer displayed at the top of the screen tracking the total game time. These display elements provide essential information to the player, contributing to a well-informed and engaging game environment.

Requirement G4 Enemy generation

Test 18 demonstrated the successful implementation of Requirement G4, showcasing the game's ability to generate enemies at specified intervals, such as every 5 seconds, with the interval value being editable to meet part (a). (b) The test also verified the randomness of enemy generation, ensuring a variety of enemy types are produced and not just the same ones repeatedly, addressing part (c). the number of enemies increases as game proceed after one minute.

Requirement G5. Score Board

Test 19 effectively demonstrated the game's score board functionality, fully meeting Requirement G5. It showed the score board displaying key information at the end of each gameplay session: the number of enemies killed by the player, the total damage caused by the player, and the total time the player survived.

Requirement G6. Items

Test 20 demonstrated the successful implementation of Requirement G6, showcasing a variety of items players can pick up during gameplay. These items include power runes, flying boots, gun powder, speed gear, enlarge potion, ancient ice, and demon force, each offering distinct enhancements such as increased damage, movement speed, attack speed, attack size, enemy motion freeze, and removing enemies from the screen, respectively. The test also confirmed that players could accumulate multiples of the same item, with effects adding up, enhancing gameplay dynamics and strategic depth. This variety and functionality of items enrich the player's experience, offering numerous ways to interact with and adapt to the game environment.

Requirement G7. Difficulties

Test 21 demonstrated the game's ability to adjust the level of difficulty over time, effectively meeting Requirement G7. It showed that as time progresses, enemies gain higher health and inflict more damage, challenging the player to adapt their strategies. Additionally, the test confirmed that enemies drop more money as the game becomes more challenging, rewarding the player for overcoming tougher obstacles. This dynamic adjustment of difficulty ensures a continuously engaging and progressively challenging gameplay experience, encouraging players to improve their skills and strategies.

Interview

I have sent a questionnaire out on social media and managed to get 4 responses by showing the video of my games and provide guide of how to play, and 2 of the interviewers has played my game.

This is the summaries of question and response.

Do you enjoy the game?

They find it quite engaging due to its intuitive controls and the rich variety it offers. The game's ability to introduce fresh challenges and character diversity keeps it entertaining over multiple playthroughs.

This reflects that the design of range of characters was successful, with different attacking style, allow user to player with their own style, and user mentioned multiple play through, which reflect user is willing to play multiple rounds of game, because each round is different and the theme of rogue is in my game, that player is encouraged to give another go and see what happens.

Do you think it's hard?

The consensus is that the game presents a balanced challenge. It's designed to be approachable for beginners but still offers enough complexity to keep more experienced players strategizing and adapting, although the item system might not have as many items for them to create their own styles.

This feedback was very useful in my opinion, because I think the difficulties in my game does flow very well with time, once the game reach 1 minute, which player should be familiar with the map and have a brief strategy for this round, the difficulties starts to come up, and it increases exponentially with time, because the number of enemy being generated increases as well as the stats of enemies. The comment on limited item to use during game I think is also valid, because I had a sort of shop system which player kill enemy to earn money, and purchase item from room, but there’s only 8 items in total in the game, a more abundant item set would make my game for interesting.

Is it easy to learn to play?

Learning to play is considered straightforward across the board. The game introduces its mechanics in an accessible manner, making it inviting for players new and old without overwhelming them from the start.

This feedback reflect the design of a combination control of mouse and keyboard being very straight forward to player, it almost take no time to learn how to play the game, because all you have to do is to move around and throw attacks in the majority of time in game, and this is designed to be easy and error free, with the game only allow player move in range that won’t cause any errors.

Do you get bored after a while?

They have suggested the game might benefit from added depth, such as more detailed character progression or an expanded narrative, to maintain interest in the long term.

The user seems to be more interested in terms of characters design in game, which I think is sensible, but I think no matter the item or character, even the map is designed to add more abundant and different game play to player, to make each round of gamer as different as possible, and to make choices to be as much as possible, therefore, as user said in order to maintain the interest from them, adding more progression and variety of character, item and maps, even side mission is definitely crucial to improve this game.

If there's one thing I should definitely add to my game, what is it?

Suggestions include integrating additional objectives or mini-quests to enrich the gameplay experience further. Also, features that foster community interaction, like leaderboards or competitive modes, could enhance engagement and replay ability.

This feed back was the most useful one in my opinion, when I designed the game I haven't thought of making mini-quests or similar to enrich the gameplay, because this not only added more possibilities of the gameplay, but also enrich the content of the game, so that the game play is not only just collect item and shoot enemy, if this is related to items or characters in game would definitely improve the game to the next level.

How should I improve my work

Over view

I think I have done a good job creating a fully functioning game that is error free, player is given choices at the beginning of the game, and map is randomly generated with random items, and player can choose different items from the chest, with item having different effect, and allowing duplicate, which I thought is a very fascinating game play experience, because it allows you to create your own styles with combination of items, and potentially you could achieve better result. When player is killed the game proceed to it's end stage, which displayed enemy killed, damage produced etc, and I think this is the "rewarding system" of the game and I think it is successful, player can benefit from reviewing its own performance.

What can be improve

1. I think the A star algorithm is a great use of application of traversal algorithm, but having large amount nodes still can cause lag when game has been running too long, when there's too much enemy but player isn't dead yet. Although I have done great amount of utilization on how each individual enemy access this traversal method, but I think it can be even better, such as have a shared path for enemy which are very close to each other, for example, enemy can set one of its friends at centre, and traverse once and rest of them can update themselves depending on the "canter".

2. I think more items should be included in game, although the current item system which I think is great, but there has to be more item to maintain the interest from player.

3. although the map is random in terms of position size, door side, chest item etc, but player can still predict the room structure very quickly after they gets familiar with the game, I think adding function such as connected rooms, chest that are fake, rooms in other rooms can definitely improve the entertainment of my game.

What can I add

1. multiplayer mode or potentially a score board allow player to compete with each other.

2. side quest, I could add boss enemy which are very powerful, but high rewarding when they are defeated.

The system codes

Main game class

```
using Microsoft.Xna.Framework;
using Microsoft.Xna.Framework.Graphics;
using Microsoft.Xna.Framework.Input;
using prototype.enemies;
using prototype.GameState;
using prototype.GameState.States;
using prototype.Utilities_classes;
using System.Collections.Generic;

namespace prototype
{
    public class Game1 : Game
    {
        private GraphicsDeviceManager graphicsdevicemanager;
        private SpriteBatch spritebatch;

        Menu menu;
        GameStates gameState;

        public Game1()
        {
            graphicsdevicemanager = new GraphicsDeviceManager(this);
            graphicsdevicemanager.PreferredBackBufferWidth = 1920;
            graphicsdevicemanager.PreferredBackBufferHeight = 1080;
            Content.RootDirectory = "Content";
            IsMouseVisible = true;
        }

        protected override void Initialize()
        {
            spritebatch = new SpriteBatch(GraphicsDevice);
            menu = new Menu(Content, spritebatch, graphicsdevicemanager);//create a menu

            gameState = new GameStates(menu);//then push this to game state
            gameState.peek().initialize();
            base.Initialize();
```

```
        }

        protected override void LoadContent()
        {
            menu.load(Content);
        }

        protected override void Update(GameTime gametime)
        {
            gameState.peek().update(gametime, gameState);
            base.Update(gametime);
        }

        protected override void Draw(GameTime gametime)
        {
            GraphicsDevice.Clear(Color.Black);
            var currentstate = gameState.peek();
            if (currentstate is Menu)
            {
                spritebatch.Begin();
                gameState.peek().draw(gametime);
                spritebatch.End();
                base.Draw(gametime);
            }
            else if(currentstate is gamestartstate)
            {
                //draw the game world
                spritebatch.Begin(SpriteSortMode.Deferred, BlendState.AlphaBlend, null, null, null, null, gamestartstate.cam-
era.transformationmat);
                gameState.peek().draw(gametime);
                spritebatch.End();
                //draw the UI
                spritebatch.Begin();
                gameState.peek().drawUI();
                spritebatch.End();
                base.Draw(gametime);
            }
            else if(currentstate is gameend)
            {
                spritebatch.Begin();
```

```
                gameState.peek().draw(gametime); spritebatch.End();
            }


        }
    }
}
```

Game State class

```
using prototype.GameState.States;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Security.Cryptography.X509Certificates;
using System.Text;
using System.Threading.Tasks;

namespace prototype.GameState
{
    internal class GameStates
    {
        public static Stack<Staterecipe> states = new Stack<Staterecipe>();
        public GameStates(Staterecipe temp)
        {
            states.Push(temp);
        }
        public void Push(Staterecipe temp)
        {
            states.Push(temp);
        }
        public void Pop()
        {
            states.Pop();
        }
        public Staterecipe peek()
        {
            return states.Peek();
        }
    }
}
```

State recipe class

```
using Microsoft.Xna.Framework;
using Microsoft.Xna.Framework.Content;
using Microsoft.Xna.Framework.Graphics;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace prototype.GameState
{
    abstract class Staterecipe
    {
        public abstract void initialize();
        public abstract void load(ContentManager cm);
        public abstract void update(GameTime gametime, GameStates gameStates);
        public abstract void draw(GameTime gametime);
        public abstract void drawUI();
    }
}
```

Menu State Class

```
using Microsoft.Xna.Framework.Content;
using Microsoft.Xna.Framework.Graphics;
using Microsoft.Xna.Framework;
using System.Collections.Generic;
using prototype.Utilities_classes;

namespace prototype.GameState.States
{
    internal class Menu : Staterecipe //inheritance
    {
        //classes(buttons)
        List<Button> buttonspage1 = new List<Button>();
        List<Button> buttonspage2 = new List<Button>();
        Texture2D menubackground;
        Texture2D page1background;
        Texture2D page2background;
        //fields that needs to be inherit from maingame class
        SpriteBatch sb;
        ContentManager cm;
        GraphicsDeviceManager gm;
        //menu flow
        int currentpage = 1;
        //the maingame state
        gamestartstate gamestartstate;
        string mapchoice;
        string characterchoice;
        public Menu(ContentManager C, SpriteBatch s, GraphicsDeviceManager gm)
        {
            sb = s;
            cm = C;
            this.gm = gm;
        }

        public override void initialize()
        {
            //add buttons here
            buttonspage1.Add(new Button("startbutton", new Vector2(800, 700), new Rectangle(800, 700, 200, 100)));
            //overloading here, and index for buton is required in page 2
            buttonspage2.Add(new Button("choosebutton", new Vector2(300, 700), new Rectangle(300, 700, 200, 100),1));
```

```
    buttonspage2.Add(new Button("choosebutton", new Vector2(900, 700), new Rectangle(900, 700, 200, 100),2));
    buttonspage2.Add(new Button("choosebutton", new Vector2(1500, 700), new Rectangle(1500, 700, 200, 100),3));
}
public override void load(ContentManager notwanted)
{
    foreach (Button b in buttonspage1)
    {
        b.load(cm);
    }
    foreach (Button b in buttonspage2)
    {
        b.load(cm);
    }
    menubackground = cm.Load<Texture2D>("menubackground");
    page1background = cm.Load<Texture2D>("page1background");
    page2background = cm.Load<Texture2D>("page3background");
}
public override void update(GameTime gametime, GameStates gameStates)
{
    switch (currentpage)
    {
        case 1:
            foreach (Button b in buttonspage1)
            {
                if(b.update())
                {
                    currentpage = 2;
                }
            }
            break;
        case 2:
            foreach(Button b in buttonspage2)
            {
                if (b.update())
                {
                    switch(b.buttonindex)
                    {
                        case 1:
                            mapchoice = "background";
                            currentpage = 3;
```

```
                    break;
                case 2:
                    mapchoice = "cave";
                    currentpage = 3;
                    break;
                case 3:
                    mapchoice = "town";
                    currentpage = 3;
                    break;
            }
        }
    }
    break;
case 3: // I can use the buttons in page2 hence it's also 3 choices
    foreach (Button b in buttonspage2)
    {
        if (b.update())
        {
            switch (b.buttonindex)
            {
                case 1:
                    characterchoice = "hero";
                    gamestartstate = new gamestartstate(gm, cm, sb,mapchoice,characterchoice);
                    gamestartstate.initialize();
                    gamestartstate.load(cm);
                    GameStates.states.Push(gamestartstate);
                    break;
                case 2:
                    characterchoice = "swordhero";
                    gamestartstate = new gamestartstate(gm, cm, sb, mapchoice, characterchoice);
                    gamestartstate.initialize();
                    gamestartstate.load(cm);
                    GameStates.states.Push(gamestartstate);
                    break;
                case 3:
                    characterchoice = "gunhero";
                    gamestartstate = new gamestartstate(gm, cm, sb, mapchoice, characterchoice);
                    gamestartstate.initialize();
                    gamestartstate.load(cm);
                    GameStates.states.Push(gamestartstate);
```

```
                                    break;
                                }
                            }
                        }
                        break;
                }
            }
            public override void draw(GameTime gametime) //display pages
            {
                switch (currentpage)
                {
                    case 1:
                        sb.Draw(menubackground, Vector2.Zero, Color.White);
                        foreach (Button b in buttonspage1)
                        {
                            b.draw(sb);
                        }
                        break;
                    case 2:
                        sb.Draw(page1background, Vector2.Zero, Color.White);
                        foreach (Button b in buttonspage2)
                        {
                            b.draw(sb);
                        }
                        break;
                    case 3:
                        sb.Draw(page2background, Vector2.Zero, Color.White);
                        foreach (Button b in buttonspage2)
                        {
                            b.draw(sb);
                        }
                        break;
                }
            }
            public override void drawUI()
            {

            }
        }
}
```

Game Start State Class

```
using Microsoft.Xna.Framework.Graphics;
using Microsoft.Xna.Framework.Input;
using Microsoft.Xna.Framework;
using prototype.enemies;
using System.Collections.Generic;
using System.Linq;
using Microsoft.Xna.Framework.Content;
using prototype.Utilities_classes;
using prototype.Map;
using prototype.enemies.enemy_treversal_algorithm;

namespace prototype.GameState.States
{
    internal class gamestartstate : Staterecipe //inheritance
    {
        //objects used in game
        player player;
        enemy enemy;
        public static Camera camera;
        attack attack;
        List<enemy> enemies = new List<enemy>();
        List<attack> attacks = new List<attack>();
        Mapgeneration map;
        Gridsystem gridsystem;
        //utilities fileds
        mousedetection mouse;
        game_inspector game_inspector;
        Texture2D background;
        //selections from menu
        string[] choices = new string[2]; //(map,character)
        //fields that needs to be inherit from maingame class
        GraphicsDeviceManager graphicsdevicemanager;
        SpriteBatch spritebatch;
        ContentManager Content;

        public gamestartstate(GraphicsDeviceManager g, ContentManager C, SpriteBatch s,string mapchoice, string character-
choice)
        {
            this.graphicsdevicemanager = g;
```

```
    this.Content = C;
    this.spritebatch = s;
    choices[0] = mapchoice;
    choices[1] = characterchoice;
}


public override void initialize()
{
    player = new player(3840, 2160, choices[1]); //player intialise
    player.screenedge = new Vector2(7680,4320);
    map = new Mapgeneration(graphicsdevicemanager, Content, spritebatch);//map initialize
    mouse = new mousedetection();  //mousedetection intialise
    game_inspector = new game_inspector(map, choices[0]);  //game inspector intialise
    game_inspector.playerpositionrecord = player.position;
    camera = new Camera();// camera intialize
    gridsystem = new Gridsystem(map);

}

public override void load(ContentManager cm)
{
    //player image
    player.hero = Content.Load<Texture2D>(choices[1]);
    player.healthbar = Content.Load<Texture2D>("healthbar");
    player.moneyfont = Content.Load<SpriteFont>("font");
    //enemy image
    enemydictionary.enemycontentload(Content);
    //fireball image
    switch (choices[1])
    {
        case "hero":
            attack.fireball = Content.Load<Texture2D>("fireball");
            break;
        case "swordhero":
            attack.fireball = Content.Load<Texture2D>("swordattackeffect");
            break;
        case "gunhero":
            attack.fireball = Content.Load<Texture2D>("gunheroattack");
            break;
```

```
            }
            attack.fireballeffect = Content.Load<Texture2D>("fireballeffect");
            background = Content.Load<Texture2D>(choices[0]);
            //chesttexture
            chest.chesttexture = Content.Load<Texture2D>("chest");
            //fonts
            game_inspector.totalgametimerfont = Content.Load<SpriteFont>("font");
            chest.chestfont = Content.Load<SpriteFont>("font");
        }
        public override void update(GameTime gametime, GameStates gameStates)
        {
            //total game time update,and record player's position
            game_inspector.gametimerupdate(gametime,player.position);

            //player update
            if (game_inspector.allowtopass(player.playerhitbox,player.positionupdate()))
            {
                player.Update(gametime,map);
            }

            //camera update
            camera.update(player);

            //enemy update
            game_inspector.enemyspawn(enemies); //(enemy spawn)

            for (int i = 0; i < enemies.Count(); i++)
            {  //stop enemy going through walls
                if(!enemies[i].getdirections(gridsystem, player.position,player.playerhitbox,player.isinaroom,attacks,Con-
tent))
                {
                    enemies.RemoveAt(i);
                }
            }
            foreach(enemy e in enemies)
            {
                e.enemymotionupdate(gametime);
            }
            game_inspector.checkhealth(enemies,player,spritebatch,Content); //(remove dead enemy)
```

```
            //fireballs update
            mouse.mouseactivityupdate();
            if (mouse.leftbuttonpressed) //(check mouse input to add attack, and attack speed control)
            {
                attack fireball = new attack(player.position, mouse.leftclickedposition, new Vector2(7680, 4320),
choices[1]);
                game_inspector.attackspeedcontrol(gametime, player, attacks, fireball); //(add fireballs and controll attack-
ing speed)
            }
            if (Keyboard.GetState().IsKeyDown(Keys.Escape)) //if esc pressed back to menu
            {
                gameStates.Pop();
            }

            foreach (attack fireball in attacks)
            {
                    fireball.update(game_inspector,player);
            }
            //chests update
            map.update(player, attacks, enemies);
            //(remove fireball hits wall)
            game_inspector.removeuselessattack(attacks, spritebatch, choices[1]);

            //check for hits
            game_inspector.attackhitcontrol(enemies, attacks, player, gametime);

        }
        public override void draw(GameTime gametime)
        {
            if (!player.isinaroom)
            {
                spritebatch.Draw(background, Vector2.Zero, Color.White);
                //player draw
                player.Draw(spritebatch);

                //enemy draw
                foreach (enemy e in enemies)
                {
                    e.Draw(spritebatch);
                }
```

```
                //attack draw
                foreach (attack fireball in attacks)
                {
                    fireball.draw(spritebatch);
                }

                // game_insepctor draw
                game_inspector.draw(spritebatch, gametime);
                //map draw
                map.draw(player.isinaroom, player.roomnumberin);
            }
            else
            {
                map.draw(player.isinaroom, player.roomnumberin);
                player.Draw(spritebatch);
            }
        }
        public override void drawUI()
        {
            //draw the player health bar
            player.drawUI(spritebatch);
            map.drawui();
            //game timer draw
            spritebatch.DrawString(game_inspector.totalgametimerfont, game_inspector.totalgametimer.ToString(@"mm\:ss"), new
Vector2(1800, 0), Color.Yellow, 0f, Vector2.Zero, 2, SpriteEffects.None, 0f);
        }
    }
}
```

Game End State Class

```
using Microsoft.Xna.Framework.Content;
using Microsoft.Xna.Framework;
using Microsoft.Xna.Framework.Graphics;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace prototype.GameState.States
{
    internal class gameend : Staterecipe
    {
        Texture2D background;
        SpriteFont spriteFont;
        //player stats
        TimeSpan totaltimelived;
        int enemykilled;
        int damagecaused;
        //fields that needs to be inherit from maingame class
        SpriteBatch spritebatch;
        public gameend(TimeSpan totaltimelived, int enemykilled,int damagecaused,SpriteBatch sb)
        {
            this.totaltimelived = totaltimelived;
            this.enemykilled = enemykilled;
            this.damagecaused = damagecaused;
            spritebatch = sb;
        }
        public override void initialize()
        {
        }
        public override void load(ContentManager cm)
        {
            background = cm.Load<Texture2D>("endpage");
            spriteFont = cm.Load<SpriteFont>("font");
        }
        public override void update(GameTime gametime, GameStates gameStates)
        {
```

```
        }
        public override void draw(GameTime gametime)
        {
            spritebatch.Draw(background, Vector2.Zero,Color.White);
            spritebatch.DrawString(spriteFont, totaltimelived.ToString(@"mm\:ss"), new Vector2(400, 800), Color.Red, 0f, Vec-
tor2.Zero, 2, SpriteEffects.None, 0f);
            spritebatch.DrawString(spriteFont, enemykilled.ToString(), new Vector2(1000, 800), Color.Red, 0f, Vector2.Zero,
2, SpriteEffects.None, 0f);
            spritebatch.DrawString(spriteFont, damagecaused.ToString(), new Vector2(1600, 800), Color.Red, 0f, Vector2.Zero,
2, SpriteEffects.None, 0f);
        }
        public override void drawUI()
        {
            throw new NotImplementedException();
        }
    }

}
```

Player class

```
using Microsoft.Xna.Framework;
using Microsoft.Xna.Framework.Graphics;
using Microsoft.Xna.Framework.Input;
using prototype.Map;
using System;
using System.Collections.Generic;
using System.Diagnostics.Metrics;
using System.Linq;
using System.Security.Cryptography.X509Certificates;
using System.Text;
using System.Threading.Tasks;

namespace prototype
{
    internal class player
    {
        //player stats field
        public string name;
        private double health;
        public double Health//a and m
        {
            get { return health; } set { health = value; }
        }

        private float attackspeedlimit;  //(attacking speed limit fields, smaller means faster speed)
        public float attacktimer;
        public float Attackspeedlimit
        {
            get { return attackspeedlimit; }
            set { attackspeedlimit = value; }
        }
        private bool speedlimitreach = false;
        public bool Speedlimitreach { get { return speedlimitreach; } set { speedlimitreach = value; } }

        // player motion fields
        public Vector2 position;
        public Vector2 velocity;
        private float speed;
        public float Speed   //speed a and m
```

```
{
    get { return speed; } set { speed = value; }
}

//appearance
public static Texture2D hero;
public static Vector2 screenedge;
public static Texture2D healthbar;

//mechanics field
public Rectangle playerhitbox;

//others
public bool isinaroom;
public int roomnumberin;
public int money = 100000;
public SpriteFont moneyfont;

Dictionary<string, string[]> characters = new Dictionary<string, string[]>()
{
    // structure: health, damage,speed, apearance,method(boolean)
    {"hero", new string[] {"100","0.35","8"} }, //(health,attackspeed,speed)
    {"swordhero", new string[] {"250","0.4","10"} },
    {"gunhero", new string[] {"80","0.8","7"} },

};
public player(float x, float y,string charactername)
{
    position = new Vector2(x, y);
    this.name = charactername;
    this.health = Convert.ToDouble(characters[charactername][0]);
    this.attackspeedlimit = float.Parse(characters[charactername][1]);
    this.speed = float.Parse(characters[charactername][2]);

}

public Vector2 positionupdate()//the position update logic for player
{
    KeyboardState control = Keyboard.GetState();
    //move for W and S
```

```
    if (control.IsKeyDown(Keys.W) && !(position.Y < 0))
    {
        velocity.Y = -speed;
    }
    else if (control.IsKeyDown(Keys.S) && !(position.Y >= screenedge.Y - hero.Height))
    {
        velocity.Y = speed;
    }
    else
    {
        velocity.Y = 0;
    }
    //move for A and D
    if (control.IsKeyDown(Keys.A) && !(position.X <= 0))
    {
        velocity.X = -speed;
    }
    else if (control.IsKeyDown(Keys.D) && !(position.X >= screenedge.X - hero.Width))
    {
        velocity.X = speed;
    }
    else
    {
        velocity.X = 0;
    }
    return velocity;
}
public void Update(GameTime gametime,Mapgeneration map)//player update
{
    position += positionupdate();
    playerhitbox = new Rectangle((int)position.X, (int)position.Y,hero.Width,hero.Height);
    foreach(Rooms r in map.rooms)
    {
        if (playerhitbox.Intersects(r.roomspace)) // this loop here check whether player is in a room
        {
            roomnumberin = r.roomnumber;
            isinaroom = true;
            break;
        }
        else { isinaroom = false;}
```

```
            }
        }
        public void Draw(SpriteBatch spriteBatch)
        {
            spriteBatch.Draw(hero, position, Color.White);
        }
        public void drawUI(SpriteBatch spritebatch)
        {
            spritebatch.Draw(healthbar, new Vector2(0, 0), new Rectangle(0, 0, (int)health * 3, 15), Color.White);  //draw
the health bar player
            string moneyText = $"Money: {money}";
            spritebatch.DrawString(moneyfont, moneyText, new Vector2(50,50), Color.Gold, 0f, Vector2.Zero, 2f, SpriteEf-
fects.None, 0f);
        }
    }
}
```

Attack Class

```
using DryIoc.ImTools;
using Microsoft.Xna.Framework;
using Microsoft.Xna.Framework.Content;
using Microsoft.Xna.Framework.Graphics;
using Microsoft.Xna.Framework.Input;
using prototype.GameState.States;
using System;
using System.Runtime.InteropServices;

namespace prototype
{
    internal class attack
    {
        //appearance
        string charactername;
        public static Texture2D fireball;
        public static Texture2D fireballeffect;
        public static Texture2D specialenemyattack;

        //fireball motion fields
        public Vector2 position;
        private float speed = 10f;
        public float Speed   //speed a and m
        {
            get { return speed; }
            set { speed = value; }
        }
        public Vector2 direction;

        //attack states
        Vector2 shootingposition;
        public Vector2 screenedge;
        public bool screenedgereached = false;
        public Rectangle fireballhitbox;  //(fireball hitbox)
        private int attackdamage;
        public static int damageadder = 0;
        public int Attackdamage
        {
            get { return attackdamage; } set { attackdamage = value; }
```

```
        }
        public bool enemyattack = false;
        //for item effect
        int widthsize = 0;
        int heightsize = 0;
        public static int sizeadder = 0;
        public static float speedadder = 0;
        public attack(Vector2 heroposition, Vector2 targetposition, Vector2 screenedge, string charactername)
        { //pass in essential field that helps generating an attack
            this.shootingposition = targetposition;
            this.charactername = charactername;
            switch (charactername)
            {
                case "hero":
                    this.speed = 10f + speedadder;
                    this.Attackdamage = 1 + damageadder;
                    widthsize = 30 + sizeadder;
                    heightsize = 30 + sizeadder;
                    break;
                case "swordhero":
                    this.speed = 15f + speedadder;
                    this.Attackdamage = 2 + damageadder;
                    widthsize = 50 + sizeadder;
                    heightsize = 50 + sizeadder;
                    break;
                case "gunhero":
                    this.speed = 30f + speedadder;
                    this.attackdamage = 4 + damageadder;
                    widthsize = 30 + sizeadder;
                    heightsize = 30 + sizeadder;
                    break;
            }
            position = heroposition;//shoot from heroposition
            //mouse position changes with camera moves, so apply a transform of mouse posstion and a invert of camera's
transformation matrix to make it the actual position on map
            Vector3 mapposition = Vector3.Transform(new Vector3(shootingposition.X, shootingposition.Y, 0), Matrix.In-
vert(gamestartstate.camera.transformationmat));
            this.shootingposition = new Vector2(mapposition.X, mapposition.Y);
            this.screenedge = screenedge;
            // normalize to make bullet travel towards the click on the screen
```

```
            direction = shootingposition - position;
            direction = normalizevectors(direction);
        }
        //overload
        public attack(bool specialenemy, Vector2 startingpos, Vector2 targetpos,Vector2 screenedge,ContentManager cm,string
enemyname)
        {
            specialenemyattack = cm.Load<Texture2D>(enemyname);
            speed = 12f;
            attackdamage = 6;
            widthsize = 30;
            heightsize = 30;
            enemyattack = specialenemy;
            this.screenedge = screenedge;
            position = startingpos;
            direction = targetpos - startingpos;
            direction = normalizevectors(direction);
        }

        public void update(game_inspector inspector,player player)
        {
            if (!enemyattack)
            {
                switch (charactername)
                {
                    case "hero":
                        if (position.X > 0 && position.Y > 0 && position.X < screenedge.X - fireball.Width && position.Y <
screenedge.Y - fireball.Height)
                        {
                            position += direction * speed;
                            fireballhitbox = new Rectangle((int)position.X, (int)position.Y, widthsize, heightsize);
                        }
                        else
                        {
                            screenedgereached = true;
                        }
                        break;
                    case "swordhero":
                        if (inspector.isitintersectingwalls(fireballhitbox, "all"))
                        {
```

```
                            screenedgereached = true;
                        }
                        else if (position.X > 0 && position.Y > 0 && position.X < screenedge.X - fireball.Width && position.Y
< screenedge.Y - fireball.Height)
                        {
                            position += direction * speed;
                            fireballhitbox = new Rectangle((int)position.X, (int)position.Y, widthsize, heightsize);
                            KeyboardState control = Keyboard.GetState();
                            if (control.IsKeyDown(Keys.E))
                            {
                                player.position = position;
                            }
                        }
                        else
                        {
                            screenedgereached = true;
                        }
                        break;
                    case "gunhero":
                        if (inspector.isitintersectingwalls(fireballhitbox, "all"))
                        {
                            screenedgereached = true;
                        }
                        else if (position.X > 0 && position.Y > 0 && position.X < screenedge.X - fireball.Width && position.Y
< screenedge.Y - fireball.Height)
                        {
                            position += direction * speed;
                            fireballhitbox = new Rectangle((int)position.X, (int)position.Y, widthsize, heightsize);
                        }
                        else
                        {
                            screenedgereached = true;
                        }
                        break;
                }
            }
            else
            {
                if (inspector.isitintersectingwalls(fireballhitbox, "all"))
                {
```

```
                    screenedgereached = true;
                }
                else if (position.X > 0 && position.Y > 0 && position.X < screenedge.X - fireball.Width && position.Y <
screenedge.Y - fireball.Height)
                {
                    position += direction * speed;
                    fireballhitbox = new Rectangle((int)position.X, (int)position.Y, widthsize, heightsize);
                }
                else
                {
                    screenedgereached = true;
                }
            }

        }

        public void draw(SpriteBatch spriteBatch)
        {
            if (!enemyattack)
            {
                spriteBatch.Draw(fireball, position, Color.White);
            }
            else
            {
                spriteBatch.Draw(specialenemyattack, position, Color.White);
            }
        }
        Vector2 normalizevectors(Vector2 directionvector) //produces a stright line motion for attack using unit vectors
        {
            float magnitude = (float)Math.Sqrt(Math.Pow(directionvector.X,2)+ Math.Pow(directionvector.Y, 2));
            if(magnitude > 0)
            {
                return new Vector2(directionvector.X/magnitude, directionvector.Y/magnitude);
            }
            return new Vector2(0,0);
        }
    }
}
```

Enemy Class

```
using Microsoft.Xna.Framework;
using Microsoft.Xna.Framework.Content;
using Microsoft.Xna.Framework.Graphics;
using prototype.enemies.enemy_treversal_algorithm;
using prototype.GameState.States;
using System;
using System.Collections.Generic;

namespace prototype.enemies
{
    internal class enemy
    {
        public Texture2D enemytexture;

        // enemy motion fields
        public Vector2 position;
        public Vector2 velocity;
        double speed;
        double speedcontrol = 0;
        Stack<Vector2> directions = new Stack<Vector2>();
        //enemy stats
        public int health;
        public double damage;
        Astar_algorithm astaralgorithm = new Astar_algorithm();
        public int coindrop;
        //enemy hitbox
        private Rectangle enmeyhitbox;
        public Rectangle Flyingpurplehitbox
        {
            get { return enmeyhitbox; }
            set { enmeyhitbox = value; }
        }
        //for traversal
        double timefromlasttraversal = 0;
        double traversecontrol;
        public bool ancientice = false;
        double ancienticetimer;
        //special enemy
        public bool specialenemy;
```

```
string charactername;

public enemy(int x, int y, string enemyname)//get all the stats for an enemy
{
    position = new Vector2(x, y);
    health = Convert.ToInt32(enemydictionary.enemystats[enemyname][0]);
    damage = Convert.ToDouble(enemydictionary.enemystats[enemyname][1]);
    speed = double.Parse(enemydictionary.enemystats[enemyname][2]);
    coindrop = Convert.ToInt16(enemydictionary.enemystats[enemyname][5]);
    specialenemy = (enemydictionary.enemystats[enemyname][4].Length == 4);
    string temp_name = enemydictionary.enemystats[enemyname][3];
    switch (temp_name)
    {
        case "enemy":
            enemytexture = enemydictionary.flyingpurple;
            enmeyhitbox = new Rectangle((int)position.X, (int)position.Y, enemytexture.Width, enemytexture.Height);
            break;
        case "purplemushroom":
            enemytexture = enemydictionary.purplemushroom;
            enmeyhitbox = new Rectangle((int)position.X, (int)position.Y, enemytexture.Width, enemytexture.Height);
            break;
        case "wizard":
            enemytexture = enemydictionary.wizard;
            enmeyhitbox = new Rectangle((int)position.X, (int)position.Y, enemytexture.Width, enemytexture.Height);
            break;
        case "nianmonster":
            enemytexture = enemydictionary.nianmonster;
            enmeyhitbox = new Rectangle((int)position.X, (int)position.Y, enemytexture.Width, enemytexture.Height);
            break;
        case "forest1":
            charactername = "forestattack";
            enemytexture = enemydictionary.forest1;
            enmeyhitbox = new Rectangle((int)position.X, (int)position.Y, enemytexture.Width, enemytexture.Height);
            break;
        case "forest2":
            charactername = "forestattack";
            enemytexture = enemydictionary.forest2;
            enmeyhitbox = new Rectangle((int)position.X, (int)position.Y, enemytexture.Width, enemytexture.Height);
            break;
        case "town1":
```

```
                    charactername = "townattack";
                    enemytexture = enemydictionary.town1;
                    enmeyhitbox = new Rectangle((int)position.X, (int)position.Y, enemytexture.Width, enemytexture.Height);
                    break;
                case "town2":
                    charactername = "townattack";
                    enemytexture = enemydictionary.town2;
                    enmeyhitbox = new Rectangle((int)position.X, (int)position.Y, enemytexture.Width, enemytexture.Height);
                    break;
                case "cave1":
                    charactername = "caveattack";
                    enemytexture = enemydictionary.cave1;
                    enmeyhitbox = new Rectangle((int)position.X, (int)position.Y, enemytexture.Width, enemytexture.Height);
                    break;
                case "cave2":
                    charactername = "caveattack";
                    enemytexture = enemydictionary.cave2;
                    enmeyhitbox = new Rectangle((int)position.X, (int)position.Y, enemytexture.Width, enemytexture.Height);
                    break;
            }
        }
        public bool getdirections(Gridsystem grid,Vector2 playerposition,Rectangle playerhitbox,bool isitinroom, List<attack>
attacks,ContentManager cm)
        {
            playerposition = new Vector2(playerposition.X + 50, playerposition.Y + 50);
            double modulusdistancetoplayer = Math.Sqrt(Math.Pow(playerposition.X-position.X, 2) + Math.Pow(playerposition.Y-
position.Y, 2));
            if(modulusdistancetoplayer > 2000) //this is how progrma adjust the frequency of calling path finding method base
on distance between player and enemy
            {
                return true;
            }
            else if(modulusdistancetoplayer > 1000 && modulusdistancetoplayer < 2000) { traversecontrol = 2; }
            else if(modulusdistancetoplayer > 500 && modulusdistancetoplayer < 1000) { traversecontrol = 1.5; }
            else if( modulusdistancetoplayer < 500) { traversecontrol = 1; }
            if(timefromlasttraversal >= traversecontrol && !enmeyhitbox.Intersects(playerhitbox) && !isitinroom)
            {// these conditions are, traverse cool down finish, enemy isn't already on player, player is not in room
                if (specialenemy) { attacks.Add(new attack(true, position, playerposition, new Vector2(7680, 4320), cm, char-
actername)); } //allow special enemy to throw attacks
                timefromlasttraversal = 0;
```

```
                directions = new Stack<Vector2>();
                directions = astaralgorithm.traversal(position, playerposition, grid.Getemptygrid());
                if (astaralgorithm.spawnfail == true) { return false; }
            }
            return true;
        }
        public void enemymotionupdate(GameTime gametime)//update enemy's velocity
        {
            if(timefromlasttraversal < 4) //just to make sure some enemy arene't met during the whole game play, so this time
won't get constantly updated, takes up too much memory space
            {
                timefromlasttraversal += gametime.ElapsedGameTime.TotalSeconds;
            }
            speedcontrol += gametime.ElapsedGameTime.TotalSeconds;
            if (ancientice) //this is the timer for a item ancient ice
            {
                ancienticetimer += gametime.ElapsedGameTime.TotalSeconds;
                if(ancienticetimer > 8)
                {
                    ancientice = false;
                }
            }
            if(speedcontrol >= speed)
            {
                if(directions != null)
                {
                    if (directions.Count != 0 && !ancientice)
                    {
                        position = directions.Pop();
                        speedcontrol = 0;
                        enmeyhitbox = new Rectangle((int)position.X, (int)position.Y, enemytexture.Width, enemytex-
ture.Height);
                    }
                }
            }
        }

        public void Update()//update enemy's position
        {
```

```
        }

        public void Draw(SpriteBatch sb)
        {
            sb.Draw(enemytexture, position, Color.White);
        }

    }
}
```

Enemy Dictionary Class

```
using Microsoft.Xna.Framework.Content;
using Microsoft.Xna.Framework.Graphics;
using System.Collections.Generic;
namespace prototype.enemies
{
    internal class enemydictionary
    {
        //appearance
        public static Texture2D flyingpurple;
        public static Texture2D purplemushroom;
        public static Texture2D wizard;
        public static Texture2D nianmonster;
        public static Texture2D forest1;
        public static Texture2D forest2;
        public static Texture2D cave1;
        public static Texture2D cave2;
        public static Texture2D town1;
        public static Texture2D town2;

        //load content
        public static void enemycontentload(ContentManager Content)
        {
            flyingpurple = Content.Load<Texture2D>("enemy");
            purplemushroom = Content.Load<Texture2D>("purplemashroom");
            wizard = Content.Load<Texture2D>("wizard");
            nianmonster = Content.Load<Texture2D>("nianmonster");
            forest1 = Content.Load<Texture2D>("forest1");
            forest2 = Content.Load<Texture2D>("forest2");
            cave1 = Content.Load<Texture2D>("cave1");
            cave2 = Content.Load<Texture2D>("cave2");
            town1 = Content.Load<Texture2D>("town1");
            town2 = Content.Load<Texture2D>("town2");

        }

        //list of their name
        public static List<string> enemynames = new List<string> { "flyingpurple", "purplemushroom", "wizard", "nianmonster",
"forest1", "forest2", "cave1", "cave2", "town1", "town2" };
        //enemy database
```

```
        public static Dictionary<string, string[]> enemystats = new Dictionary<string, string[]>()
        {
            // structure: health, damage,speed, apearance,special,coint drop + throw attack damage
            {"flyingpurple", new string[] {"1","1","0.3","enemy","false","1"} },
            {"purplemushroom", new string[] {"3","1","0.5","purplemushroom","false","100"} },
            {"wizard", new string[] {"2","2", "1.5625", "wizard","false","2"} },
            {"nianmonster",new string[] {"3", "2", "1.5625", "nianmonster","false","2"} },
            {"forest1" , new string[] {"5","3", "3", "forest1", "true","5","6"} },
            {"forest2" , new string[] {"5","3", "3", "forest2", "true","5","6"} },
            {"cave1" , new string[] {"5","3", "3", "cave1", "true","5","6"} },
            {"cave2" , new string[] {"5","3", "3", "cave2", "true","5","6"} },
            {"town1" , new string[] {"5","3", "3", "town1", "true","5","6"} },
            {"town2" , new string[] {"5","3", "3", "town2", "true","5","6"} },
        };

    }
}
```

Astar Algorithm Class

```
using Microsoft.Xna.Framework;
using Microsoft.Xna.Framework.Content;
using prototype.Map;
using System;
using System.Collections.Generic;
using System.Globalization;
using System.IO.IsolatedStorage;
using System.Linq;
using System.Security.Cryptography.X509Certificates;
using System.Text;
using System.Threading;
using System.Threading.Tasks;

namespace prototype.enemies.enemy_treversal_algorithm
{
    internal class Astar_algorithm
    {
        //153x86 grid
        public bool spawnfail;

        public Stack<Vector2> traversal(Vector2 current_pos, Vector2 Target_pos, Node[,] emptyGrid)
        {
            Node[,] Grid = emptyGrid;
            minheap openset = new minheap();
            hashsetfornodes closeset = new hashsetfornodes(400);
            int numofiteration = 0;
            spawnfail = false;
            // find the node position of current node and target node
            int currentnode_x = (int)(current_pos.X / 50);
            //if(currentnode_x > 0) { currentnode_x -= 1; }//to keep it the same index as the 2d array
            int currentnode_y = (int)(current_pos.Y / 50);
            //if(currentnode_y > 0) { currentnode_y -= 1; }
            int targetnode_x = (int)(Target_pos.X/ 50);
            //if(targetnode_x > 0) {  targetnode_x -= 1; }
            int targetnode_y = (int)(Target_pos.Y / 50);
            //if(targetnode_y > 0) { targetnode_y -= 1; }

            Node startnode = Grid[currentnode_x, currentnode_y];
            startnode.setgcost(0);
```

```
            Node endnode = Grid[targetnode_x, targetnode_y];
            closeset.insertnode(startnode);

            //this is intialization of the openset, and after I can begin the algorithm to find the target
            foreach (Node n in findneibours(currentnode_x, currentnode_y,targetnode_x,targetnode_y,true,Grid))
            {
                if(n == endnode)
                {
                    Stack<Vector2> output = new Stack<Vector2>();
                    output.Push(endnode.position);
                    return output;
                }
                if (n.iswalkable)    //walkable openset, otherwise closeset
                {
                    n.setparent(startnode);
                    openset.addnode(n);
                }
                else
                {
                    closeset.insertnode(n);
                }
            }
            if(openset.nodes.Count == 0) //if the open set is empty, return a empty list for deleting enemy
            {
                spawnfail = true;
                return null;
            }
            //the algorithm begins here
            while (openset.nodes.Count > 0 && numofiteration < 6500)
            {//if the closest node found is the target node, out put the position of it
                numofiteration++;
                Node tempnode = openset.nodes[0];
                //this is a loop to test if it gointo a deadend
                List<Node> listtoaddtoopenset = new List<Node>();
                foreach (Node n in findneibours(tempnode.nodexindex_x, tempnode.nodeyindex_y, targetnode_x, targetnode_y,
false,Grid))
                {
                    if (n == endnode) //if the neibour is the target
                    {
                        endnode.setparent(tempnode);
```

```
                        Stack<Vector2> output = new Stack<Vector2>();
                        Node temp = endnode;
                        CreatePath(temp, startnode, output);
                        return output;
                    }
                    if(n.iswalkable && !closeset.has(n)) //if it's walkable and not in close set
                    {
                        if (openset.nodes.Contains(n)) //if the node already in open set
                        {
                            //if the gcost to walk from this neibour to a node in the open set is shorter than calculated
before, use this gcost and set
                            //this node's(already in openset) parent the node we currently at

                            Node nodetocheck = n; //this is the neiboured found (the one already in open set)
                            nodetocheck.addgcost(tempnode.gcost); //calculate the new gcost to walk from node we currently at
                            int thenalreadytheregcost = n.gcost; //this node's original g cost
                            int thenewpathgcosttoadd = nodetocheck.gcost; //the new calculated gcost
                            if (thenewpathgcosttoadd < thenalreadytheregcost)
                            {
                                n.setgcost(thenewpathgcosttoadd); //set g cost for this node the new g cost calculated
                                n.sumcost();
                                n.setparent(tempnode); //sets the parent for this node(already there) the node we currently
add

                            }
                        }
                        else //walkable, and not in openset
                        {
                            n.setparent(tempnode);
                            n.addgcost(tempnode.gcost);
                            listtoaddtoopenset.Add(n);
                        }
                    }
                    else if (!n.iswalkable) //if not walkable add to close set
                    {
                        closeset.insertnode(n);
                    }
                }
                if (listtoaddtoopenset == null)//that means all of neibours is not walkable or already in close set, this is
a deadend
                {
```

```
                    closeset.insertnode(openset.removenode());//remove this node we are not going to this one
                }
                else
                {
                    closeset.insertnode(openset.removenode());//remove this node, it's fully explored, and put this node in
close set
                    foreach (Node n in listtoaddtoopenset)
                    {
                        openset.addnode(n);
                    }
                }
            }
            return null;
        }
        List<Node> findneibours(int x, int y,int targetx,int targety,bool addgcost, Node[,] Grid) //this will return the
valid neibours of a node it is currently on
        {
            List<Node> temp = new List<Node>();
            if(x-1 > 0)
            {
                if(y >= 0 && y < 86)
                {
                    temp.Add(Grid[x - 1, y]);
                    Grid[x - 1, y].addhcost(Manhhatandistance(x - 1, targetx, y, targety));
                    if (addgcost)
                    {
                        Grid[x - 1, y].addgcost(Grid[x,y].gcost);
                    }
                    Grid[x - 1, y].sumcost();
                }
            }
            if(x+1 < 153)
            {
                if (y >= 0 && y < 86)
                {
                    temp.Add(Grid[x + 1, y]);
                    Grid[x + 1, y].addhcost(Manhhatandistance(x + 1, targetx, y, targety));
                    if (addgcost)
                    {
                        Grid[x + 1, y].addgcost(Grid[x, y].gcost);
```

```
            }
            Grid[x + 1, y].sumcost();
        }
    }
    if(y-1 > 0)
    {
        if(x >= 0 && x < 153)
        {
            temp.Add(Grid[x, y - 1]);
            Grid[x, y - 1].addhcost(Manhhatandistance(x, targetx, y - 1, targety));
            if (addgcost)
            {
                Grid[x, y - 1].addgcost(Grid[x, y].gcost);
            }
            Grid[x, y - 1].sumcost();
        }
    }
    if (y+1 <86)
    {
        if (x >= 0 && x < 153)
        {
            temp.Add(Grid[x, y + 1]);
            Grid[x, y + 1].addhcost(Manhhatandistance(x, targetx, y + 1, targety));
            if (addgcost)
            {
                Grid[x, y + 1].addgcost(Grid[x, y].gcost);
            }
            Grid[x, y + 1].sumcost();
        }
    }
    return temp;
}

int Manhhatandistance(int x1, int x2, int y1, int y2) //this is the method to estimate the h cost between node
{
    return(Math.Abs(x1-x2) + Math.Abs(y1-y2));
}

void CreatePath(Node n,Node startnode, Stack<Vector2> output)
{
```

```
            output.Push(n.position);
            n = n.parent;
            if (n != startnode && n != null)
            {
                CreatePath(n, startnode, output); //recurstion
            }
        }
    }
}
```

Grid Class

```
using Microsoft.Xna.Framework;
using Microsoft.Xna.Framework.Graphics;
using prototype.Map;
using System.Dynamic;
using System.Runtime.CompilerServices;

namespace prototype.enemies.enemy_treversal_algorithm
{
    internal class Gridsystem
    {
        Node[,] Grid = new Node[153, 86];//153x86 grid
        public Gridsystem(Mapgeneration map)
        {
            for (int i = 0; i < 153; i++)//put node into grid
            {
                for (int j = 0; j < 86; j++)
                {
                    Rectangle tempnode = new Rectangle(i * 50, j * 50, 50, 50);
                    foreach (Rooms r in map.rooms)//if intersects any walls mark as non walkable
                    {
                        Rectangle temproom = new Rectangle((int)r.roompostition.X - 50, (int)r.roompostition.Y - 50,
r.roomspace.Width + 100, r.roomspace.Height - 100);

                            if(tempnode.Intersects(temproom))
                            {
                                Grid[i, j] = new Node(new Vector2(i * 50, j * 50), false,i,j);
                                break;
                            }
```

```
                        else
                        {
                            Grid[i, j] = new Node(new Vector2(i * 50, j * 50), true,i,j);
                        }

                    }
                }
            }
        }
        public Node[,] Getemptygrid()
        {
            return Grid;
        }
    }
}
```

Hash Table for Nodes Class

```
using Microsoft.Xna.Framework.Content;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Xml;

namespace prototype.enemies.enemy_treversal_algorithm
{
    internal class hashsetfornodes
    {
        List<Node>[] hashtable;
        int size;
        public hashsetfornodes(int size)
        {
            this.size = size;
            hashtable = new List<Node>[size];
            for (int i = 0; i < size; i++)
            {
                hashtable[i] = new List<Node>();
            }
        }
        int gethashidex(Node node)//calculate the index of this node using its grid index * a prime number
        {
            return (node.nodexindex_x * 23 + node.nodeyindex_y) % size;
        }
        public void insertnode(Node n)
        {
            int index = gethashidex(n);
            hashtable[index].Add(n);
        }
        public bool has(Node n)//check whether the hashset contians this node
        {
            if (hashtable[gethashidex(n)].Contains(n))
            {
                return true;
            }
            else
```

```
            {
                return false;
            }

        }

    }
}
```

Min Heap Class

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Linq;
using System.Text;
using System.Threading;
using System.Threading.Tasks;

namespace prototype.enemies.enemy_treversal_algorithm
{
    internal class minheap
    {
        public List<Node> nodes = new List<Node>();
        Node output;
        public void swap(int a, int b)//swap 2 item in the list
        {
            Node tempnode = nodes[a];
            nodes[a] = nodes[b];
            nodes[b] = tempnode;
        }
        public void upheap(int index)//when add item at the end, put it at right position int he list
        {
            while(index > 0)
            {
                if (nodes[index].fcost < nodes[(index - 1) / 2].fcost)
                {
                    swap(index, (index - 1) / 2);
                    index = (index - 1) / 2;
                }
                else
```

```
        {
            break;
        }
    }
}
public void downheap(int index)//when remove item, restructure the bianry tree to correct order
{
    int length = nodes.Count;
    while(index < length)
    {
        int currentindex = index;
        int leftchildindex = index * 2 + 1;
        int rightchildindex = index * 2 + 2;
        if (leftchildindex < length && nodes[leftchildindex].fcost < nodes[currentindex].fcost)
        {
            currentindex = leftchildindex;
        }
        if (rightchildindex < length && nodes[rightchildindex].fcost < nodes[currentindex].fcost)
        {
            currentindex = rightchildindex;
        }
        if(currentindex != index)
        {
            swap(currentindex, index);
            index = currentindex;
        }
        else
        {
            break;
        }
    }
}
public Node removenode()//put swap first and last node, remove the last node, and restructure the tree
{
    output = nodes[0];
    swap(0,nodes.Count()-1);
    nodes.RemoveAt(nodes.Count()-1);
    downheap(0);
    return output;
}
```

```
        public Node recordthisnode()
        {
            removenode();
            return output;
        }
        public void addnode(Node n)//inserting node
        {
            nodes.Add(n);
            upheap(nodes.Count - 1);
        }
    }
}
```

Nodes Class

```
using System;
using System.Collections.Generic;
using System.Linq;
using Microsoft.Xna.Framework;
using System.Text;
using System.Threading.Tasks;
using System.Xml;

namespace prototype.enemies.enemy_treversal_algorithm
{
    internal class Node
    {
        public Vector2 position;
        public bool iswalkable;
        public int gcost;
        int hcost;
        public int fcost;
        public Node parent;
        public int nodexindex_x;
        public int nodeyindex_y;

        public Node(Vector2 position, bool iswalkable,int x, int y)
        {
            this.position = position;
            this.iswalkable = iswalkable;
            setnodeindex(x, y);
```

```
        }
        public void addhcost(int hcost)
        {
            this.hcost = hcost;
        }
        public void addgcost(int x)
        {
            gcost = x+1;
        }
        public void setgcost(int gcost)
        {
            this.gcost = gcost;
        }
        public void sumcost()
        {
            fcost  = gcost + hcost;
        }
        public void setparent(Node n)
        {
            parent = n;
        }
        public void setnodeindex(int x, int y)
        {
            nodexindex_x = x;
            nodeyindex_y = y;
        }
    }
}
```

Map Class

```
using Microsoft.Xna.Framework.Content;
using Microsoft.Xna.Framework.Graphics;
using Microsoft.Xna.Framework;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using DryIoc.ImTools;
using Microsoft.Xna.Framework.Input;
using prototype.GameState.States;
using prototype.enemies;

namespace prototype.Map
{
    internal class Mapgeneration
    {
        //rooms, chest and items
        public List<Rooms> rooms;
        public chest[] chests;
        public string[] items = { "powerrunes", "flyingboots", "gunpowder", "speedgear", "enlargepotion", "ancientice",
"Demonforce"};
        Texture2D powerrunes;
        Texture2D flyingboots;
        Texture2D gunpowder;
        Texture2D speedgear;
        Texture2D enlargepotion;
        Texture2D ancientice;
        Texture2D Demonforce;
        mousedetection mouse = new mousedetection();
        //fields that needs to be inherit from maingame class
        SpriteBatch spritebatch;
        ContentManager Content;
        //fileds responsible for draw
        List<Texture2D> itemtodraw = new List<Texture2D>();
        List<Vector2> itemposition = new List<Vector2>();
        int drawingposition = 500;

        public Mapgeneration(GraphicsDeviceManager g, ContentManager C, SpriteBatch s)
```

```
        {
            this.Content = C;
            this.spritebatch = s;
            //load all items texture
            load();
            //this constructor will generate a list of rooms, without overlapping each other
            rooms = new List<Rooms>();
            Random a = new Random();//number of rooms
            Random b = new Random();  //settings for each room
            Rooms temproom = new Rooms(b.Next(1000, 1500), b.Next(800, 1000), new Vector2(b.Next(200, 5980), b.Next(200,
3120)), Content,0);
            rooms.Add(temproom);
            for (int i = 0; i < a.Next(4,9); i++) //generate rooms
            {
                bool overlap = true;
                whilestart: //use of labels
                while (overlap)
                {
                    temproom = new Rooms(b.Next(1000, 1500), b.Next(800, 1000), new Vector2(b.Next(200, 5980), b.Next(200,
3120)), Content,i+1);
                    foreach (Rooms r in rooms)
                    {
                        if (temproom.roomspace.Intersects(r.roomspace))
                        {
                            goto whilestart;
                        }
                    }
                    overlap = false;
                }
                rooms.Add(temproom);
            }
            //code here is handling chest and item
            chests = new chest[rooms.Count];
            int counter = 0;
            foreach(Rooms r in rooms)
            {
                chest c = new chest(new Vector2(r.roomspace.X + r.roomspace.Width / 2, r.roomspace.Y + r.roomspace.Height /
2), r.roomnumber, items[a.Next(0, 7)]);
                chests[counter] = c; counter++;
            }
```

```
        }
        void load()
        {
            powerrunes = Content.Load<Texture2D>("powerrunes");
            flyingboots = Content.Load<Texture2D>("flyingboots");
            gunpowder = Content.Load<Texture2D>("gunpowder");
            speedgear = Content.Load<Texture2D>("speedgear");
            enlargepotion = Content.Load<Texture2D>("enlargepotion");
            ancientice = Content.Load<Texture2D>("acientice");
            Demonforce = Content.Load<Texture2D>("demonforce");
        }
        public void update(player player, List<attack> attacks, List<enemy> enemies) //check player open a chest and add on
according effect
        {
            mouse.mouseactivityupdate(true);

            if(mouse.leftbuttonpressed)
            {
                chest c;
                foreach (chest temp in chests)
                {
                    if (temp.roomnumberin == player.roomnumberin)
                    {
                        c = temp;
                        Vector3 mapposition = Vector3.Transform(new Vector3(mouse.leftclickedposition.X, mouse.leftclickedpo-
sition.Y, 0), Matrix.Invert(gamestartstate.camera.transformationmat));
                        Vector2 clickposition = new Vector2(mapposition.X, mapposition.Y);
                        if (new Rectangle((int)clickposition.X,(int) clickposition.Y, 1, 1).Intersects(c.chestspace) &&
player.money >= c.chestvalue)
                        {
                            player.money -= c.chestvalue;
                            switch (c.itemname)
                            {
                                case ("powerrunes"): //increase damage by 4
                                    attack.damageadder += 4;
                                    itemtodraw.Add(powerrunes);
                                    itemposition.Add(new Vector2(drawingposition, 0));
                                    drawingposition += 50;
                                    break;
                                case ("flyingboots"): //increases player's moving speed by 1
```

```
                player.Speed += 1;
                itemtodraw.Add(flyingboots);
                itemposition.Add(new Vector2(drawingposition, 0));
                drawingposition += 50;
                break;
            case ("gunpowder"): //increases attacks flying speed of player by 1
                attack.speedadder += 5f;
                itemtodraw.Add(gunpowder);
                itemposition.Add(new Vector2(drawingposition, 0));
                drawingposition += 50;
                break;
            case ("speedgear"): //increases attack frequency by 0.05 secibds
                player.Attackspeedlimit -= 0.05f;
                itemtodraw.Add(speedgear);
                itemposition.Add(new Vector2(drawingposition, 0));
                drawingposition += 50;
                break;
            case ("enlargepotion"): //make the hitbox of attacks bigger
                attack.sizeadder += 10;
                itemtodraw.Add(enlargepotion);
                itemposition.Add(new Vector2(drawingposition, 0));
                drawingposition += 50;
                break;
            case ("ancientice"): //freeze enemy for 8 seconds
                foreach(enemy e in enemies)
                {
                    e.ancientice = true;
                }
                itemtodraw.Add(ancientice);
                itemposition.Add(new Vector2(drawingposition, 0));
                drawingposition += 50;
                break;
            case ("Demonforce"): // remove all enemy currently inthe game
                enemies.Clear();
                itemtodraw.Add(Demonforce);
                itemposition.Add(new Vector2(drawingposition, 0));
                drawingposition += 50;
                break;
        }
    }
```

```
                    break;
                }
            }
        }
    }
    public void draw(bool isplayerinroom,int roomnumber)
    {
        if (isplayerinroom) //only draws the room the player is currently in
        {
            rooms[roomnumber].draw(spritebatch, isplayerinroom);
            foreach(chest c in chests)
            {
                if(c.roomnumberin == roomnumber)
                {
                    c.draw(spritebatch);
                    break;
                }
            }
        }
        else
        {
            foreach (Rooms r in rooms) //draw all room if player isn't in any of them
            {
                r.draw(spritebatch, isplayerinroom);
            }
        }

    }
    public void drawui()
    {
        for (int i = 0; i < itemtodraw.Count(); i++)
        {
            spritebatch.Draw(itemtodraw[i], itemposition[i], Color.White);
        }
    }

}
}
```

Rooms Class

```
using Microsoft.Xna.Framework;
using Microsoft.Xna.Framework.Content;
using Microsoft.Xna.Framework.Graphics;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace prototype.Map
{
    internal class Rooms
    {
        //room position and size
        Texture2D roomfloortexture;
        Texture2D rooftexture;
        Texture2D doorhorizontaltexture;
        Texture2D doorverticaltexture;
        Texture2D walltexture;
        public Vector2 roompostition;
        public Rectangle roomspace;
        public int roomnumber;
        //room structure
        Rectangle horizontalwall1;
        Rectangle horizontalwall3;
        Rectangle verticalwall2;
        Rectangle verticalwall4;
        public Rectangle door;
        public Rectangle[] horizontalwalls;
        public Rectangle[] verticalwalls;
        public Rectangle[] walls;
        int doorside;
        public Rooms(int width, int height, Vector2 roomposition, ContentManager cm, int roomnumber)
        {
            this.roompostition = roomposition;
            roomspace = new Rectangle((int)roomposition.X, (int)roomposition.Y, width, height);
            //load textures
            roomfloortexture = cm.Load<Texture2D>("floortexture");
            doorhorizontaltexture = cm.Load<Texture2D>("doorhorizontal");
```

```
            doorverticaltexture = cm.Load<Texture2D>("doorvertical");
            walltexture = cm.Load<Texture2D>("walltexture");
            rooftexture = cm.Load<Texture2D>("rooftexture");
            //randomly select which side is the door in the room
            Random r = new Random();
            doorside = r.Next(1, 5);
            //generate door and walls
            wallgenerate();
            doorgenerate();
            this.roomnumber = roomnumber;
        }
        public void wallgenerate()//generate walls aroudn the room space, 2 vertical and 2 horizontal
        {
            horizontalwalls = new Rectangle[2];
            verticalwalls = new Rectangle[2];
            walls = new Rectangle[4];
            //horizontal walls  //ababa
            horizontalwall1 = new Rectangle((int)roompostition.X, (int)roompostition.Y - 50, roomspace.Width, 50);
            horizontalwalls[0] = horizontalwall1;
            walls[0] = horizontalwall1;
            horizontalwall3 = new Rectangle((int)roompostition.X, (int)(roompostition.Y + roomspace.Height), roomspace.Width,
50);
            horizontalwalls[1] = horizontalwall3;
            walls[1] = horizontalwall3;
            //vertical walls
            verticalwall4 = new Rectangle((int)(roompostition.X - 50), (int)(roompostition.Y-50),50,roomspace.Height+100);
            verticalwalls[0] = verticalwall4;
            walls[2] = verticalwall4;
            verticalwall2 = new Rectangle((int)(roompostition.X + roomspace.Width), (int)(roompostition.Y-50), 50,
roomspace.Height +100);
            verticalwalls[1] = verticalwall2;
            walls[3] = verticalwall2;
        }
        public void doorgenerate()//randomly place the door on one of the wall
        {
            switch (doorside)
            {
                case 1://door on wall 1
                    door =  new Rectangle((int)(roompostition.X + (roomspace.Width / 2) - 100), (int)(roompostition.Y - 50),
200, 50);
```

```
                    break;
                case 2://door on wall 2
                    door = new Rectangle((int)(roompostition.X + roomspace.Width), (int)(roompostition.Y + (roomspace.Height
/ 2) - 100), 50, 200);
                    break;
                case 3: // door on wall3
                    door = new Rectangle((int)(roompostition.X + (roomspace.Width / 2) - 100), (int)(roompostition.Y +
roomspace.Height), 200, 50);
                    break;
                case 4: //door on wall4
                    door = new Rectangle((int)(roompostition.X - 50),(int)(roompostition.Y+(roomspace.Height/2)-100),50,200);
                    break;
            }
        }
        public void draw(SpriteBatch sb, bool isplayerinroom)
        {
            if(isplayerinroom)
            {
                sb.Draw(roomfloortexture, roompostition, roomspace, Color.White);//draw the floor
            }
            else
            {
                sb.Draw(rooftexture, roompostition, roomspace, Color.White); // only draw roof when player isn't in room
            }
            foreach (Rectangle r in walls)//draw the walls
            {
                sb.Draw(walltexture, new Vector2(r.X, r.Y), r, Color.White);
            }
            switch (doorside)//draw the doors
            {
                case 1://this is for the horizontal wall texture
                case 3:
                    sb.Draw(doorhorizontaltexture, new Vector2(door.X, door.Y), Color.White);
                    break;
                case 2://this is for the vertical texture
                case 4:
                    sb.Draw(doorverticaltexture, new Vector2(door.X, door.Y), Color.White);
                    break;
            }
```

```
        }

    }

}
```

Chest Class

```
using Microsoft.Xna.Framework;
using Microsoft.Xna.Framework.Content;
using Microsoft.Xna.Framework.Graphics;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Numerics;
using System.Text;
using System.Threading.Tasks;
using Vector2 = Microsoft.Xna.Framework.Vector2;

namespace prototype.Map
{
    internal class chest
    {
        //information for a chest
        Vector2 chestposition;
        public static Texture2D chesttexture;
        public static SpriteFont chestfont;
        public int chestvalue;
        public Rectangle chestspace;
        public int roomnumberin;
        public string itemname;
        public chest(Vector2 chestposition,int roomnumber, string itemname)
        {
            this.chestposition = chestposition;
            Random r = new Random();
            chestvalue = r.Next(10, 20); //randomly generate chest value
            chestspace = new Rectangle((int)chestposition.X, (int)chestposition.Y, 80, 80);
            roomnumberin = roomnumber;
            this.itemname = itemname;
        }
        public void draw(SpriteBatch sb)
```

```
        {
            sb.Draw(chesttexture,chestposition,Color.White);
            sb.DrawString(chestfont, chestvalue.ToString(), new Vector2(chestposition.X - 100, chestposition.Y - 130),
Color.Cyan, 0f, Vector2.Zero, 3, SpriteEffects.None, 0f);
            sb.DrawString(chestfont, itemname, new Vector2(chestposition.X-100,chestposition.Y-80), Color.Red, 0f, Vec-
tor2.Zero, 3, SpriteEffects.None, 0f);
        }
    }
}
```

Button Class

```
using Microsoft.Xna.Framework;
using Microsoft.Xna.Framework.Content;
using Microsoft.Xna.Framework.Graphics;
using Microsoft.Xna.Framework.Input;
using prototype.GameState;
using prototype.GameState.States;

namespace prototype.Utilities_classes
{
    internal class Button
    {
        public string name;
        public int buttonindex;
        Texture2D buttontexture;
        Vector2 position;
        Rectangle clickbox;
        //classes used
        //MouseState mousestate;
        mousedetection mouse = new mousedetection();
        // name, position, clickbox rectangle
        public Button(string name, Vector2 position, Rectangle clickbox)
        {
            this.name = name;
            this.position = position;
            this.clickbox = clickbox;
        }
        public Button(string name, Vector2 position, Rectangle clickbox,int buttonindex)
        {
            this.name = name;
            this.position = position;
```

```
            this.clickbox = clickbox;
            this.buttonindex = buttonindex;
        }

        public void load(ContentManager cm)
        {
            buttontexture = cm.Load<Texture2D>(name);
        }
        public bool update()
        {
            mouse.mouseactivityupdate(true);//check mouse click and check if it was click on the button
            if(new Rectangle ((int)mouse.leftclickedposition.X,(int)mouse.leftclickedposition.Y, 1,1).Intersects(clickbox)&&
mouse.leftbuttonpressed)
            {
                return true;
            }
            return false;

        }
        public void draw(SpriteBatch spritebatch)
        {
            spritebatch.Draw(buttontexture, position, Color.White);
        }
    }
}
```

Camera Class

```
using Microsoft.Xna.Framework;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace prototype.Utilities_classes
{
    internal class Camera
    {
        public Matrix transformationmat;
        public Vector2 position;
        public void update(player player)
        {
            position = player.position;
            //clamp method keep the positions of the camera focus in this range, so I doesn't show the view beyond the back-
ground size
            position.X = MathHelper.Clamp(position.X, 960, 6720);
            position.Y = MathHelper.Clamp(position.Y, 540, 3780);
            //first translation moves the map opposite direction to the player, second translation kept player at center of
the screen
            transformationmat = Matrix.CreateTranslation(new Vector3(-position.X, -position.Y, 0)) * Matrix.CreateTransla-
tion(new Vector3(960,480, 0));
        }

    }
}
```

Game Inspector Class

```
using Microsoft.Xna.Framework;
using Microsoft.Xna.Framework.Content;
using Microsoft.Xna.Framework.Graphics;
using prototype.enemies;
using prototype.GameState;
using prototype.GameState.States;
using prototype.Map;
using System;
using System.Collections.Generic;
using System.Linq;

namespace prototype
{
    internal class game_inspector
    {
        public List<Vector2> attackeffectpositionlist = new List<Vector2>();  //to draw fireball disappearing effect
        //timers
        double temptimer_damage;
        double temptimer_enemyspawn;
        double temptimer_specialenemyspawn;
        double temptimer_difficulties;
        //esssiential game field
        public static SpriteFont totalgametimerfont;
        public static TimeSpan totalgametimer;
        public Vector2 playerpositionrecord;
        public Vector2 playercurrentpositionrecord;
        //map
        Mapgeneration map;
        string mapname;
        //field for generating enemy
        public Queue<Vector2> enemygenration = new Queue<Vector2>();
        Random spawnenemyrandom = new Random();
        bool oneminpass = false;
        int difficulties = 2;
        double adjustdifficulties;
        //player record
        int enemykilled = 0;
        int damagecaused = 0;
        gameend gameend;
```

```
        public game_inspector(Mapgeneration map, string mapname)//initial spawn of enemy
        {
            this.map = map;
            for (int i = 0; i < 25; i++) // push 25 position into a queue that is not intersecting any walls, and enemy will
spawn at these positions
            {
                Rectangle temprectangle = new Rectangle(spawnenemyrandom.Next(0, 7200), spawnenemyrandom.Next(0, 4000), 50,
50);
                while (isitintersectingwalls(temprectangle, "rooms"))
                {
                    temprectangle = new Rectangle(spawnenemyrandom.Next(0, 7200), spawnenemyrandom.Next(0, 4000), 50, 50);
                }
                enemygenration.Enqueue(new Vector2(temprectangle.X, temprectangle.Y));

            }

            this.mapname = mapname;
        }
        public void checkhealth(List<enemy> enemies,player player,SpriteBatch sb,ContentManager cm) //remove 0 hp enemy
        {
            if(player.Health <= 0)
            {
                gameend = new gameend(totalgametimer, enemykilled, damagecaused, sb);
                gameend.load(cm);
                GameStates.states.Push(gameend);
            }
            for(int i = 0; i < enemies.Count; i++)
            {
                if(enemies[i].health <= 0)
                {
                    player.money += enemies[i].coindrop;
                    enemykilled++;  //record enemy killed
                    enemies.RemoveAt(i);
                }
            }
        }

        public void removeuselessattack(List<attack> attacks,SpriteBatch spriteBatch,string charactername) //remove attack
when it reach screen edge
        {
```

```
            for(int i =0; i < attacks.Count; i++)
            {
                if (attacks[i].screenedgereached)
                {
                    attackeffectpositionlist.Add(attacks[i].position);
                    attacks.RemoveAt(i);
                }
            }
        }

        public void attackhitcontrol(List<enemy> enemies,List<attack> attacks, player player, GameTime gametime)   //colli-
sion detection, check for any damage
        {
            foreach (enemy e in enemies)
            {
                for(int i =0; i<attacks.Count; i++)//check attack hit enemy
                {
                    if (attacks[i].fireballhitbox.Intersects(e.Flyingpurplehitbox) && !attacks[i].enemyattack)
                    {
                        e.health -= attacks[i].Attackdamage;
                        damagecaused += attacks[i].Attackdamage; //record player's damage
                        attackeffectpositionlist.Add(e.position);
                        attacks.RemoveAt(i);
                    }
                    if(i< attacks.Count&&attacks[i].fireballhitbox.Intersects(player.playerhitbox) && attacks[i].enemyattack)
                    {
                        player.Health -= attacks[i].Attackdamage;
                        attackeffectpositionlist.Add(e.position);
                        attacks.RemoveAt(i);
                    }
                }
                if(e.Flyingpurplehitbox.Intersects(player.playerhitbox)&& temptimer_damage >= 0.1)//check enemy hit hero(also
limit the speed of attack, not decreasing health every frame)
                {
                    player.Health -= e.damage;
                    temptimer_damage = 0;
                }
            }
            temptimer_damage += gametime.ElapsedGameTime.TotalSeconds;
        }
```

```
        public void attackspeedcontrol(GameTime gameTime,player player,List<attack> fireballs, attack fireball)  //control
attacking speed
        {
            if (player.attacktimer >= player.Attackspeedlimit) { player.Speedlimitreach = false; }
            if (player.Speedlimitreach == false)
            {
                fireballs.Add(fireball);
                player.Speedlimitreach = true;
                player.attacktimer = 0;
            }
            else { player.attacktimer += (float)gameTime.ElapsedGameTime.TotalSeconds; }
        }

        public void draw(SpriteBatch spriteBatch,GameTime gametime)   //draw the attack hit effect
        {
            for (int i = 0; i < attackeffectpositionlist.Count; i++)
            {
                spriteBatch.Draw(attack.fireballeffect, attackeffectpositionlist[i], Color.White);
                attackeffectpositionlist.RemoveAt(i);
            }
        }

        public void gametimerupdate(GameTime gametime,Vector2 position) //update all the timers, and adjust the difficulties
        {
            temptimer_enemyspawn += gametime.ElapsedGameTime.TotalSeconds;
            temptimer_difficulties += gametime.ElapsedGameTime.TotalSeconds;
            temptimer_specialenemyspawn += gametime.ElapsedGameTime.TotalSeconds;
            totalgametimer += gametime.ElapsedGameTime;
            adjustdifficulties += gametime.ElapsedGameTime.TotalSeconds;
            playercurrentpositionrecord = position;

            //start to spawn special enemy after 1min
            if (totalgametimer.TotalMinutes >= 1) //after 1 mins, special enemy starts to spawn
            {
                if (temptimer_specialenemyspawn >= 4) //new position for enemy generation gets push into the queue every 4
seconds
                {
                    temptimer_specialenemyspawn = 0;
                    for (int j = 0; j < difficulties +2; j++) //spawn random amount of enemies and not within any rooms.
```

```
                    {
                        Rectangle temprectangle = new Rectangle(spawnenemyrandom.Next(0, 7200), spawnenemyrandom.Next(0,
4000), 50, 50);
                        while (isitintersectingwalls(temprectangle, "rooms"))
                        {
                            temprectangle = new Rectangle(spawnenemyrandom.Next(0, 7200), spawnenemyrandom.Next(0, 4000), 80,
80);
                        }
                        enemygenration.Enqueue(new Vector2(temprectangle.X, temprectangle.Y));
                    }
                }
            }
        }


        public void enemyspawn(List<enemy> enemies) //spawn enemies according to time and difficulties
        {
            if(totalgametimer.TotalMinutes > 1)// when game pass 1 minitue
            {
                if(adjustdifficulties>= 30) // every 30 seconds increase the difficulties
                {
                    updateenemystats();
                    adjustdifficulties = 0;
                    difficulties++;
                }
                if(temptimer_enemyspawn >= 5)//spawn enemy 5 seconds
                {
                    temptimer_enemyspawn = 0;
                    for (int i = 0; i < difficulties; i++)
                    {
                        Vector2 tempposition = enemygenration.Dequeue();
                        Random r = new Random();
                        if(r.Next(2) == 0)
                        {
                            enemies.Add(new enemy((int)tempposition.X, (int)tempposition.Y, enemydictionary.enemy-
names[spawnenemyrandom.Next(0,4)])); //use dequeue enemy when spawn
                        }
                        else
                        {
                            switch (mapname)
```

```
                            {
                                case "background":
                                    enemies.Add(new enemy((int)tempposition.X, (int)tempposition.Y, enemydictionary.enemy-
names[spawnenemyrandom.Next(4,6)]));
                                    break;
                                case "cave":
                                    enemies.Add(new enemy((int)tempposition.X, (int)tempposition.Y, enemydictionary.enemy-
names[spawnenemyrandom.Next(6, 8)]));
                                    break;
                                case "town":
                                    enemies.Add(new enemy((int)tempposition.X, (int)tempposition.Y, enemydictionary.enemy-
names[spawnenemyrandom.Next(8, 10)]));
                                    break;
                            }
                        }
                    }
                }
            }
            else
            {
                if (temptimer_enemyspawn >= 5)
                {
                    for (int i = 0; i < 1; i++)
                    {
                        Vector2 tempposition = enemygenration.Dequeue();
                        enemies.Add(new enemy((int)tempposition.X, (int)tempposition.Y, enemydictionary.enemy-
names[spawnenemyrandom.Next(difficulties - 2, difficulties)])); //use dequeue enemy when spawn
                    }
                    temptimer_enemyspawn = 0;
                }
            }
        }


        public bool allowtopass(Rectangle characterspace,Vector2 charactervelocity)//to stop any chracters walk through a
wall
        {                       //current position, the velocity vector, the rectangle of the character space, the current
map object
            for(int i = 0; i < map.rooms.Count; i++)//this is essetially every room in map
            {
                if (charactervelocity.Y == 0)// if player is wallking horizontal, is not going to intersects horizontal walls
```

```
                {
                    foreach(Rectangle wall in map.rooms[i].verticalwalls)
                    { //while player intersecting walls but not intersecting doors
                        while (characterspace.Intersects(wall) && (!characterspace.Intersects(map.rooms[i].door)))//this is
to locate a specific wall in the map player met
                        {
                            Vector2 temp = new Vector2((int)characterspace.X, (int)characterspace.Y) + charactervelocity;
                            Rectangle futureposition = new Rectangle((int)temp.X, (int)temp.Y, characterspace.Width, charac-
terspace.Height);
                            if (futureposition.Intersects(wall))
                            {
                                return false;
                            }
                            else
                            {
                                return true;
                            }
                        }
                    }
                }
                else
                {
                    for(int j = 0; j < 4; j++)
                    {
                        while (characterspace.Intersects(map.rooms[i].walls[j]) && (!characterspace.Inter-
sects(map.rooms[i].door)))//this is to locate a specific wall in the map player met
                        {
                            Vector2 temp = new Vector2((int)characterspace.X, (int)characterspace.Y) + charactervelocity;
                            Rectangle futureposition = new Rectangle((int)temp.X, (int)temp.Y, characterspace.Width, charac-
terspace.Height);
                            if (futureposition.Intersects(map.rooms[i].walls[j]))
                            {
                                return false;
                            }
                            else
                            {
                                return true;
                            }
                        }
                    }
```

```
            }
        }
        return true;
    }

    public bool isitintersectingwalls(Rectangle checkingpace, string walltype) //general function that check for inter-
section of walls
    {
        if(walltype == "verticle")
        {
            foreach(Rooms r in map.rooms)
            {
                for(int i = 0; i < 2; i++)
                {
                    if (checkingpace.Intersects(r.verticalwalls[i]))
                    {
                        return true;
                    }
                }
            }
            return false;
        }
        else if (walltype == "horizontal")
        {
            foreach (Rooms r in map.rooms)
            {
                for (int i = 0; i < 2; i++)
                {
                    if (checkingpace.Intersects(r.horizontalwalls[i]))
                    {
                        return true;
                    }
                }
            }
            return false;
        }
        else if (walltype == "all")
        {
            foreach (Rooms r in map.rooms)
            {
```

```
                    for (int i = 0; i < 4; i++)
                    {
                        if (checkingpace.Intersects(r.walls[i]))
                        {
                            return true;
                        }
                    }
                }
                return false;
            }
            else if(walltype == "rooms")
            {
                foreach(Rooms r in map.rooms)
                {
                    Rectangle temproomspace = new Rectangle((int)r.roompostition.X- 50,(int)r.roompostition.Y -
50,r.roomspace.Width + 100,r.roomspace.Height + 100);
                    if (temproomspace.Intersects(checkingpace))
                    {
                        return true;
                    }
                }
                return false;
            }
            return true;
        }
        void updateenemystats() //increase the health, damage, and coin drop from the enemy
        {
            foreach(var key in enemydictionary.enemystats.Keys.ToList())
            {
                var stats = enemydictionary.enemystats[key];
                stats[0] = Convert.ToString(Convert.ToInt16(stats[0])+2); //HEALTH +2
                stats[1] = Convert.ToString(Convert.ToInt16(stats[1])+1); // DAMAGE +1
                stats[5] = Convert.ToString(Convert.ToInt16(stats[5])+5); // COIN + 5
            }
        }
    }
}
```

Mouse Detection Class

```
using Microsoft.Xna.Framework;
using Microsoft.Xna.Framework.Input;

namespace prototype
{
    internal class mousedetection
    {
        MouseState mousenow;
        MouseState mousebefore;
        public bool leftbuttonpressed = false;
        public Vector2 leftclickedposition;
        public Vector2 mouseactivityupdate()
        {
            mousenow = Mouse.GetState();
            if (mousenow.LeftButton == ButtonState.Pressed)
            {
                leftbuttonpressed = true;
                leftclickedposition = new Vector2(mousenow.X, mousenow.Y);
            }
            else
            {
                leftbuttonpressed = false;
            }
            return leftclickedposition;
        }
        public Vector2 mouseactivityupdate(bool fullpress)
        {
            mousenow = Mouse.GetState();
            if (mousenow.LeftButton == ButtonState.Released && mousebefore.LeftButton == ButtonState.Pressed)
            {
                leftbuttonpressed = true;
                leftclickedposition = new Vector2(mousenow.X, mousenow.Y);
            }
            else
            {
                leftbuttonpressed = false;
            }
            mousebefore = mousenow;
            return leftclickedposition;
```

```
            }
        }
    }
```